Master's Thesis

Evaluating Adaptive Timestep Sampling for the Efficient Training of Flow Matching Models

Computer Vision & Learning Group (CompVis)
Faculty of Mathematics, Computer Science, and Statistics
Ludwig-Maximilians-Universität München

Adrian Leon Scholl

Munich, August 6th, 2026



Submitted in partial fulfillment of the requirements
for the degree of M.Sc. in Data Science

Advisor: M.Sc. Felix Krause
Supervisor: Prof. Dr. Björn Ommer

Declaration of Authorship

I hereby declare that the thesis submitted is my own unaided work. All direct or indirect sources used are acknowledged as references. I am aware that the thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the thesis as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future theses submitted. Further rights of reproduction and usage, however, are not granted here. This thesis was not previously presented to another examination board and has not been published.

Disclosure of AI-based tools

The following generative AI tools were used during the preparation of this thesis:

- **Anthropic Claude (Claude Code)**: used to evaluate the experimental runs, i.e. to aggregate the logged measurements into summary statistics, figures, and tables. All output was reviewed by me and verified against the underlying run logs.
- **Google Gemini**: used for language refinement of text written by me, i.e. spelling, grammar, punctuation, and stylistic consistency. No passages of this thesis were generated by this tool.

No AI tool was used to generate the scientific content of this thesis. The research question, the hypotheses, the experimental design, the selection and interpretation of the results, and the conclusions drawn from them are my own. The cited literature was read and assessed by me. All AI-assisted output was checked and corrected where necessary. I take full responsibility for the content and the correctness of this thesis.

Munich, August 6th, 2026

Adrian Leon Scholl

Abstract

Flow Matching has emerged as a simulation-free successor to diffusion models that preserves sample quality while enabling fewer-step inference on straight Conditional Optimal-Transport (CondOT) paths. As such models scale, training has become increasingly compute-intensive, making the allocation of training iterations across the timestep axis of the conditional flow matching objective a central concern. Uniform timestep sampling is known to be suboptimal, which motivated static non-uniform densities such as the logit-normal prior. These, however, cannot react to the evolving capacity of the network as it learns. The reinforcement-learning-based Adaptive Timestep Sampling (ATS) addresses exactly this deficit for denoising diffusion. This thesis asks whether ATS transfers from discrete ε-prediction diffusion to continuous CondOT velocity regression. We port ATS to Flow Matching, propose a variance-reduced Advantage Actor-Critic variant (A2C-ATS) and a curvature-aware variant (LTE-ATS), and benchmark all three against uniform and static priors on CIFAR-10 under a shared U-Net backbone, analyzing their effect on training efficiency (E1), learning dynamics (E2), and inference geometry (E3). Our comparison suite shows that the advantages of ATS do not transfer to CondOT Flow Matching. Adaptive timestep sampling matches uniform to within run-to-run variation (FID 14.18 versus 13.83) while adding roughly 10% wall-clock training time, because the loss-reduction reward on CondOT paths is minuscule and decays to near zero as the backbone converges. This leaves no stable non-uniform optimum and deprives the A2C critic of the structure it would need to lower variance. We further find the per-timestep loss and gradient-variance profile to be structured but to settle into a stationary U-shape after the first epochs (E2), so that a static prior already captures the exploitable structure effectively. This structure is directional: over-allocating training to the noise-proximal regime proves actively harmful to sample quality (FID up to $+3.51$ relative to uniform), whereas a data-biased prior does not degrade it. The curvature-driven LTE-ATS collapses entirely (FID 395.2), since the marginal curvature its reward targets is unreachable by the conditional objective, consistent with the E3 finding that training-time biases leave the geometry of the learned flow unchanged. Beyond this negative transfer result, the U-shaped, largely stationary account of per-timestep difficulty and the finding that noise-biased sampling degrades sample quality advance the understanding of Flow Matching training dynamics and inform the principled design of future timestep-sampling methods.

Contents

List of Abbreviations **VII**

1 Introduction **1**
1.1 The Generative Modeling Landscape 1
1.2 Timestep Sampling in Flow Matching 3
1.3 Contributions 4

2 Theoretical Foundations **7**
2.1 Generative Modeling with Flow Matching 7
2.1.1 Continuous Normalizing Flows 8
2.1.2 Flow Matching 9
2.1.3 Inference via Numerical ODE Integration 13
2.2 Architectural Developments in Generative Modeling 14
2.2.1 Convolutional Networks and the U-Net Architecture 14
2.2.2 The Attention Mechanism and Vision Transformers 16
2.2.3 Diffusion Transformers 18
2.2.4 The Multimodal Diffusion Transformer 19
2.2.5 Latent Generative Modeling 19
2.3 Reinforcement Learning and Policy Optimization 22
2.3.1 Reinforcement Learning Concepts and Terminology 22
2.3.2 The REINFORCE Algorithm 23

3 Related Work **27**
3.1 Variance Reduction via Optimized Boundary Couplings 27
3.2 Shifting the Optimization Focus across the Generative Trajectory 29
3.2.1 Noise Scheduling 30
3.2.2 Loss Weighting 31
3.2.3 Non-Uniform Timestep Sampling 31

4 Adaptive Timestep Sampling Methods **37**
4.1 Adaptive Timestep Sampling 37
4.1.1 The Adaptive Timestep Sampling Framework 37
4.1.2 Transfer to Flow Matching 43
4.2 Advantage Actor-Critic Adaptive Timestep Sampling 45
4.3 Curvature-Aware Adaptive Timestep Sampling 48

5 Experiments **51**
5.1 Experimental Setup . . . 52
5.1.1 Dataset and Preprocessing . . . 52
5.1.2 Model Architecture . . . 53
5.1.3 Training Configuration and Hyperparameters . . . 55
5.1.4 Evaluation Metrics . . . 56
5.2 E1 – Generation Quality and Training Efficiency . . . 61
5.3 E2 – Insights into the Flow Matching Learning Process . . . 74
5.4 E3 – Impact on Inference Trajectories . . . 82

6 Conclusion **91**
6.1 Summary of Contributions . . . 91
6.2 Summary of Findings . . . 92
6.3 Limitations . . . 94
6.4 Future Work . . . 95

Bibliography **99**

A Appendix **105**
A.1 Discrepancy in ATS Entropy Regularization . . . 105
A.2 Supplementary Experimental Material . . . 111

List of Abbreviations

A2C	Advantage Actor-Critic
A2C-ATS	Advantage Actor-Critic Adaptive Timestep Sampling
A3C	Asynchronous Advantage Actor-Critic
AFM	Augmented Flow Matching
AMP	Automatic Mixed Precision
ATS	Adaptive Timestep Sampling
B-TTDM	Beta-Tuned Timestep Diffusion Model
CFM	Conditional Flow Matching
CIFAR-10	Canadian Institute for Advanced Research, 10-class image dataset
CLIP	Contrastive Language–Image Pre-training
CNF	Continuous Normalizing Flow
CNN	Convolutional Neural Network
CondOT	Conditional Optimal Transport
DDIM	Denoising Diffusion Implicit Model
DDP	Distributed Data Parallel
DDPM	Denoising Diffusion Probabilistic Model
DiT	Diffusion Transformer
DSM	Denoising Score Matching
E1	Experiment 1 — Generation Quality and Training Efficiency
E2	Experiment 2 — Insights into the Flow Matching Learning Process
E3	Experiment 3 — Impact on Inference Trajectories
ELBO	Evidence Lower Bound
EMA	Exponential Moving Average
FID	Fréchet Inception Distance
FLOPs	Floating-Point Operations
FM	Flow Matching
FP16/FP32	16-bit / 32-bit Floating-Point Precision
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
IVP	Initial Value Problem
KL	Kullback–Leibler (divergence)
LDM	Latent Diffusion Model
LLM	Large Language Model
LTE	Local Truncation Error
LTE-ATS	Local-Truncation-Error Adaptive Timestep Sampling
MLP	Multilayer Perceptron
MM-DiT	Multimodal Diffusion Transformer
NFE	Neural Function Evaluations
ODE	Ordinary Differential Equation
OT	Optimal Transport
OT-CFM	Optimal Transport Conditional Flow Matching
P2	Perception-Prioritized (loss weighting)
RL	Reinforcement Learning
SGD	Stochastic Gradient Descent
SNR	Signal-to-Noise Ratio
STF	Stable Target Field
VAE	Variational Autoencoder
VE	Variance Exploding
ViT	Vision Transformer
VP	Variance Preserving

1 Introduction

In recent years, the field of artificial intelligence has witnessed a profound transformation driven by rapid developments in generative modeling. Unlike traditional predictive systems that primarily map inputs to labels or decisions, generative AI models seek to learn the underlying probability distribution of complex datasets, enabling the synthesis of novel, high-fidelity samples. This capability has expanded the scope of artificial intelligence beyond prediction toward tasks involving content creation, simulation, and open-ended generation, unlocking a wide range of applications across diverse domains. Today, generative systems have evolved far beyond academic research into widely adopted tools for learning, work, creative assistance, and scientific discovery. While Large Language Models (LLMs) have largely driven public adoption in natural language processing, parallel advances have emerged in high-dimensional visual, temporal, and scientific domains. These developments have driven major breakthroughs, including text-to-image generation models like Stable Diffusion 3 (Esser et al., 2024), video generation systems such as OpenAI's Sora (Brooks et al., 2024), and scientific frameworks like AlphaFold 3 (Abramson et al., 2024) for biomolecular structure prediction.

1.1 The Generative Modeling Landscape

Conceptually, generative modeling in the domain of image and video creation can be framed as learning a continuous mapping that transforms completely unstructured, random noise into highly structured and meaningful data. To solve this generation problem, a succession of increasingly powerful generative paradigms has emerged, each introducing distinct trade-offs between sample quality, training stability, inference efficiency, and likelihood tractability.

Variational Autoencoders (VAEs) (Kingma and Welling, 2014) introduced a principled probabilistic framework for generative modeling by combining deep autoencoder networks with variational inference. They optimize a variational lower bound (ELBO) on the data likelihood, thereby enabling tractable approximate maximum likelihood learning in high-dimensional latent variable models. By encoding data into a continuous latent space and sampling from a learned prior, VAEs allow for efficient and structured generation. However, the reliance on a pixel-wise reconstruction objective and the variational relaxation often leads to overly smooth latent representations and consequently blurred samples.

Subsequently, *Generative Adversarial Networks (GANs)* (Goodfellow et al., 2014) circumvented explicit likelihood estimation entirely by formulating generation as a minimax game between two competing neural networks: a generator synthesizing data and a discriminator distinguishing

synthetic samples from the true distribution. By relying on this adversarial loss rather than pixel-wise error bounds, GANs achieved unprecedented perceptual sharpness. However, maintaining the delicate dynamic equilibrium between these networks introduced severe training instabilities and a high susceptibility to mode collapse, severely limiting sample diversity.

More recently, the field has become dominated by iterative denoising approaches, conceptually rooted in non-equilibrium thermodynamics (Sohl-Dickstein et al., 2015) and subsequently formalized as Denoising Diffusion Probabilistic Models (DDPMs) (Ho et al., 2020). These models define a fixed, Markovian forward diffusion process that progressively corrupts the target data distribution into a tractable isotropic Gaussian prior over a sequence of discrete timesteps. Generative sampling is achieved by training a parameterized neural network to approximate the reverse Markov chain, iteratively removing the injected noise. The corresponding optimization objective is derived from a variational lower bound, which can be simplified into a noise-prediction task. Crucially, this surrogate objective establishes a strict mathematical equivalence to *Denoising Score Matching (DSM)* (Ho et al., 2020), demonstrating that the neural network implicitly estimates the time-dependent score function—the spatial gradient of the log-probability density—of the perturbed data distributions.

While DDPMs achieve exceptional synthesis quality, their strict reliance on a stochastic Markovian reverse process imposes severe inference latencies. To alleviate this computational bottleneck, *Denoising Diffusion Implicit Models (DDIMs)* (Song et al., 2021a) introduce a generalized class of non-Markovian forward processes that preserve the identical marginal probability densities of the DDPM formulation. This generalization facilitates a completely deterministic generative process, effectively modeling the generation trajectory via a probability flow Ordinary Differential Equation (ODE). By eliminating stochastic noise injection during sampling, DDIMs decouple the generative path from Markovian constraints and permit substantially larger integration step sizes, thereby significantly accelerating the sampling process. Despite the computational acceleration provided by DDIMs, the fundamental geometry of the probability paths dictated by standard diffusion formulations remains highly curved. High trajectory curvature imposes a strict upper bound on the integration step size, as numerical ODE solvers inherently accumulate severe truncation errors when traversing curved vector fields with large discrete steps (Karras et al., 2022; Liu et al., 2023). Consequently, generating high-fidelity samples still necessitates dozens to hundreds of sequential neural function evaluations.

To fundamentally overcome this geometric bottleneck, *Continuous Normalizing Flows (CNFs)* have regained significant attention, particularly through the simulation-free training paradigm of *Flow Matching (FM)* (Lipman et al., 2023). By abstracting the generative process from the specific Gaussian corruption schedules of diffusion models, these frameworks learn deterministic vector fields that parameterize an arbitrary probability flow. In contrast to classical likelihood-based CNF training, Flow Matching reformulates the problem as the direct regression of vector fields along predefined probability paths, completely avoiding expensive ODE simulation during training. Crucially, this framework permits the explicit design of the interpolation trajectories between noise and data. When using Conditional Optimal Transport (CondOT) displacement interpolants, the resulting couplings induce more direct transport paths, which in turn were shown to reduce the curvature of the induced probability flow (Lipman et al., 2023). This geometric regularity mitigates numerical integration errors during sampling and forms the foundation for advanced trajectory-straightening algorithms. Most notably, the *Rectified Flow*

framework introduces an iterative *Reflow* procedure that recursively straightens the simulated ODE trajectories (Liu et al., 2023). By progressively linearizing the flow and subsequently distilling the mapping into a single network evaluation, this methodology effectively bridges the gap between continuous-time dynamical systems and highly efficient one-step generative models.

1.2 Timestep Sampling in Flow Matching

Despite these improvements in inference speed, training Flow Matching models remains exceptionally compute-intensive. For instance, scaling the Stable Diffusion 3 architecture (Esser et al., 2024) to 8 billion parameters necessitates up to 5×10^{22} floating-point operations (FLOPs) for training convergence, while continuous-time video generation frameworks, such as Meta Movie Gen, even scale to capacities of up to 30 billion parameters (Polyak et al., 2024). To mitigate this growing computational demand, it is imperative to efficiently allocate resources during Flow Matching training.

A key observation is that the optimization difficulty of training the Flow Matching target fluctuates drastically across the temporal domain. Depending on the specific signal-to-noise ratio (SNR) at a given timestep t, the neural network encounters highly non-stationary learning dynamics: certain temporal regimes present trivial interpolation tasks, whereas others are characterized by high structural complexity and high gradient variance (Kim et al., 2025a).

In classical diffusion models, allocating training compute across these noise regimes relies on three coupled mechanisms: the noise schedule, loss weighting, and the timestep sampling distribution (Karras et al., 2022). However, altering the noise schedule to shift the SNR focus inherently curves the probability path, degrading inference efficiency. Flow Matching effectively decouples these mechanisms. To preserve fast inference, modern frameworks predominantly adopt a straight CondOT path, thereby locking the geometric noise schedule (Liu et al., 2023). Concurrently, introducing an explicit time-dependent loss weight $w(t)$ is mathematically equivalent to altering the timestep sampling distribution $p(t)$ (Esser et al., 2024). Consequently, adjusting the sampling distribution functions as an implicit loss re-weighting mechanism. By explicitly biasing the expectation in the Conditional Flow Matching (CFM) objective, $p(t)$ concentrates training iterations on specific, more complex noise levels without compromising the straight-path geometry.

The original Flow Matching formulation samples timesteps uniformly, $t \sim \mathcal{U}(0, 1)$, potentially leading to inefficient compute allocation (Lipman et al., 2023). To address this, subsequent research applied static, log-normal sampling distributions—initially established for diffusion (Karras et al., 2022) and later adapted for large-scale Rectified Flows (Esser et al., 2024)—which significantly improved convergence and sample quality. However, static distributions ignore the evolving capacity of the neural network: the conditional losses across different timesteps are interdependent and fluctuate as the model learns (Kim et al., 2025a). Thus, Kim et al. (2025a) proposed Adaptive Timestep Sampling (ATS), an online reinforcement learning-based algorithm that dynamically shifts the sampling distribution to prioritize the most critical timesteps in real time. Adapting and evaluating this dynamic sampling mechanism for the continuous-time domain of CondOT Flow Matching constitutes the central objective of this thesis.

1.3 Contributions

While ATS provides a robust theoretical foundation for dynamic resource allocation in diffusion models, its direct transposition to the deterministic ODE framework of Flow Matching remains largely unexplored. Furthermore, the underlying reinforcement learning optimization exhibits high variance during policy updates, and its default reward signal is strictly training-focused, neglecting its potential implications on the geometric curvature of the resulting sampling trajectories. This thesis addresses these limitations by formalizing, extending, and evaluating dynamic timestep sampling for CondOT Flow Matching architectures. Its main contributions are the following:

- **Adaptation and evaluation of ATS for CondOT Flow Matching (E1).** We port ATS (Kim et al., 2025a) to continuous-time Flow Matching and benchmark it against uniform and static priors. We demonstrate that oversampling the noise regime actively degrades final sample quality. Furthermore, the adaptive policy quickly reverts to uniform, failing to offset its $\approx 10\%$ compute overhead, whereas a simple, noise-avoiding static prior achieves comparable quality at no additional cost.
- **A variance-reduction actor-critic variant, A2C-ATS (E1).** We identify the high variance of the ATS policy updates and introduce an Advantage Actor-Critic variant that subtracts a learned value-function baseline from the reward. In the CondOT setting the baseline does not reduce the policy-gradient variance and in fact mildly increases it, because the near-zero and highly non-stationary loss-reduction reward leaves too little structure for the critic to track.
- **A curvature-aware reward, LTE-ATS, and its failure mode (E1).** We formulate a novel, inference-focused reward from the Euler local truncation error, intended to bias training toward the high-curvature timesteps where an ODE solver struggles most. We show that naively rewarding curvature collapses the sampling policy onto the data-proximal timesteps and starves the rest of the trajectory rather than straightening it, because that curvature is a property of the marginal field that conditional training at a single timestep cannot reduce.
- **A characterization of the CondOT per-timestep learning landscape (E2).** We measure the per-timestep loss and gradient-variance landscapes of the objective and find both non-uniform and stationary after the first few epochs, with the loss largest at the data boundary and the gradient variance largest at the noise boundary. We attribute the two arms of the loss profile, and their asymmetry, to the irreducible variance of the Flow Matching target and show that biasing the sampler leaves these landscapes essentially unchanged, altering only the share of compute each timestep regime receives.
- **An inference-side geometric analysis of the sampling bias (E3).** We show that the training timestep distribution leaves no meaningful imprint on converged inference geometry. Despite massive variations in training density, trajectory curvature and truncation error very nearly collapse onto a shared profile. Consequently, reducing the Euler integration budget penalizes sample quality identically across all models. We conclude that timestep priors solely reallocate training updates, offering no leverage to produce straighter or cheaper-to-integrate flows.

The remainder of this thesis is structured as follows. Chapter 2 establishes the mathematical foundations of Flow Matching, reviews the backbone architectures that parameterize its vector field and their scaling behavior, and formalizes the reinforcement-learning policy optimization on which ATS is built. Chapter 3 reviews the historical strategies employed to manage training focus, including noise schedules, loss weighting, and static sampling distributions. Chapter 4 presents the core methodology, detailing the adaptation of ATS to Flow Matching and the rigorous mathematical formulation of both the A2C-ATS and LTE-ATS algorithms. Chapter 5 details the empirical experiments, evaluating generation quality and training efficiency (E1), the per-timestep learning landscape (E2), and inference-trajectory geometry (E3). Finally, Chapter 6 summarizes the contributions and discusses avenues for future research.

2 Theoretical Foundations

This chapter establishes the theoretical foundations underlying generative modeling with Flow Matching and adaptive policy optimization through reinforcement learning. In Section 2.1, we trace the evolution of the Flow Matching framework from continuous normalizing flows to the scalable, simulation-free CFM objective. Subsequently, we detail the mechanics of generative inference via numerical ODE integration. Section 2.2 then reviews the backbone architectures that parameterize this vector field, tracing the shift from convolutional U-Nets to Transformer-based models and the latent-generative modeling. Finally, Section 2.3 presents the fundamental concepts of reinforcement learning, with a particular focus on policy optimization via the REINFORCE algorithm, establishing the mathematical framework for the adaptive timestep sampling methods central to this thesis.

2.1 Generative Modeling with Flow Matching

Generative modeling fundamentally aims to estimate and sample from complex, unknown empirical data distributions. Recently, continuous-time generative frameworks, particularly diffusion-based models (Ho et al., 2020; Song et al., 2021b; Song et al., 2021a), have driven unprecedented advances in high-dimensional data synthesis. These models systematically corrupt data into noise via a stochastic forward process and learn to reverse this trajectory for generation. However, the strict reliance on standard diffusion processes inherently restricts the generative dynamics to a confined space of highly curved probability paths. This trajectory curvature induces a high number of sequential integration steps during inference, rendering the generative process computationally burdensome (Lipman et al., 2023; Liu et al., 2023).

To transcend the structural limitations of diffusion models, *Continuous Normalizing Flows* offer a generalized, deterministic framework capable of modeling arbitrary probability paths (Lipman et al., 2023). Historically, optimizing CNFs relied on maximum likelihood estimation, which strictly requires the execution of computationally prohibitive numerical ODE solvers during both the forward and backward passes. This simulation-heavy requirement severely hindered the scalability of CNFs to high-dimensional domains, like image or video generation. *Flow Matching* solves this computational bottleneck by proposing a simulation-free training paradigm that completely bypasses exact likelihood estimation. By directly regressing vector fields against fixed, predefined probability paths, FM enables the scalable and tractable optimization of CNFs, effectively unifying and generalizing continuous-time generative models under a single, highly efficient framework.

2.1.1 Continuous Normalizing Flows

Continuous Normalizing Flows are deterministic generative models that transform a simple, tractable prior distribution $p_0(x_0)$, typically an isotropic Gaussian, into a complex target data distribution $q(x_1)$ through a continuous-time dynamical system (Chen et al., 2018).

The Vector Field and ODE

This transformation is governed by an ordinary differential equation. The continuous trajectory of a spatial state $x_t \in \mathbb{R}^d$ over time $t \in [0, 1]$ is defined by the initial value problem

$$\frac{d}{dt}x_t = v_t(x_t, t), \quad x_0 \sim p_0(x_0). \tag{2.1}$$

Here, $v_t : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d, (x_t, t) \mapsto v_t(x_t, t)$ is a time-dependent spatial vector field, which takes a spatial state and a scalar time as inputs and outputs the instantaneous velocity vector of that state. In deep generative modeling, this vector field is typically parameterized by a neural network (Lipman et al., 2023; Chen et al., 2018). Because standard neural architectures inherently employ finite weights and Lipschitz-bounded activation functions, the resulting vector field is intrinsically Lipschitz continuous (Holderrieth and Erives, 2025; Chen et al., 2018). Consequently, the Picard–Lindelöf theorem guarantees that the solution to the corresponding ODE exists and is strictly unique. This analytical regularity ensures that the generative transformation smoothly warps the state space without the microscopic trajectories ever intersecting.

The Flow and Probability Path

Integrating the vector field over time yields the *flow*, denoted as $\phi_t : \mathbb{R}^d \to \mathbb{R}^d$. The flow is a time-dependent diffeomorphism that deterministically maps an initial state x_0 to its new position at time t, given by

$$\phi_t(x_0) = x_0 + \int_0^t v_s(\phi_s(x_0), s)ds. \tag{2.2}$$

Applied to the entire initial probability distribution, the flow deterministically transports the global probability mass, yielding an evolving probability path. This time-dependent marginal probability density path is denoted as $p_t(x_t)$. The strict mathematical relationship between the initial prior p_0 and this evolving density path is defined via the push-forward operator, $p_t = [\phi_t]_* p_0$, which is explicitly evaluated using the change-of-variables formula (Lipman et al., 2023; Chen et al., 2018),

$$p_t(x_t) = p_0(\phi_t^{-1}(x_t)) \det \left(\frac{\partial \phi_t^{-1}(x_t)}{\partial x_t} \right). \tag{2.3}$$

This operator enforces absolute mass conservation. It evaluates the prior density along the inverse flow trajectory and systematically scales it by the Jacobian determinant, thereby geometrically accounting for any local spatial expansion or compression induced by the vector field.

The Continuity Equation

While the push-forward operator captures the global trajectory, the absolute conservation of mass can also be defined instantaneously. To guarantee that the density remains a valid probability distribution ($\int p_t(x_t)dx_t = 1$) at all times, any valid probability path $p_t(x_t)$ and its driving spatial vector field $v_t(x_t, t)$ must strictly satisfy the continuity equation (Holderrieth and Erives, 2025),

$$\frac{\partial}{\partial t}p_t(x_t) + \nabla \cdot \big(p_t(x_t)v_t(x_t, t)\big) = 0, \tag{2.4}$$

where the temporal rate of change in local density ($\frac{\partial}{\partial t}p_t$) is perfectly balanced by the spatial divergence of the probability current ($\nabla\cdot(p_t v_t)$), geometrically measuring the physical net outflow of probability mass from an infinitesimally small volume around x_t. This relationship ensures that no mass is artificially created or destroyed during the spatial transformation. Because deriving a target vector field that fulfills this differential constraint for complex data distributions is generally intractable, Flow Matching frameworks are designed to construct tractable conditional vector fields that are guaranteed to satisfy this exact equation (Lipman et al., 2023).

2.1.2 Flow Matching

Traditionally, CNFs are optimized via maximum likelihood estimation. However, directly maximizing the exact data likelihood via the push-forward operator, defined in Section 2.1.1, is computationally prohibitive. Evaluating this operator requires computing the inverse flow $\phi_t^{-1}(x_1)$ to explicitly trace a data sample back to its origin in the prior, alongside calculating its Jacobian determinant. This inherently necessitates executing costly numerical ODE simulations to integrate the neural vector field backward in time during every single training step.

To overcome this computational barrier, Lipman et al. (2023) introduced *Flow Matching*, a scalable, simulation-free training paradigm for CNFs. Instead of calculating exact likelihoods, this framework explicitly defines a mathematically tractable true target marginal vector field $u_t(x_t)$ that generates the target data distribution. It then directly regresses a parameterized neural vector field $v_t(x_t; \theta)$ against this ground-truth target. Consequently, the standard marginal Flow Matching objective is defined as the mean squared error loss

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,x_t}\left[\|v_t(x_t;\theta) - u_t(x_t)\|^2\right], \tag{2.5}$$

where θ denotes the learnable network parameters, time $t \sim \mathcal{U}(0, 1)$ is sampled uniformly, and the spatial state x_t is sampled directly from the marginal probability path $p_t(x_t)$.

The Marginalization Trick

While the fundamental objective of Flow Matching provides a clear optimization target, the framework itself is initially underspecified. The boundary conditions are strictly predefined: the continuous-time generative process must initialize from a tractable noise prior $p_0(x)$ and terminate at a distribution $p_1(x)$ that closely approximates the empirical data distribution $q(x_1)$. However, there exist infinitely many valid probability paths $p_t(x_t)$ and corresponding target vector fields $u_t(x_t)$ that successfully connect these two boundary distributions (Lipman et al., 2023). Consequently, the specific probability path p_t is left as a critical design choice.

Yet, even when a theoretical target path p_t is chosen, the exact marginal vector field $u_t(x_t)$ required to generate that global path cannot be tractably calculated in closed form for complex, high-dimensional datasets. To bypass this intractability, the marginalization trick is applied. Instead of modeling the global data distribution directly, this methodology constructs a simple, analytically tractable, per-sample conditional probability path $p_t(x_t|x_1)$ that transports a noise particle to a single, specific data point $x_1 \sim q(x_1)$. The desired global marginal probability path is subsequently recovered by aggregating these individual conditional paths over the entire empirical data distribution to obtain

$$p_t(x_t) = \int p_t(x_t|x_1)q(x_1)dx_1. \tag{2.6}$$

Correspondingly, the marginal target velocity field required to generate this global path is constructed by taking the expectation of the individual conditional velocities. Here, each conditional vector field $u_t(x_t|x_1)$ is weighted by the posterior probability $q(x_1|x_t)$ that a specific data point x_1 generated the currently observed noisy interpolant x_t (Holderrieth and Erives, 2025). Expanding this posterior via Bayes' rule as $q(x_1|x_t) = \frac{p_t(x_t|x_1)q(x_1)}{p_t(x_t)}$ results in the marginal vector field (Lipman et al., 2023)

$$u_t(x_t) = \int u_t(x_t|x_1)\frac{p_t(x_t|x_1)q(x_1)}{p_t(x_t)}dx_1. \tag{2.7}$$

Lipman et al. (2023) formally establish that if the conditional vector fields $u_t(x_t|x_1)$ generate the conditional probability paths $p_t(x_t|x_1)$, then the aggregated marginal vector field $u_t(x_t)$ strictly generates the true marginal probability path $p_t(x_t)$ for any arbitrary data distribution $q(x_1)$. This fundamental guarantee ensures that complex, intractable global dynamics can be mathematically broken down into a mixture of highly tractable, sample-wise conditional vector fields.

Conditional Flow Matching

Despite the theoretical elegance of the marginalization, computing the marginal probability path $p_t(x_t)$ and marginal vector field $u_t(x_t)$ remains computationally prohibitive. Evaluating the integrals in Equations (2.7) and (2.6) requires computing expectations over the entire high-dimensional data space $q(x_1)$ at every training step. To circumvent this integration, Lipman et al. (2023) replace the intractable marginal target by directly regressing on the conditional vector field $u_t(x_t|x_1)$, yielding the CFM objective

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U},x_1\sim q(x_1),x_t\sim p_t(x_t|x_1)}\|v_t(x_t;\theta) - u_t(x_t|x_1)\|^2. \tag{2.8}$$

Crucially, Lipman et al. (2023) analytically prove that this conditional objective and the original marginal Flow Matching objective are equivalent up to a constant, possessing identical gradients with respect to the network parameters θ. Because the objectives are defined as expectations, evaluating $\mathcal{L}_{\text{CFM}}$ on empirical mini-batches yields an unbiased Monte Carlo estimate of the intractable marginal loss gradient. By following these unbiased gradient updates, the neural network is mathematically guaranteed to converge to the true marginal vector field, enabling highly efficient, simulation-free training.

Theorem: Flow Matching Objective Equivalence (Lipman et al., 2023)

For any valid conditional vector field $u_t(x_t|x_1)$, the gradients of the marginal and conditional Flow Matching objectives with respect to the network parameters θ are mathematically identical:

$$\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta). \tag{2.9}$$

Consequently, computing the gradient of the tractable conditional objective on empirical mini-batches provides an **unbiased estimate** of the gradient for the intractable marginal objective, guaranteeing that the network converges to the true marginal vector field.

Proof after Holderrieth and Erives (2025). The equivalence is proven by expanding the mean-squared error into three components and removing constants with respect to the network parameters θ. First, we expand the marginal Flow Matching objective by separating the squared terms to obtain

$$\begin{aligned}
\mathcal{L}_{\text{FM}}(\theta) &= \mathbb{E}_{t\sim\mathcal{U},x_t\sim p_t(x_t)}\left[\|v_t(x_t;\theta) - u_t(x_t)\|^2\right] && \text{(Definition Eq. (2.5))} \\
&= \mathbb{E}_{t\sim\mathcal{U},x_t\sim p_t(x_t)}\left[\|v_t(x_t;\theta)\|^2 - 2\langle v_t(x_t;\theta), u_t(x_t)\rangle\right] + C_1, && \text{(Expand square)}
\end{aligned} \tag{2.10}$$

where $C_1 = \mathbb{E}_{t\sim\mathcal{U},x_t\sim p_t(x_t)}[\|u_t(x_t)\|^2]$ is a constant independent of θ. Because the marginal density is defined as $p_t(x_t) = \int p_t(x_t|x_1)q(x_1)dx_1$, we can re-express the expected squared network norm over the joint distribution as

$$\mathbb{E}_{t\sim\mathcal{U},x_t\sim p_t(x_t)}\left[\|v_t(x_t;\theta)\|^2\right] = \mathbb{E}_{t\sim\mathcal{U},x_1\sim q(x_1),x_t\sim p_t(x_t|x_1)}\left[\|v_t(x_t;\theta)\|^2\right]. \tag{2.11}$$

Next, we rewrite the cross-term by expressing the expectation as an integral, which allows us to substitute the definition of the marginal vector field (Equation (2.7)) and sequentially transform the expression into a conditional expectation given by

$$\begin{aligned}
&\mathbb{E}_{t\sim\mathcal{U},x_t\sim p_t(x_t)}\left[\langle v_t(x_t;\theta), u_t(x_t)\rangle\right] \\
&= \int\int \langle v_t(x_t;\theta), u_t(x_t)\rangle p_t(x_t)dx_t dt && \text{(Integral expansion)} \\
&= \int\int \left\langle v_t(x_t;\theta), \int u_t(x_t|x_1)p_t(x_t|x_1)q(x_1)dx_1 \right\rangle dx_t dt && \text{(Substitute Eq. (2.7))} \\
&= \int\int\int \langle v_t(x_t;\theta), u_t(x_t|x_1)\rangle p_t(x_t|x_1)q(x_1)dx_1 dx_t dt && \text{(Linearity of integrals)} \\
&= \mathbb{E}_{t\sim\mathcal{U},x_1\sim q(x_1),x_t\sim p_t(x_t|x_1)}\left[\langle v_t(x_t;\theta), u_t(x_t|x_1)\rangle\right]. && \text{(Convert to expectation)}
\end{aligned} \tag{2.12}$$

This is the crucial step of the proof since it translates the expectation over the intractable marginal vector field into an expectation over the tractable conditional vector field. Plugging these derived components back into the expanded form of $\mathcal{L}_{\text{FM}}$ allows us to complete the square and establish

the final equivalence as

$$\begin{aligned}
\mathcal{L}_{\mathrm{FM}}(\theta) &= \mathbb{E}_{t,x_1,x_t}\left[\|v_t(x_t;\theta)\|^2 - 2\langle v_t(x_t;\theta), u_t(x_t|x_1)\rangle\right] + C_1 && \text{(Substitute derivations)} \\
&= \mathbb{E}_{t,x_1,x_t}\left[\|v_t(x_t;\theta)\|^2 - 2\langle v_t(x_t;\theta), u_t(x_t|x_1)\rangle\right. \\
&\quad \left. + \|u_t(x_t|x_1)\|^2 - \|u_t(x_t|x_1)\|^2\right] + C_1 && \text{(Add/subtract target norm)} \\
&= \mathbb{E}_{t,x_1,x_t}\left[\|v_t(x_t;\theta) - u_t(x_t|x_1)\|^2\right] + C_2 + C_1 && \text{(Complete the square)} \\
&= \mathcal{L}_{\mathrm{CFM}}(\theta) + C, && \text{(Define } C = C_1 + C_2\text{)}
\end{aligned} \tag{2.13}$$

where $C_2 = -\mathbb{E}_{t,x_1,x_t}[\|u_t(x_t|x_1)\|^2]$ and $C = C_1 + C_2$. Because $\mathcal{L}_{\mathrm{FM}}(\theta)$ and $\mathcal{L}_{\mathrm{CFM}}(\theta)$ differ only by a constant independent of θ, their gradients are strictly identical. □

Choice of Conditional Probability Path and Vector Field

Because the CFM framework is entirely agnostic to the specific geometry of the conditional distribution, the exact shape of the conditional target trajectory acts as a primary design choice. The most universally applied formulation is the family of Gaussian conditional probability paths, governed by a time-dependent mean scaling schedule α_t and standard deviation schedule β_t. These noise schedules are defined as continuously differentiable, monotonic scalar functions mapping into $\mathbb{R}$ that must satisfy the boundary conditions $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. This constraint guarantees that the distribution seamlessly interpolates from a standard normal prior at $t = 0$ to the clean data target at $t = 1$. This class of paths is analytically expressed as

$$p_t(x_t|x_1) = \mathcal{N}(x_t; \alpha_t x_1, \beta_t^2 I). \tag{2.14}$$

By restricting the spatial mapping to an affine transformation, the unique conditional vector field that deterministically generates this Gaussian path can be analytically derived in closed form. This velocity profile relies purely on the logarithmic time derivatives of the specified noise schedules (Holderrieth and Erives, 2025), yielding

$$u_t(x_t|x_1) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) x_1 + \frac{\dot{\beta}_t}{\beta_t} x_t. \tag{2.15}$$

While this generalized Gaussian framework naturally subsumes the highly curved trajectories of variance preserving (VP) (Ho et al., 2020) and variance exploding (VE) (Song et al., 2021b) diffusion paths, the CondOT path has emerged as the structural standard in generative modeling with Flow Matching.

The CondOT formulation explicitly parametrizes the interpolation to progress at a constant temporal speed by setting $\alpha_t = t$ and $\beta_t = 1 - t$. Geometrically, this configuration constitutes the true Optimal Transport displacement map between the conditional prior and the data point, explicitly minimizing the conditional kinetic energy of the generative system (Liu et al., 2023). This deterministic structure guarantees that individual conditional trajectories follow perfect, non-intersecting straight lines pointing directly from the sampled noise x_0 to the target x_1.

Although the aggregated marginal trajectory is not perfectly straight due to the complex superposition of these localized fields, it remains substantially less curved than traditional score-based

diffusion paths. This reduced spatial curvature significantly simplifies the neural regression task and enables low-error synthesis utilizing exceedingly few numerical integration steps (Lipman et al., 2023). Finally, substituting the linear CondOT schedules into the general CFM formulation elegantly collapses the training objective to regress directly against the constant velocity vector $(x_1 - x_0)$, simplifying the empirical loss to

$$\mathcal{L}_{\text{CondOT}}(\theta) = \mathbb{E}_{t\sim\mathcal{U},x_1\sim q(x_1),x_0\sim\mathcal{N}(0,I)}\|v_t(tx_1 + (1-t)x_0;\theta) - (x_1 - x_0)\|^2. \tag{2.16}$$

Flow models employing these CondOT probability paths are now frequently, albeit somewhat imprecisely, referred to as Rectified Flow models (Liu et al., 2023; Esser et al., 2024).

2.1.3 Inference via Numerical ODE Integration

Once the neural vector field $v_t(x_t;\theta)$ has been learned via the CFM objective, data generation is formulated as an Initial Value Problem (IVP). Starting from a noise sample $x_0 \sim \mathcal{N}(0, I)$, the model synthesizes a data sample x_1 by integrating the learned ODE from $t = 0$ to $t = 1$. Since the resulting differential equation generally admits no closed-form solution, the continuous trajectory must be approximated numerically using an ODE solver. During this process, the temporal domain $[0, 1]$ is traversed through a finite sequence of timesteps, at which the vector field is evaluated to iteratively update the current state.

The resolution of this temporal discretization is strictly governed by the hyperparameter N, denoting the total number of integration steps, which analytically defines the discrete step size $\Delta t = 1/N$. This hyperparameter N functions as the direct computational control mechanism over the integration precision; it dictates the magnitude of the local truncation error introduced at each discrete step, which systematically accumulates over the integration sequence to form the global truncation error, thereby strictly bounding the final output quality of the generated samples (Karras et al., 2022).

Beginning from pure noise, the integration transports the sample along the learned probability flow toward regions of high density under the target data distribution.

First-Order Integration: The Euler Method

The most fundamental numerical integration scheme is the first-order Euler method (Iserles, 2009). Given a discrete timestep Δt, the state update relies on a simple, first-order approximation that evaluates the instantaneous velocity strictly at the current state and extrapolates it linearly across the entire interval, yielding

$$x_{t+\Delta t} = x_t + \Delta t \cdot v_t(x_t;\theta). \tag{2.17}$$

While this scheme is computationally highly efficient, requiring exactly one neural network evaluation per integration step, its linear approximation mechanism introduces fundamental numerical limitations. If the true marginal probability path exhibits significant geometric curvature, the linear projection intrinsically diverges from the exact continuous-time trajectory, inducing a local truncation error of $\mathcal{O}(\Delta t^2) = \mathcal{O}(\frac{1}{N^2})$ at each discrete step (Karras et al., 2022). As these local misalignments accumulate over the integration process, the resulting global truncation error strictly degrades the final output quality (Karras et al., 2022). This degradation

becomes particularly severe when employing a small number of integration steps on generative trajectories that are inherently curved, such as the classical probability paths constructed by standard diffusion models (Lipman et al., 2023; Liu et al., 2023).

Second-Order Integration: The Heun Method

To mitigate the structural errors induced by path curvature, higher-order numerical ODE solvers, such as the Heun method (a second-order Runge–Kutta variant), are frequently employed. The Heun method operates via a predictor-corrector mechanism. It first computes a preliminary first-order Euler extrapolation to estimate the future state. Subsequently, the solver evaluates the neural vector field at this estimated future state and averages the initial and future velocities to compute the final, curvature-corrected update (Iserles, 2009):

$$\tilde{x}_{t+\Delta t} = x_t + \Delta t \cdot v_t(x_t; \theta) \quad \text{(Predictor Step)} \tag{2.18}$$

$$x_{t+\Delta t} = x_t + \frac{\Delta t}{2}\Big(v_t(x_t; \theta) + v_{t+\Delta t}(\tilde{x}_{t+\Delta t}; \theta)\Big) \quad \text{(Corrector Step).} \tag{2.19}$$

While this integration scheme strictly doubles the computational cost by requiring two neural network evaluations per step, it systematically suppresses the local truncation error to $\mathcal{O}(\Delta t^3) = \mathcal{O}(\frac{1}{N^3})$. This reduction provides a significantly superior tradeoff between computational overhead and numerical precision specifically for the integration of highly curved generative trajectories, such as those inherent to standard diffusion models (Karras et al., 2022).

2.2 Architectural Developments in Generative Modeling

The Conditional Flow Matching objective of Section 2.1.2 reduces generative modeling to a supervised regression problem, in which a single neural network $v_t(x_t; \theta)$ must approximate the marginal vector field across all noise levels $t \in [0, 1]$ and all spatial states x_t. The framework itself, however, is entirely agnostic to how this network is parameterized. It is the backbone architecture that determines which inductive biases constrain the regression, how expensive each network evaluation is, and, most consequentially, how reliably additional parameters and training compute translate into improved generative fidelity. Architecturally, the field has undergone a fundamental transition, moving from convolutional U-Nets, whose strong spatial priors enable sample-efficient learning at moderate scale (Ronneberger et al., 2015; Ho et al., 2020), toward attention-based Transformers with vastly superior and highly predictable scaling behavior (Vaswani et al., 2017; Peebles and Xie, 2023; Esser et al., 2024). This section traces that transition and its computational consequences, which directly motivate the training-efficiency perspective of this thesis.

2.2.1 Convolutional Networks and the U-Net Architecture

The convolutional neural network (CNN) constitutes the architectural foundation of modern computer vision (LeCun et al., 1989; LeCun et al., 1998). Rather than densely connecting every input pixel to every neuron, a convolutional layer learns a small set of filter kernels that are slid across the image, computing local weighted sums at every spatial position. This design

hard-codes two structural assumptions directly into the network topology. The first is *locality*: informative correlations in natural images are concentrated within spatially adjacent pixels, so each unit observes only a small receptive field. The second is *translation equivariance*: because identical kernel weights are shared across all spatial positions, a feature detector learned at one location automatically applies to every other location. LeCun et al. (1989) demonstrated that constraining the network in this way drastically reduces the number of free parameters while improving generalization. Complementary *pooling* (subsampling) operations aggregate neighboring activations (typically via averaging or maximization) and progressively reduce the spatial resolution. This pooling hierarchy yields exponentially growing effective receptive fields, approximate invariance to small local translations, and a feature hierarchy that composes edges into motifs, motifs into parts, and parts into objects (LeCun et al., 1998).

Because the vector field regression is inherently an image-to-image task, in which the network consumes a noisy state $x_t \in \mathbb{R}^{C\times H\times W}$ and must emit a velocity estimate of identical dimensionality, early diffusion models (Ho et al., 2020) adopted the U-Net (Ronneberger et al., 2015), a fully convolutional encoder–decoder architecture originally developed for biomedical image segmentation. The U-Net consists of a contracting and an expansive path. The contracting encoder alternates convolutional blocks with downsampling operations, successively halving the spatial resolution while increasing the channel depth, thereby distilling the input into a coarse but semantically rich bottleneck representation. The expansive decoder mirrors this structure with upsampling operations, restoring the original resolution. Crucially, *skip connections* concatenate each encoder feature map onto the decoder activations of matching resolution. These lateral shortcuts allow high-frequency spatial detail to bypass the lossy bottleneck entirely, so the decoder can fuse global semantic context with precisely localized detail, enabling the precise prediction of per-pixel velocities.

Diffusion and flow models extend this template in two ways. First, the network must represent an entire continuum of denoising tasks, so the scalar time t is injected as a global conditioning signal. The timestep is mapped to a sinusoidal positional embedding (Vaswani et al., 2017), transformed by a small MLP, and added to the intermediate activations of every residual block (Ho et al., 2020). Second, purely convolutional stacks proved insufficient for globally coherent synthesis, and practitioners progressively complemented them with global self-attention. Ho et al. (2020) inserted a single self-attention layer at the 16×16 feature resolution, and Dhariwal and Nichol (2021) subsequently found this single-resolution, single-head design suboptimal, placing multi-head attention at several resolutions (32×32, 16×16, and 8×8) and conditioning on the timestep through adaptive group normalization. This gradual substitution of convolution by attention was an early indication that convolution alone struggles to capture the long-range dependencies required for globally coherent images.

This reliance on a convolutional prior confers both a strong regularizing advantage and an inherent structural limitation. On bounded datasets and at moderate resolutions, locality and translation equivariance act as a powerful regularizer, since the network does not need to learn the basic statistics of natural images from scratch, which renders U-Nets exceptionally parameter- and FLOP-efficient. At scale, however, the same prior becomes a constraint. Global dependencies can be mediated only indirectly, through deep stacks of local operations and repeated resolution reduction, and the heterogeneous, hand-crafted structure of the U-Net, with its resolution-specific channel widths, manually placed attention layers, and fixed skip topology, offers no canonical

axis along which the architecture can be grown. Empirically, convolutional backbones lack the clean, predictable relationship between invested compute and attained quality that Transformers exhibit (Peebles and Xie, 2023; Kaplan et al., 2020), which ultimately motivated the shift to attention-based Vision Transformers.

2.2.2 The Attention Mechanism and Vision Transformers

The Transformer architecture (Vaswani et al., 2017) dispenses with convolution entirely and instead relies on *attention* as its sole mechanism for exchanging information between sequence elements. The underlying principle is content-based routing. Rather than aggregating neighbors according to a fixed spatial pattern, every element of a sequence dynamically decides, at each layer, which other elements are relevant to it. Formally, a sequence of n tokens $X \in \mathbb{R}^{n \times d}$ is linearly projected into queries, keys, and values, $Q = XW^Q$, $K = XW^K$, $V = XW^V$, and the *scaled dot-product attention* operation computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{2.20}$$

where d_k denotes the key dimensionality. Each row of the softmax term measures the compatibility of one token's query with the keys of all n tokens, and the resulting output token is the corresponding convex combination of all value vectors. The normalization by $\sqrt{d_k}$ prevents the dot products from growing so large that the softmax saturates into regions of vanishing gradient (Vaswani et al., 2017). In *multi-head* attention, the model dimension is split into h groups, and a separate attention operation runs in parallel within each lower-dimensional subspace of size d/h; their outputs are concatenated and re-projected, allowing different heads to specialize in different relational patterns. A full Transformer block alternates multi-head self-attention with a position-wise MLP, wrapped in residual connections and layer normalization. Two properties of Equation (2.20) are decisive. First, the routing is *global and content-dependent*, in that any token can influence any other token within a single layer, with interaction weights recomputed from the data at every forward pass, in sharp contrast to the static, local kernels of a CNN. Second, this expressiveness carries a price, as the pairwise compatibility matrix induces a computational and memory complexity of $\mathcal{O}(n^2)$ in the sequence length.

The Vision Transformer (ViT) (Dosovitskiy et al., 2021) transfers this sequence architecture to images through *patching*. An image $x \in \mathbb{R}^{H \times W \times C}$ is partitioned into $N = HW/P^2$ non-overlapping patches of size $P \times P$, each of which is flattened and linearly projected into a token embedding. Since self-attention is permutation-invariant and would otherwise be blind to spatial arrangement, learned positional embeddings are added to the patch tokens before they are processed by a standard Transformer encoder. Figure 2.1 illustrates the complete architecture. This construction deliberately retains almost no image-specific structure: apart from the patch extraction itself and the positional embeddings, neither locality nor translation equivariance is built into the model, and all spatial relationships must be learned from data. Dosovitskiy et al. (2021) document the consequence directly. On mid-sized datasets, ViTs trail comparably sized CNNs precisely because they lack these convolutional inductive biases, yet once pre-trained at sufficient scale, the additional data overrides the missing prior and ViTs match or surpass the strongest convolutional networks.

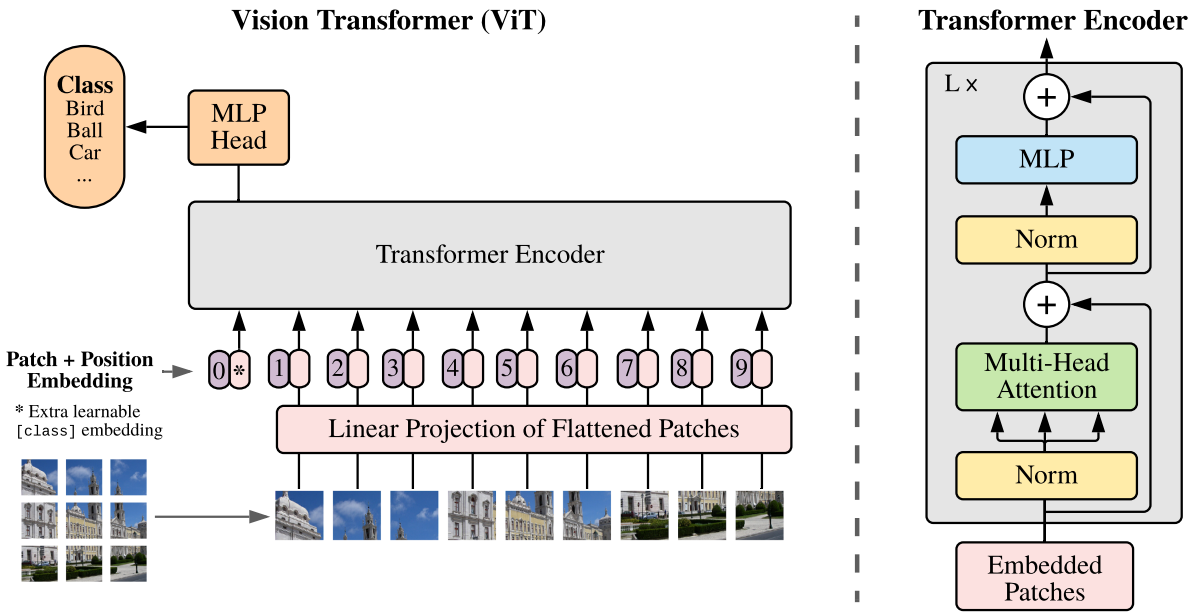


Figure 2.1: **The Vision Transformer (ViT).** *Left:* the input image is split into fixed-size patches, each of which is flattened and linearly projected into a token embedding; learned position embeddings are added and an extra learnable `[class]` token is prepended, after which the sequence is processed by a standard Transformer encoder. In the original classification setting an MLP head reads the class token to predict the label. *Right:* a single Transformer encoder block applies multi-head self-attention and a position-wise MLP, each preceded by layer normalization and wrapped in a residual connection, repeated L times. Figure from Dosovitskiy et al. (2021).

2.2.3 Diffusion Transformers

The Diffusion Transformer (DiT) (Peebles and Xie, 2023) establishes the ViT recipe as a drop-in replacement for the U-Net backbone in diffusion and flow models. The noisy (latent) state x_t is patchified into a token sequence, processed by an isotropic stack of identical Transformer blocks operating at a constant token resolution, and finally decoded by a linear layer back into the spatial velocity (or noise) prediction. The pyramidal encoder–decoder structure and the skip connections of the U-Net are discarded entirely.

The second structural difference from the U-Net concerns conditioning. As in the U-Net, the scalar timestep t is first mapped to a frequency-based (sinusoidal) *timestep embedding* and passed through an MLP, and a class label, when present, receives a learned embedding. These two embeddings are summed into a single conditioning vector c. Rather than adding c to the intermediate feature maps, however, DiT injects it through *adaptive layer normalization* (adaLN): the per-channel scale and shift parameters of every layer normalization, $\gamma(c)$ and $\beta(c)$, are regressed from c by a lightweight MLP rather than learned as static weights. The best-performing variant, adaLN-Zero, additionally regresses per-block gating parameters $\alpha(c)$ that rescale each residual branch before it is merged; these are initialized to zero, so that every block acts as the identity at initialization, which substantially stabilizes and accelerates training (Peebles and Xie, 2023). This modulation-based conditioning applies a global, spatially uniform transformation to all tokens, adds negligible computational overhead, and empirically outperforms both in-context conditioning and cross-attention in the class-conditional setting (Peebles and Xie, 2023).

The inductive bias of this architecture is deliberately minimal: beyond the patch grid and its positional embeddings, the DiT imposes essentially no spatial prior, and Peebles and Xie (2023) conclude that the U-Net's convolutional bias is not a critical ingredient for diffusion models. What the Transformer offers instead is *scalability*. Its uniform block structure exposes clean scaling axes, namely depth, width, and token count, along which capacity can be grown mechanically, and Transformer families are empirically governed by remarkably regular power-law *scaling laws* that relate model size, dataset size, and training compute to attainable loss (Kaplan et al., 2020).

Peebles and Xie (2023) verify this behavior in the generative setting: across their model family, sample quality, measured by the Fréchet Inception Distance (FID), improves monotonically with the forward-pass compute of the backbone. Generative performance thereby becomes a near-predictable function of invested resources, allowing current frontier systems to reliably convert marginal compute budgets into commensurate improvements in sample quality. Consequently, the most capable contemporary generative models, including the 8-billion-parameter rectified flow models of Esser et al. (2024) and large-scale video diffusion systems (Brooks et al., 2024), have abandoned convolutional networks in favor of Transformer backbones. To manage the complex text-conditioning required by these large-scale systems, this isotropic template must be adapted to process multiple modalities simultaneously, which leads to the multimodal architectures discussed next.

2.2.4 The Multimodal Diffusion Transformer

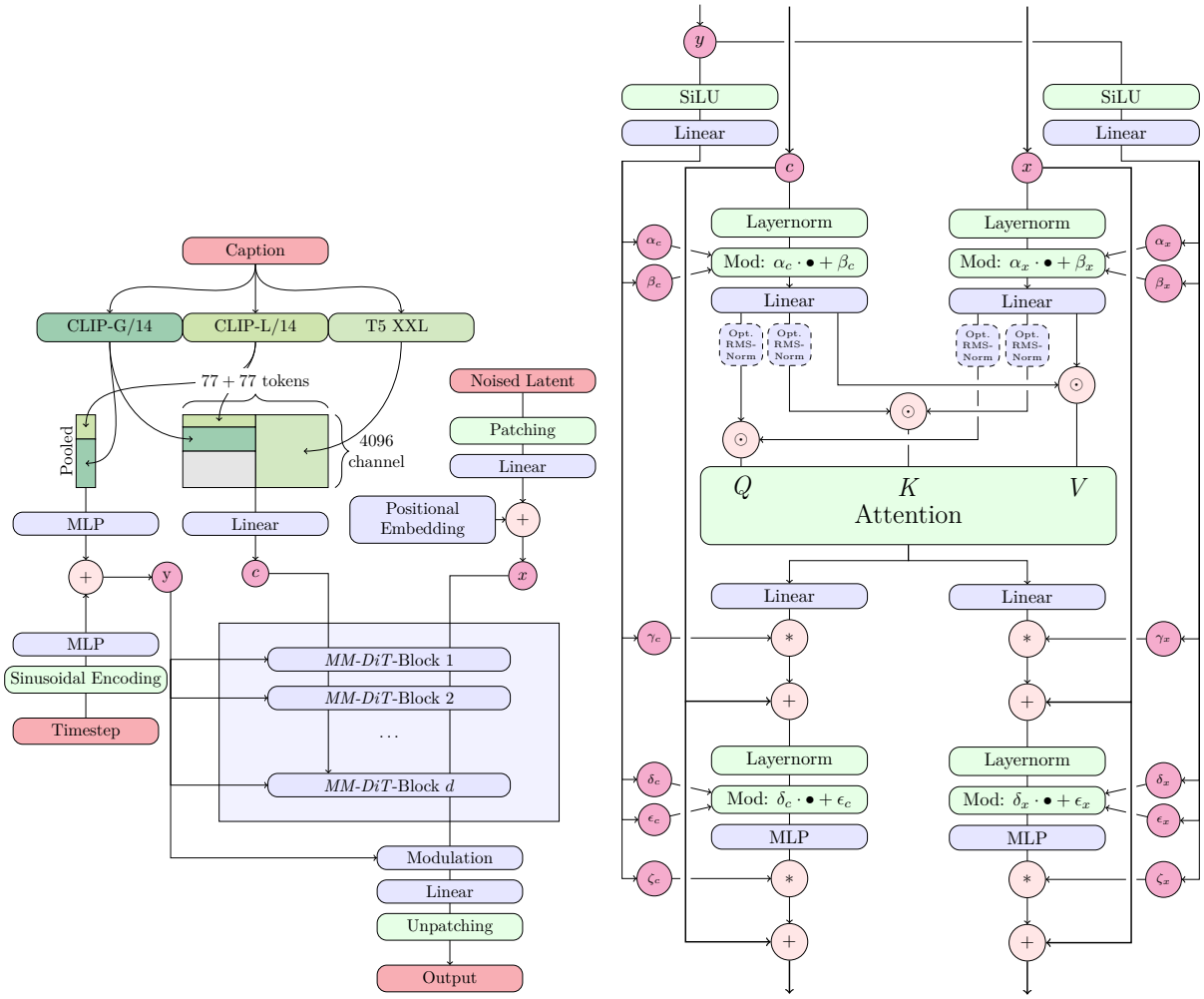
Text-to-image generation extends the regression target to a conditional vector field $v_t(x_t, c_{\text{text}}; \theta)$, and the dominant conditioning mechanism in U-Net-based systems was *cross-attention*, in which image features emit queries that attend to keys and values computed from a frozen text encoding (Rombach et al., 2022). This coupling is one-directional, since the text representation remains static across the entire network and is never informed by the evolving image state. For their large-scale rectified flow models (Stable Diffusion 3), Esser et al. (2024) therefore introduced the *Multimodal Diffusion Transformer* (MM-DiT), a DiT variant that treats text and image as two token sequences of equal standing. The architecture is depicted in Figure 2.2.

The conditioning inputs are assembled from pre-trained frozen encoders: two CLIP text encoders and a T5-XXL encoder supply a token-level context sequence, while the pooled CLIP text vectors are combined with the sinusoidal timestep embedding into a single global conditioning vector y. The core architectural choice concerns how the two modalities are processed. Motivated by the observation that text and image embeddings occupy conceptually different representational spaces, each MM-DiT block maintains *two separate sets of weights*, namely independent adaLN modulation layers, query/key/value projections, and MLPs for the text and image streams, which is equivalent to running two independent, modality-specific Transformers (Esser et al., 2024).

The two streams are coupled through exactly one operation per block: *joint attention*. The queries, keys, and values of both modalities are concatenated into a single sequence, one global self-attention operation (Equation (2.20)) is computed over all text and image tokens jointly, and the resulting sequence is split back into its two streams. Each modality thus operates in its own parameter space while still exchanging information with the other in both directions: image tokens reshape the text representation layer by layer just as text tokens steer the image tokens—in contrast to cross-attention, where the text side is frozen. Global, spatially uniform signals enter separately through *modulation*. As in the DiT, the vector y, which carries the noise level t and the coarse, pooled caption semantics, modulates both streams via adaLN scale, shift, and gating parameters, while the fine-grained token-level caption content flows through the joint attention. In controlled comparisons at matched compute, this bidirectional two-stream design outperforms both the cross-attention DiT variant and a shared-weight single-stream baseline, and it exhibits stable, predictable scaling up to 8B parameters (Esser et al., 2024).

2.2.5 Latent Generative Modeling

The scaling laws of Transformers carry a critical computational implication: state-of-the-art generative fidelity demands enormous training compute. A large part of this cost is spatial, since the number of variables to be modeled grows with the image resolution, and operating a generative backbone directly in pixel space at high resolution is computationally prohibitive. Moreover, training a competitive model requires billions of vector field evaluations over the course of optimization. Orthogonal to the efficiency methods studied in this thesis, which focus on timestep sampling during training, a complementary line of work targets this *spatial* axis of the computational burden. Most prominently, *latent diffusion models* (LDMs) (Rombach et al., 2022), popularized as Stable Diffusion, compress the spatial representation and thereby drastically reduce the cost of every network evaluation.



(a) Overview of all components.

(b) A single MM-DiT block.

Figure 2.2: **The Multimodal Diffusion Transformer (MM-DiT) architecture of Stable Diffusion 3.** *(a)* Caption and noised latent are embedded into a text and an image token stream; the timestep embedding and the pooled text representation form the global modulation signal y. *(b)* Within each block, the two streams are processed by modality-specific weights and interact exclusively through a joint attention operation over the concatenated token sequences. Concatenation is indicated by $\odot$ and element-wise multiplication by $*$. Figure from Esser et al. (2024).

The principle underlying latent diffusion is a two-stage separation of perceptual and semantic compression (Rombach et al., 2022). In the first stage, a convolutional autoencoder is trained to perform *perceptual compression*. An encoder $\mathcal{E}$ maps an image $x \in \mathbb{R}^{H\times W\times 3}$ to a compact, continuous latent representation $z = \mathcal{E}(x) \in \mathbb{R}^{h\times w\times c}$ with a spatial downsampling factor $f = H/h$, while a decoder $\mathcal{D}$ reconstructs the image from this latent space. To ensure these reconstructions are sharp and locally realistic, the training objective combines a standard reconstruction loss, a perceptual loss, and a patch-based adversarial discriminator. Crucially, to prevent the latent space from becoming arbitrarily scaled or highly disjoint—which would destabilize the subsequent diffusion training—the architecture is formulated as a weakly regularized variational autoencoder (VAE) (Kingma and Welling, 2014). This is achieved by applying a slight Kullback–Leibler (KL) penalty that gently forces the latent representations toward a standard normal distribution, ensuring a smooth and well-structured continuous space. In the second stage, the generative model is trained *entirely within this latent space*. The probability paths, the objective, and the numerical ODE integration of Section 2.1.3 carry over directly, merely treating z rather than x as the data variable. At inference time, the learned flow is integrated in latent space, and the decoder $\mathcal{D}$ is invoked exactly once to map the terminal latent state to pixels.

The acceleration mechanism follows from an empirical property of natural images: most bits of a digital image encode perceptually near-irrelevant high-frequency detail, and generative models trained in pixel space expend a large fraction of their capacity and compute on modeling exactly these imperceptible statistics (Rombach et al., 2022). Delegating this perceptual detail to a cheap, universally reusable autoencoder allows the expensive generative backbone to operate on a representation whose spatial dimensionality is reduced by a factor of f^2. For convolutional backbones, the per-evaluation cost shrinks accordingly, and for Transformer backbones the effect compounds, since the token count drops by f^2 and the attention cost therefore by up to f^4. In practice, this reduces training and inference cost by orders of magnitude and enabled high-resolution synthesis at a small fraction of the compute of contemporaneous pixel-space models (Rombach et al., 2022).

This efficiency, however, is not free, and the cost is paid in sample quality. The autoencoder compression is lossy, so any information the encoder discards is irrecoverable to the generative model, and the reconstruction quality of the autoencoder therefore constitutes a hard upper bound on attainable sample fidelity. Rombach et al. (2022) characterize the resulting trade-off explicitly: small downsampling factors leave most of the perceptual compression to the diffusion model and yield little acceleration, whereas overly aggressive factors discard information in the first stage and visibly degrade fidelity, with moderate factors ($f \in \{4, 8\}$) striking the best balance. Symptomatically, fine textures, small faces, and legible text are the first casualties of aggressive compression. Consistently, Esser et al. (2024) show that the capacity of the latent space itself bottlenecks achievable image quality, and accordingly equip Stable Diffusion 3 with a higher-capacity autoencoder ($c = 16$ latent channels) to raise this reconstruction ceiling.

Latent-space compression thus makes each network evaluation cheaper. It leaves untouched, however, a second and independent axis of training efficiency, namely *where* along the generative trajectory those evaluations are best spent. Under the standard CFM objective (Equation (2.16)), every update draws $t \sim \mathcal{U}(0, 1)$ and thereby allocates identical amounts of the training budget to every segment of the trajectory, regardless of how difficult, how noisy, or how useful the corresponding regression targets currently are for learning.

ATS instead treats this temporal budget allocation as a decision to be learned, repeatedly choosing timesteps, observing how the model's training signal responds, and refining its choices accordingly. This is precisely the structure of a sequential decision-making problem under uncertainty, and the mathematical framework for solving it, reinforcement learning, is the subject of the following section.

2.3 Reinforcement Learning and Policy Optimization

While generative modeling with Flow Matching focuses on learning underlying data distributions, *Reinforcement Learning (RL)* provides a mathematical framework for optimizing sequential decision-making processes. In the context of this thesis, RL serves as the theoretical foundation for dynamically guiding the Flow Matching training process.

First, this section introduces the fundamental concepts and terminology of RL. We then detail the foundational REINFORCE algorithm (Williams, 1992), which provides the mathematical solution for model-free, on-policy optimization—the central mechanism driving the ATS methodology presented in Chapter 4. The section concludes by establishing two algorithmic enhancements for stable policy training: the integration of a baseline function to reduce gradient variance and the application of entropy regularization to encourage sustained exploration.

2.3.1 Reinforcement Learning Concepts and Terminology

In the standard reinforcement learning setting, an active decision-making *agent* interacts with an uncertain *environment* over a sequence of discrete timesteps. At each timestep t, the agent receives a *state* representation s_t and selects an *action* a_t according to its *policy* $\pi(a_t|s_t)$, which defines a mapping (or probability distribution) from states to actions. In return, the environment yields the next state s_{t+1} and a scalar *reward* r_t. The fundamental objective of the agent is to maximize the expected cumulative return $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, where $\gamma \in (0, 1]$ is a discount factor. This goal is often formalized using a *value function* $V^\pi(s) = \mathbb{E}[R_t|s_t = s]$, which estimates the expected long-term return starting from state s and following policy π (Sutton and Barto, 2018).

In environments with discrete, low-dimensional spaces, the value function or policy can be represented exactly using tabular methods. However, because the state space of high-resolution images is vastly high-dimensional and the action space of sampling a timestep $t \in [0, 1]$ is strictly continuous, such tabular methods are mathematically intractable. This necessitates *Deep Reinforcement Learning* (Deep RL) approaches, which utilize neural networks as highly expressive function approximators (Sutton and Barto, 2018).

In *value-based* model-free reinforcement learning, a neural network is used to approximate the optimal action-value function $Q^*(s, a)$. The agent then implicitly defines its policy by selecting the action that yields the highest predicted value. However, finding the maximum value across a continuous action space is analytically and computationally prohibitive. Consequently, *policy-based* model-free methods bypass value estimation to directly parameterize the policy $\pi(a|s; \phi)$ with a neural network. The network parameters are then updated by performing gradient ascent directly on the expected return $\mathbb{E}[R_t]$ (Mnih et al., 2016). This policy-based interaction loop is

conceptually illustrated in Figure 2.3.

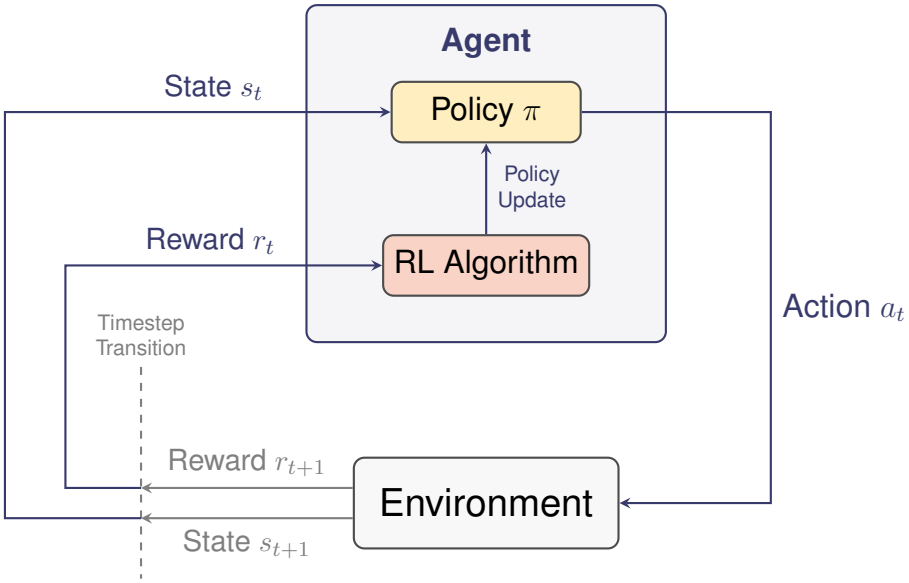


Figure 2.3: **The reinforcement learning loop for policy-based model-free methods.** The agent internally maps states to actions via a parameterized policy, which is iteratively improved by the RL algorithm using the reward signal. The dashed line delineates the timestep transition from t to $t + 1$. This architecture reflects the core learning mechanism utilized by the Adaptive Timestep Sampling (ATS) framework.

In this thesis, RL is employed to learn a dynamic timestep sampling policy that steers the training focus of the Flow Matching model toward the most critical timesteps. This is achieved through the ATS framework (Kim et al., 2025a), detailed in Chapter 4. Within this context, the general RL formulation is heavily constrained and operates strictly as a *single-step contextual bandit problem*. Specifically, the state is defined as the clean image representation (x_1), the action is the sampled continuous timestep $t \in [0, 1]$, and the reward is the measured reduction in the Flow Matching loss ($\tilde{\Delta}_k^t$). Because the episode length is strictly 1, the sequential dependency of standard RL environments vanishes ($\gamma = 0$), and the task of maximizing the expected *cumulative* return simplifies entirely to maximizing the expected *immediate* reward.

To solve this continuous RL optimization problem, ATS leverages a policy-based model-free architecture, specifically utilizing the *REINFORCE* algorithm (Williams, 1992) to directly optimize the parameterized timestep sampling policy.

2.3.2 The REINFORCE Algorithm

The *REINFORCE* family of algorithms provides a general mathematical solution for optimizing networks of stochastic units (Williams, 1992). Originally developed by Williams in the late 1980s and formally consolidated in his canonical 1992 paper, REINFORCE is explicitly designed to maximize expected reinforcement in associative learning tasks without requiring an explicit model of the environment.

The objective of a policy-based agent is to find network parameters ϕ that maximize the expected

reward $J(\phi) = \mathbb{E}_{a\sim\pi_\phi}[R]$. To apply gradient ascent, we require the gradient $\nabla_\phi J(\phi)$. However, the standard backpropagation gradient cannot pass through the stochastic sampling operation of drawing an action $a \sim \pi_\phi$.

To solve this optimization problem, Williams (1992) defines any algorithm as a REINFORCE algorithm if its parameter update rule takes the form $\Delta\phi = \alpha(R - b)\mathbf{e}$. The name itself is an acronym detailing these exact algorithmic components:

- **RE**ward **I**ncrement: The parameter update vector $\Delta\phi$.
- **N**onnegative **F**actor: The scalar learning rate α.
- **O**ffset **R**einforcement: The scalar reward R offset by a baseline b.
- **C**haracteristic **E**ligibility: The gradient vector of the log-probability of the chosen action, $\mathbf{e} = \nabla_\phi \log \pi_\phi$.

A critical mathematical assumption for this algorithm is that the reinforcement baseline b must be conditionally independent of the chosen action given the current state. Under this assumption, Williams (1992) proves a powerful guarantee:

Theorem: REINFORCE Gradient Guarantee (Williams, 1992)

For any REINFORCE algorithm, the expected parameter update vector lies exactly in the direction of the true performance gradient:

$$\mathbb{E}[\Delta\phi|\phi] = \alpha\nabla_\phi\mathbb{E}[R|\phi] \tag{2.21}$$

Consequently, the update rule provides an **unbiased estimate** of the gradient.

The Log-Derivative Trick. To bypass the non-differentiable sampling operation, REINFORCE employs the *log-derivative trick*. By expressing the expectation as an integral, pulling the gradient inside using the Leibniz rule, and applying the mathematical identity $\nabla\pi = \pi\nabla\log\pi$, the intractable gradient of an expectation is analytically transformed into a computable expectation of a gradient:

$$\begin{aligned}
\nabla_\phi J(\phi) &= \nabla_\phi \mathbb{E}_{a\sim\pi_\phi}[R] \\
&= \nabla_\phi \int \pi_\phi(a|s) R \, da && \text{(Expectation as integral)} \\
&= \int \nabla_\phi \pi_\phi(a|s) R \, da && \text{(Leibniz rule)} \\
&= \int \pi_\phi(a|s) \nabla_\phi \log \pi_\phi(a|s) R \, da && \text{(Log-derivative trick)} \\
&= \mathbb{E}_{a\sim\pi_\phi}\big[\nabla_\phi \log \pi_\phi(a|s) \cdot R\big]. && \text{(Integral back to expectation)}
\end{aligned} \tag{2.22}$$

This transformation allows the true gradient to be estimated via Monte Carlo simulation by simply sampling actions from the current policy and observing the resulting rewards, with the $\nabla_\phi \log \pi_\phi(a|s)$ term serving as the characteristic eligibility $\mathbf{e}$ defined above.

Variance Reduction via Baselines and Advantage Functions. While REINFORCE guarantees an unbiased gradient estimate, the choice of the baseline b critically impacts the variance of this estimate. In the trivial case where no baseline is utilized ($b = 0$), the characteristic eligibility is scaled solely by the raw, absolute reward R. Because empirical rewards can often be arbitrarily scaled or inherently noisy depending on the environment, relying strictly on the raw reward can introduce massive statistical variance into the policy gradient. This uncentered multiplication frequently leads to unstable parameter updates and slow convergence (Konda and Tsitsiklis, 1999; Williams, 1992).

This variance can be substantially mitigated by introducing a carefully chosen, state-dependent baseline $b(s)$. As long as the baseline is conditionally independent of the sampled action a, subtracting it evaluates to zero in expectation. This can be shown by factoring the baseline out of the gradient and integrating over the probability density:

$$\begin{aligned}\mathbb{E}_{a\sim\pi_\phi}\big[b(s)\nabla_\phi \log \pi_\phi(a|s)\big] &= b(s)\int \pi_\phi(a|s)\nabla_\phi \log \pi_\phi(a|s)\, da \\ &= b(s)\nabla_\phi \int \pi_\phi(a|s)\, da \\ &= b(s)\nabla_\phi(1) = 0. \end{aligned} \tag{2.23}$$

Therefore, subtracting any state-dependent baseline preserves the unbiased nature of the REINFORCE gradient estimate. However, while any action-independent baseline is unbiased, it does not guarantee variance reduction. The variance of the gradient estimate scales with the variance of the offset term, $\mathrm{Var}(R - b(s))$. To effectively minimize this variance, a proper baseline should closely track the expected magnitude of the reward.

In modern Advantage Actor-Critic architectures (Mnih et al., 2016), this baseline is typically provided by an auxiliary *value network* (the Critic) that learns to predict the expected average reward for the current state, defined as $V^\pi(s) = \mathbb{E}[R|s]$. The offset reinforcement term then naturally becomes the *advantage function*

$$A(s, a) = R - V^\pi(s). \tag{2.24}$$

Substituting the advantage into the REINFORCE update strictly centers the learning signal: actions that perform better than the expected average ($A > 0$) are pushed higher in probability, while those performing worse than expected ($A < 0$) are suppressed.

Exploration via Entropy Regularization. A fundamental risk when optimizing parameterized policies over continuous spaces is that the predicted distributions can prematurely collapse into Dirac delta functions, resulting in strictly deterministic behavior. To enforce sustained, continuous exploration and prevent this premature policy convergence, *entropy regularization* is incorporated into the optimization objective. This methodology was pioneered by Williams and Peng (1991) and remains a cornerstone of modern actor-critic frameworks like *Asynchronous Advantage Actor Critic (A3C)* (Mnih et al., 2016).

The differential entropy of a policy distribution mathematically quantifies the degree of randomness or uncertainty in the agent's action selection. Higher entropy values indicate a broader,

flatter distribution, which inherently translates to a wider exploration of the available action space.

To explicitly incentivize this exploratory behavior, the analytical gradient of the entropy term $\mathcal{H}[\pi_\phi]$ is introduced as an additive component scaled by an entropy coefficient η that dictates the exploration-exploitation trade-off. This updates the total policy gradient objective to

$$\nabla_\phi J_{\text{total}}(\phi) = \mathbb{E}_{a\sim\pi_\phi}\big[A(s,a)\cdot\nabla_\phi \log \pi_\phi(a|s)\big] + \eta\nabla_\phi\mathcal{H}[\pi_\phi], \tag{2.25}$$

which aligns with established practices for regularized policy optimization (Mnih et al., 2016).

3 Related Work

As established in the previous chapter, Flow Matching provides state-of-the-art generation quality across various domains (Lipman et al., 2023; Esser et al., 2024). While techniques such as the *Reflow* procedure (Liu et al., 2023) have been introduced to systematically straighten probability paths and thereby enhance inference efficiency, the exceptionally high computational cost of the training phase remains a significant algorithmic challenge (Li et al., 2024). Training high-capacity Flow Matching models to convergence requires massive computational resources; for instance, scaling the rectified flow formulation to an 8B parameter model demands up to 5×10^{22} training FLOPs (Esser et al., 2024). This immense compute requirement poses a substantial barrier to accessibility, rapid model iteration, and environmental sustainability (Kim et al., 2025a).

To systematically mitigate these computational bottlenecks inherent to the optimization process, recent literature has explored two orthogonal dimensions: optimization of data-noise boundary couplings, and objective-level methods reallocating the training focus across the temporal domain. This chapter reviews both paradigms, establishing the theoretical foundations of marginal couplings before delving deeply into the Flow Matching timestep compute allocation strategies that form the primary focus of this thesis.

3.1 Variance Reduction via Optimized Boundary Couplings

In standard Conditional Flow Matching, the initial noise sample x_0 and the terminal data sample x_1 are drawn strictly independently from their respective marginal distributions (Lipman et al., 2023). However, this randomized, independent coupling of noise and data inherently leads to overlapping conditional probability paths, as demonstrated in the topological toy example in Figure 3.1. These intersecting trajectories produce conflicting target signals for the neural vector field during stochastic gradient descent (SGD). This structural ambiguity manifests as high gradient variance, which fundamentally slows down model training (Pooladian et al., 2023; Yoon and Lee, 2024; Tong et al., 2024).

To alleviate this high stochastic gradient variance inherent to the independent marginal sampling in standard Flow Matching, spatial variance reduction methods have been developed to directly alter the joint distribution of the boundary conditions or augment the spatial data representations. By systematically restructuring the boundary pairings, these approaches prevent conflicting vector fields and straighten the learned marginal probability paths.



(a) Independent Coupling (Standard CFM) (b) Optimal Transport Coupling (OT-CFM)

Figure 3.1: **Geometric illustration of probability paths under different boundary couplings.** *(a)* Standard independent coupling randomly pairs prior noise representations x_0 (dark purple) with data samples x_1 (red). This randomized assignment results in heavily intersecting trajectories (pink lines) and conflicting vector field targets during training. *(b)* Mini-batch Optimal Transport coupling dynamically assigns pairs to minimize the approximated 2-Wasserstein distance, measuring the transport cost. This alignment strictly untangles the trajectories, yielding straight, non-overlapping flows (orange lines) that eliminate target ambiguity and reduce stochastic gradient variance (Tong et al., 2024; Pooladian et al., 2023).

Mini-batch Optimal Transport and Multisample Couplings. Tong et al. (2024) and Pooladian et al. (2023) concurrently propose substituting the independent marginal couplings with dynamic, transport-cost-minimizing joint distributions. By applying Optimal Transport (OT) algorithms within mini-batches, these methods construct a joint coupling that explicitly aligns noise and data pairs to minimize the 2-Wasserstein distance. This spatial alignment systematically straightens the probability paths and substantially reduces the variance of the regression target (Tong et al., 2024). As mathematically established by Pooladian et al. (2023), increasing the mini-batch size monotonically decreases the transport cost. In the theoretical limit of infinite batch sizes, this alignment approaches a zero-variance limit under the assumptions of their analysis, thereby accelerating training without necessitating architectural modifications.

Immiscible Diffusion. Approaching the boundary coupling problem from a physics-inspired perspective, Li et al. (2024) identify the "miscibility" phenomenon in standard diffusion models, wherein individual data samples are diffused across the entire noise space. This dynamic creates highly entangled mappings that severely complicate the optimization of the denoising function. To enforce "immiscibility" and maintain distinguishable generation paths, the authors introduce a noise assignment technique that pairs mini-batches of sampled noise to images by minimizing their pairwise L_2 distances via a quantized linear sum assignment algorithm. Operating as a computationally efficient approximation of exact OT, this method strictly restricts the target space assignment. Empirical evaluations demonstrate that this spatial restriction significantly stabilizes the denoising targets. Li et al. (2024) report up to a $3\times$ training acceleration for Consistency Models. Furthermore, the approach substantially accelerates convergence and improves FID when applied to multi-step models such as DDIM, and it yields consistent performance gains across unconditional, class-conditional, and fine-tuning tasks utilizing Stable Diffusion v1.4 on large-scale ImageNet datasets.

Augmented Flow Matching. Rather than directly optimizing the discrete transport map between source and target data, Yoon and Lee (2024) introduced *Augmented Flow Matching (AFM)*, a framework that incorporates correlated auxiliary variables to stabilize the learning process. Augmenting the data representation with auxiliary random variables mapped to the training data via simple linear operations provides the model with additional degrees of freedom to untangle intersecting flow trajectories in a higher-dimensional space. Conditioning the flow on these augmented latent representations systematically reduces the conditional variance of the Flow Matching estimator. Quantitative evaluations on the CIFAR-10 benchmark demonstrate that AFM consistently improves the FID as well as Precision and Recall metrics across various Neural Function Evaluations (NFE) regimes compared to standard Flow Matching baselines. Crucially, Yoon and Lee (2024) demonstrate that these substantial performance gains and variance reductions are achieved with functionally negligible computational overhead, requiring less than a 0.5% increase in both memory consumption and total training FLOPs.

Stable Target Field. Bridging the gap between spatial data manipulation and target regularization, Xu et al. (2023) proposed the *Stable Target Field (STF)* to address the high variance inherent in denoising score-matching. STF isolates the variance bottleneck in the highly volatile intermediate regime of the forward process, where multiple overlapping data modes exert competing influences on the score function. To stabilize this ambiguous intermediate noise regime, a spatial reference batch is constructed and self-normalized importance sampling is employed to approximate the conditional expectation of the score. By calculating a more consistent weighted average of potential targets across this spatial reference batch, STF substantially reduces the trace-of-covariance of the stochastic training targets. While this self-normalization introduces a theoretical bias to the estimator, Xu et al. (2023) prove that this bias rigorously vanishes as the reference batch size increases, effectively trading a negligible bias for a substantial acceleration in model convergence and a measurable improvement in image quality.

3.2 Shifting the Optimization Focus across the Generative Trajectory

In Flow Matching, the neural vector field $v_\theta(x_t, t)$ is parameterized to depend strictly on the intermediate latent state x_t and the continuous time variable t. Consequently, the specific values of x_t and t sampled during SGD determine the empirical data distribution observed by the model. This temporal sampling mechanism fundamentally governs how the neural backbone allocates its representational capacity and optimization focus across the different phases of the generative trajectory.

This allocation of training capacity is governed by three distinct algorithmic mechanisms. These mechanisms become mathematically explicit when substituting the general Gaussian probability path, $x_t = \alpha_t x_1 + \beta_t x_0$, into the CFM objective. By applying the reparameterization trick to draw the prior noise $x_0 \sim \mathcal{N}(0, I)$ directly, the objective is reformulated as

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t\sim\pi(t),\, x_1\sim q(x_1),\, x_0\sim\mathcal{N}(0,I)} \left[w(t) \left\| v_t(\alpha_t x_1 + \beta_t x_0; \theta) - u_t(\alpha_t x_1 + \beta_t x_0 \mid x_1) \right\|^2 \right]. \quad (3.1)$$

Within this expanded formulation, the training focus is explicitly controlled by three independent dials:

1. **The Noise Schedule (α_t, β_t):** Defines the geometric interpolation between the data and noise distributions, controlling the SNR observed at a timestep t.
2. **The Loss Weighting ($w(t)$):** A time-dependent scalar that directly amplifies or suppresses the magnitude of the gradient at specific points of the timestep interval $t \in [0, 1]$.
3. **The Timestep Sampling Distribution ($\pi(t)$):** The probability density from which timesteps are drawn during the Monte Carlo simulation of the objective, explicitly dictating the frequency of SGD updates allocated to specific temporal regimes.

In DDPMs (Ho et al., 2020) these three components were heavily entangled. Because the DDPM objective was derived from the variational lower bound, modifying the noise schedule inherently altered the implicit loss weights (Karras et al., 2022). In contrast, the regression-based nature of the Flow Matching objective completely decouples these mechanisms (Lipman et al., 2023). This modularity enables the independent tuning of each component to maximize gradient efficiency.

3.2.1 Noise Scheduling

The noise schedule dictates the geometric form of the probability path, governing the rate of interpolation between the prior noise distribution and the clean data manifold. By defining how rapidly the data signal emerges from the prior noise, the schedule fundamentally controls the SNR of the objective at any given timestep t (Karras et al., 2022; Esser et al., 2024). Assuming a Gaussian probability path, as established in Equation (2.14) (Section 2.1), the temporal evolution of the intermediate state simplifies to the affine mixture $x_t = \alpha_t x_1 + \beta_t x_0$. Assuming the standard formulation where the endpoints x_0 and x_1 are treated as independent random variables drawn from their respective marginal distributions, the SNR for these paths is formally defined as the variance ratio

$$\mathrm{SNR}(t) = \frac{\mathrm{Var}(\alpha_t x_1)}{\mathrm{Var}(\beta_t x_0)} = \frac{\alpha_t^2 \mathrm{Var}(x_1)}{\beta_t^2 \mathrm{Var}(x_0)} \propto \frac{\alpha_t^2}{\beta_t^2}. \tag{3.2}$$

In classical diffusion paradigms, adjusting this schedule provided a primary dial for accelerating model convergence. Early frameworks, such as DDPMs, utilized a VP formulation with a linear noise schedule, which systematically scales down the data while injecting noise (Kim et al., 2025a). Conversely, VE formulations maintain a constant data scale ($\alpha_t = 1$) while exponentially increasing the noise variance (Karras et al., 2022). Because the standard linear VP schedule was empirically found to destroy the semantic signal too rapidly, subsequent architectures introduced a cosine schedule to induce a more gradual decay of the SNR, thereby preventing premature signal destruction and optimizing the intermediate learning difficulty (Esser et al., 2024).

However, the widespread adoption of CondOT probability paths in Flow Matching restricts the interpolation to straight-line trajectories (Lipman et al., 2023; Esser et al., 2024; Liu et al., 2023). This structural constraint fixes the interpolation coefficients to $\alpha_t = t$ and $\beta_t = 1 - t$, yielding the closed-form signal-to-noise ratio

$$\mathrm{SNR}_{\mathrm{CondOT}}(t) = \frac{t^2}{(1-t)^2} = \left(\frac{t}{1-t}\right)^2. \tag{3.3}$$

As a consequence, the temporal signal-noise profile induced by the probability path is no longer a tunable design choice but is intrinsically determined by the straight-line flow. Accordingly,

steering the training emphasis of modern Flow Matching models primarily relies on adapting complementary temporal mechanisms, such as loss weighting and timestep sampling.

3.2.2 Loss Weighting

In the context of standard ϵ-prediction diffusion models, researchers identified that the implicit loss weighting of the original objective is highly suboptimal, often prioritizing visually imperceptible high-frequency details while neglecting global semantic structure. Because these traditional frameworks typically maintain a uniform timestep sampling distribution $\pi = \mathcal{U}(0, 1)$, they steer the optimization focus exclusively by manipulating the time-dependent loss weight $w(t)$. For instance, Choi et al. (2022) introduced *Perception-Prioritized (P2) weighting*, which explicitly scales the objective to prioritize intermediate integration regimes, as these phases are empirically shown to encapsulate the richest semantic visual concepts. Furthermore, the ϵ-prediction formulation suffers from severe gradient instability and conflicting optimization directions at extremely low SNR regimes near pure noise. To mitigate this structural flaw, Hang et al. (2023) proposed the *Min-SNR strategy*, which aggressively clamps the weighted loss function at extreme noise levels to prevent gradient divergence. Complementary approaches have expanded upon this by employing *uncertainty weighting* (Go et al., 2023) or *SNR-based debiasing* mechanisms (Yu et al., 2024) to further stabilize the objective landscape.

While these explicit weighting heuristics were critical for stabilizing ϵ-prediction architectures, the paradigm fundamentally shifts under the Flow Matching framework. Because CondOT-FM regresses directly against the finite-variance velocity target $u_t = x_1 - x_0$, it naturally avoids the SNR singularities and exploding gradients inherent to ϵ-prediction. Consequently, the standard CondOT Flow Matching objective employs a uniform loss weight $w(t) = 1$ across the entire integration interval (Esser et al., 2024). Rather than manipulating the loss magnitudes via $w(t)$, optimizing the generative process in modern Flow Matching relies primarily on adapting the temporal sampling density $\pi(t)$ to achieve the desired semantic focus.

Mathematically, scaling the loss objective by a time-dependent factor is equivalent in expectation to drawing $t \sim \pi(t)$ from a non-uniform distribution, provided the sampling density is proportional to the applied weight (Esser et al., 2024). However, these two mechanisms operate fundamentally differently regarding computational efficiency. Loss weighting requires executing full forward and backward passes for uniformly sampled timesteps, only to scale down the resulting gradients for deprioritized regions—effectively wasting compute on trivially predictable regimes. Conversely, non-uniform timestep sampling $\pi(t)$ explicitly alters the batch composition, allocating a higher volume of actual SGD updates to the most complex intermediate trajectory phases. This ensures that the network capacity and training compute are efficiently concentrated on the most challenging aspects of the vector field, establishing timestep sampling as the primary mechanism for accelerating convergence in Flow Matching.

3.2.3 Non-Uniform Timestep Sampling

In standard Flow Matching (Lipman et al., 2023), the marginal vector field is conventionally approximated by drawing timesteps uniformly distributed over the temporal domain, $t \sim \mathcal{U}(0, 1)$. However, allocating training compute uniformly across the entire temporal interval has been

empirically and theoretically demonstrated to be highly suboptimal by several recent studies (Esser et al., 2024; Kim et al., 2025a; Zheng et al., 2024b; Zheng et al., 2024a). The difficulty of the vector field regression task fluctuates significantly depending on the noise level. Theoretical analyses characterize this temporal non-uniformity from various complementary perspectives, including the highly variable stochastic gradient variance (Kim et al., 2025a), perceptual relevance and target ambiguity of intermediate integration scales (Esser et al., 2024) and the asymmetric evolution of the marginal distributions (Zheng et al., 2024a; Zheng et al., 2024b).

Consequently, various static non-uniform sampling strategies have been proposed to systematically bias the training distribution toward these most informative temporal regions. Unlike pure loss weighting, which scales gradients post-hoc for uniformly sampled batches, non-uniform sampling explicitly reallocates actual training compute (FLOPs) and SGD updates to the desired noise regimes, yielding substantial improvements in training efficiency.

Static Logit-Normal Timestep Sampling

Karras et al. (2022) and Esser et al. (2024) hypothesize that the difficulty of the prediction task varies significantly across the temporal domain. At the extreme boundaries near pure noise ($t \approx 0$) or pure data ($t \approx 1$), the optimal predictions trivially collapse toward the prior mean or the dataset average, respectively, making gradient updates in these regions largely uninformative. Conversely, they reason that the most complex vector field dynamics and perceptually critical learning occur in the intermediate temporal regime ($t \approx 0.5$). Consequently, both works advocate for systematically biasing the timestep sampling distribution to shift training weight away from the trivial prediction tasks at the tails and toward the semantically richer intermediate stages.

Building upon this rationale, Karras et al. (2022) originally introduced a log-normal sampling distribution tailored for the unbounded noise schedules ($t \in (0, \infty)$) of Variance Exploding diffusion models. To adapt this strategy to Flow Matching, where the time domain is strictly confined to $t \in [0, 1]$, Esser et al. (2024) replaced it with a logit-normal distribution. This substitution naturally respects the finite boundaries while preserving the crucial property of concentrating training compute in the middle of the trajectory.

The continuous logit-normal probability density function is defined as

$$\pi_{\mathrm{LN}}(t; m, s) = \frac{1}{s\sqrt{2\pi}} \frac{1}{t(1-t)} \exp\left(-\frac{\big(\mathrm{logit}(t) - m\big)^2}{2s^2} \right), \tag{3.4}$$

where the inverse sigmoid mapping is formalized as $\mathrm{logit}(t) = \ln(\frac{t}{1-t})$. The location parameter m shifts the sampling bias toward either the noise or data manifolds, while the scale s controls the concentration of the samples around the central mode (see Figure 3.2). In practice, the distribution is efficiently implemented by sampling a latent variable z from a standard normal distribution and subsequently mapping the result through a standard logistic sigmoid function, yielding

$$t = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z \sim \mathcal{N}(m, s^2). \tag{3.5}$$

Extensive empirical evaluations by Esser et al. (2024) demonstrate that a zero-centered, unit-variance parameterization ($m = 0, s = 1$) consistently achieves superior performance across

various metrics and datasets. This specific logit-normal parameterization decisively outperforms baseline rectified flow formulations relying on uniform timestep sampling. This performance advantage was shown to remain consistent regardless of the number of integration steps used during sampling.

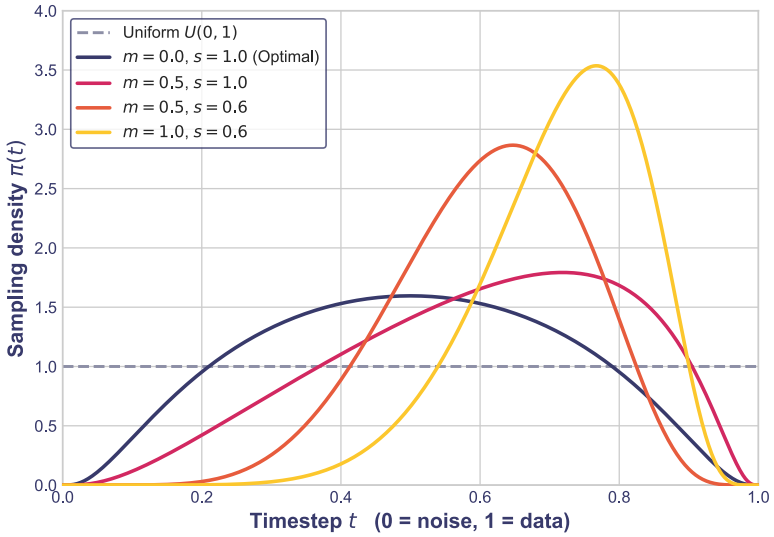


Figure 3.2: **Probability density functions for logit-normal timestep sampling.** The optimal configuration ($m = 0.0, s = 1.0$) concentrates training focus heavily on the intermediate steps of the trajectory, compared against alternative parameterizations reported by Esser et al. (2024). The standard uniform distribution $\mathcal{U}(0, 1)$ is provided as a reference baseline.

Static Bernoulli Timestep Sampling

Zheng et al. (2024a) derive an alternative paradigm for non-uniform timestep sampling from Optimal Transport theory. By analyzing the forward diffusion process of DDPMs as a Wasserstein gradient flow, they prove that the upper bound of the 2-Wasserstein distance between different initial distributions decreases exponentially over time. This theoretical insight implies that the rate of change in the state distribution is strictly non-uniform, with the most significant distributional disparities occurring within a specific initial time interval.

To exploit this structural property, Zheng et al. (2024a) formulate a Bernoulli-based timestep sampling strategy to prioritize regions of the trajectory exhibiting the largest distributional changes. Adapted to the standard Flow Matching notation—where the temporal direction is reversed such that $t = 1$ denotes the clean data manifold—the time domain is partitioned into a highly significant, data-adjacent interval $t \in (t_\delta, 1]$ and a less significant, noise-adjacent interval $t \in [0, t_\delta]$. The timestep is selected by first sampling an interval governed by a Bernoulli parameter p, and subsequently drawing t uniformly within that chosen interval. To rigorously translate this method into a valid timestep distribution, the interval weights must be normalized by the partition function $Z = (1 - p)\, t_\delta + p\,(1 - t_\delta)$. This yields the piecewise continuous

probability density function

$$\pi_{\text{Bernoulli}}(t; p, t_\delta) = \begin{cases} \dfrac{1-p}{Z} & \text{for } t \in [0, t_\delta] \quad \text{(noise side)}, \\ \dfrac{p}{Z} & \text{for } t \in (t_\delta, 1] \quad \text{(data side)}, \end{cases} \tag{3.6}$$

which corresponds to their empirical proposal.

By setting $p > 0.5$, the training procedure actively prioritizes the temporal regions exhibiting the steepest gradients in the Wasserstein space (see Figure 3.3). Through extensive ablation studies, Zheng et al. (2024a) demonstrated that oversampling the interval $t \in (0.2, 1]$, corresponding to the 80% of the trajectory closest to the data manifold, with a weight of $p = 0.8$ yields the best performance for their *DDPM-BS* model. This reallocation of training compute toward regions exhibiting the largest distributional changes was shown to accelerate convergence and improve sample quality.

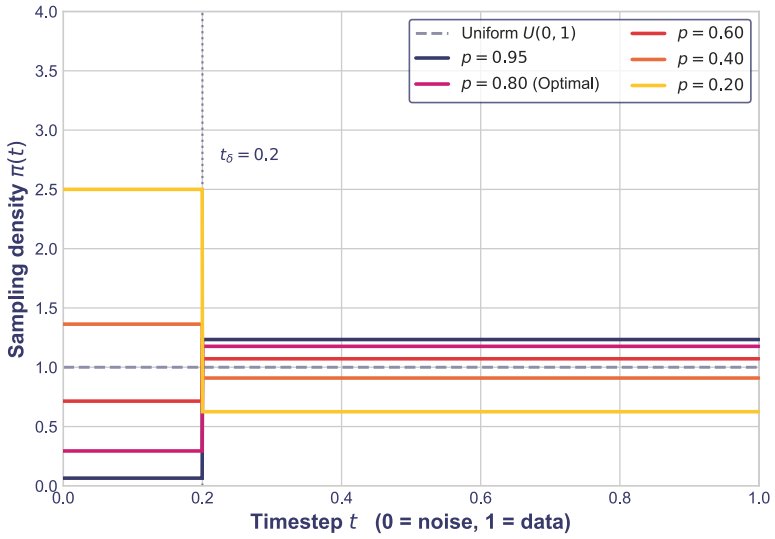


Figure 3.3: **Probability density functions for the Bernoulli sampling strategy proposed by Zheng et al. (2024a).** The interval threshold is fixed at $t_\delta = 0.2$. Values of $p > 0.5$ progressively shift the network's training capacity toward the data manifold ($t \to 1$), while $p < 0.5$ biases the distribution toward the prior noise.

Static Beta Timestep Sampling

A subsequent study by Zheng et al. (2024b) approaches the sampling problem from a complementary perspective, corroborating the finding that distributional variations during the generative process are highly asymmetric and predominantly concentrated near the data manifold. Rather than evaluating the Wasserstein distance, their theoretical analysis focuses on the KL divergence between the evolving marginal distribution $p_t(x_t)$ and the Gaussian prior $p_0(x_0) = \mathcal{N}(0, I)$. Specifically, they prove that the upper bound of this KL divergence decays exponentially as the distribution evolves away from the data manifold. Adapted to the standard Flow Matching

notation, where $t \in [0, 1]$ with the data manifold residing at $t = 1$, this decay is governed by the inequality

$$\mathrm{KL}(p_t(x_t) \parallel \mathcal{N}(0, I)) \leq e^{-(1-t)}\mathrm{KL}(p_1(x_1) \parallel \mathcal{N}(0, I)). \tag{3.7}$$

This exponential relationship formally establishes that the most significant distributional shifts occur near the end of the generative trajectory as the latent distribution approaches the data manifold, whereas the initial dynamics near the Gaussian prior remain relatively flat and stable (Zheng et al., 2024b).

To formulate a continuous sampling strategy that reflects this rapid decay near the data manifold, Zheng et al. (2024b) utilize the Beta distribution. The Beta distribution provides a highly flexible parametric family defined on the interval $[0, 1]$, characterized by the probability density function

$$f(t; \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} t^{\alpha-1}(1 - t)^{\beta-1}, \tag{3.8}$$

where $\Gamma(\cdot)$ denotes the Gamma function. The shape hyperparameters α and β regulate the concentration of the probability mass: increasing α shifts the density toward the right boundary ($t \to 1$), whereas increasing β shifts the density toward the left boundary ($t \to 0$). Furthermore, parameter values strictly less than 1 induce a singularity at their respective boundaries.

To explicitly align the sampling density with the theoretically identified asymmetric distribution dynamics, Zheng et al. (2024b) restrict the parameter space to $\alpha = 1$ and $\beta < 1$, translated here to the standard Flow Matching convention used throughout this thesis. This configuration induces a J-shaped, strictly monotonically increasing density function that heavily prioritizes regions of the trajectory immediately adjacent to the data manifold, where the distribution undergoes its most rapid changes (see Figure 3.4). Empirically, they identify the parameter combination of $\alpha = 1$ and $\beta = 0.8$ as optimal for capturing the asymmetric distributional dynamics.

Applying this Beta-distributed timestep sampling, formalized for DDPMs as the Beta-Tuned Timestep Diffusion Model (B-TTDM), yields substantial computational and performance gains compared to standard uniform timestep sampling. Zheng et al. (2024b) report up to a 4.71-fold acceleration in training convergence on large-scale datasets such as ImageNet, while simultaneously achieving higher sample fidelity.

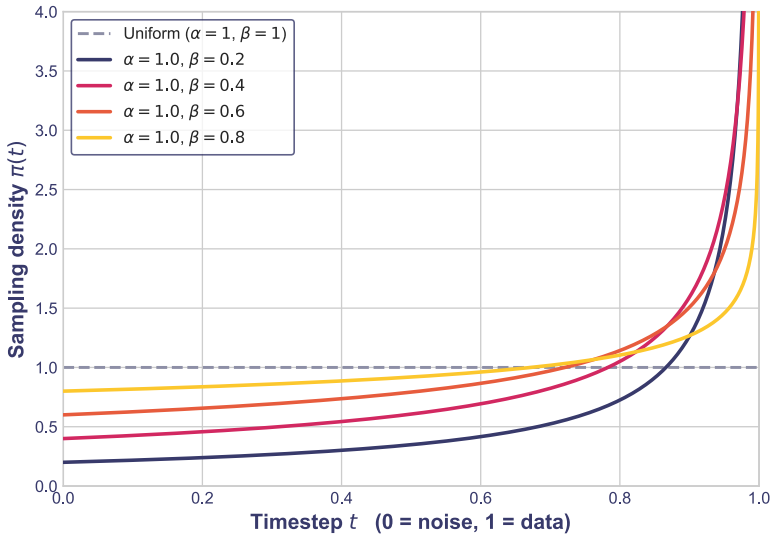


Figure 3.4: **Probability density functions for Beta-Tuned timestep sampling, adapted to the continuous Flow Matching temporal domain.** By fixing $\alpha = 1.0$ and decreasing the β parameter, a strictly monotonically increasing, J-shaped distribution is induced, asymptotically prioritizing the final trajectory near the data manifold.

4 Adaptive Timestep Sampling Methods

The existing approaches to improve training efficiency, presented in Chapter 3, often rely on static loss weighting or rigid, predefined non-uniform timestep sampling distributions. While these methods can emphasize specific noise regimes, they fail to account for the dynamic nature of the training process, where the difficulty of different timesteps can shift as the model converges.

To overcome the limitations of static sampling, this chapter introduces the core methodology of this thesis, which focuses on dynamically learning the optimal timestep sampling distribution during training. Section 4.1 formally introduces the foundation of this approach, namely the ATS framework originally proposed by Kim et al. (2025a).

Building upon this foundational framework, we subsequently propose two novel extensions designed to stabilize and accelerate the dynamic learning process. Section 4.2 introduces an Advantage Actor-Critic (A2C) formulation to reduce policy gradient variance. Finally, Section 4.3 reevaluates the solely training-focused adaptation objective and proposes a novel inference-oriented, curvature-aware reward signal based on the local truncation error (LTE). The conceptual foundations and implementation details for both ATS variations are detailed in the following sections. Additionally, we sketch the theoretical motivations behind these methods to justify their design. An empirical evaluation of these methods is subsequently provided in Chapter 5.

4.1 Adaptive Timestep Sampling

The core methodology for the optimization of training efficiency that we investigate in this thesis is the ATS framework by Kim et al. (2025a). It was originally developed and theoretically grounded in the context of discrete diffusion models. Therefore, we first detail its core concepts in the foundational formulation for diffusion models. Subsequently, we present the mathematical adaptations necessary to translate these concepts into the continuous-time domain of Flow Matching models.

4.1.1 The Adaptive Timestep Sampling Framework

Kim et al. (2025a) identify two critical observations regarding the training dynamics of diffusion models. First, they observe that gradient variance fluctuates significantly across different timesteps, creating a fundamental bottleneck for optimization. High gradient variance slows down the convergence of SGD optimizers, as the update directions become increasingly noisy

(Garrigos and Gower, 2024). Consequently, correcting these imprecise updates necessitates either a larger number of training steps or a reduced learning rate. Furthermore, the authors note that the highest gradient variance typically manifests at timesteps closer to the data regime ($t = 1$). Based on this, they conclude that uniform timestep sampling is inherently inefficient. It allocates equal computational resources to all noise regimes, thereby neglecting the higher optimization difficulty of these high-variance regimes.

Second, Kim et al. (2025a) observe that while statically oversampling specific timesteps successfully improves the training loss within those targeted regimes, it comes at a cost. They demonstrate that the timestep-conditional losses, $\mathcal{L}_t$, are not mutually independent. Disproportionately focusing training efforts on a specific timestep regime can lead to a severe degradation of the model's fit in undersampled areas. To resolve this trade-off, the authors propose a reinforcement learning-based approach that dynamically adapts the non-uniform timestep sampling distribution throughout the training process. By doing so, the framework continuously shifts the training focus toward the most difficult timesteps that contribute most significantly to the overall objective, thereby maximizing training efficiency without causing degradation in other regimes.

Adaptive Beta Sampling. To address these inefficiencies, Kim et al. (2025a) conceptually couple the timestep sampling distribution directly with the ultimate goal of minimizing the training objective. The core idea is to preferentially sample timesteps that yield gradient updates with the highest expected global loss reduction.

To achieve this without relying on prior training data or handcrafted heuristics, the adaptive sampler, denoted as π_{ϕ_k}, is parameterized as a lightweight neural network. This network processes an input image x_1 and outputs two positive real scalars, α and β. These scalars serve as the shape parameters for a Beta distribution, from which the subsequent timestep t is drawn. The high flexibility of the Beta distribution allows the sampler to adopt a wide variety of sampling shapes, as illustrated in Figure 4.1, and to adaptively shift its focus as training progresses. The theoretical objective is to optimize the sampler's parameters ϕ_k at training step k to maximize the expected loss reduction, expressed as

$$\phi_k^* = \arg\max_{\phi_k} \mathbb{E}_{x_1, x_0, t \sim \pi_{\phi_k}(\cdot|x_1)} \left[\Delta_k^t \right], \tag{4.1}$$

where Δ_k^t denotes the global loss reduction over all diffusion timesteps achieved by taking an optimization step at the sampled timestep t.

Approximation of Expected Loss Reduction. The crux of this method lies in reliably estimating the expected global loss reduction Δ_k^t. This estimation is highly complex due to the sequential nature of neural network optimization: each gradient update alters the current model parameters, making the updates of each SGD iteration dependent on the entire history of previous steps.

To render the problem computationally feasible, Kim et al. (2025a) restrict the evaluation to a greedy loss reduction, considering only the immediate impact of the next parameter update. The local loss reduction at a specific evaluation timestep $\tau \in \{1, \ldots, T\}$, caused by a gradient step taken at the sampled timestep t, is defined as

$$\delta_{k,\tau}^t = \mathcal{L}_{\theta_k}(x_1, \tau) - \mathcal{L}_{\theta_k - \gamma \nabla_{\theta_k} \mathcal{L}(x_1, t)}(x_1, \tau), \tag{4.2}$$

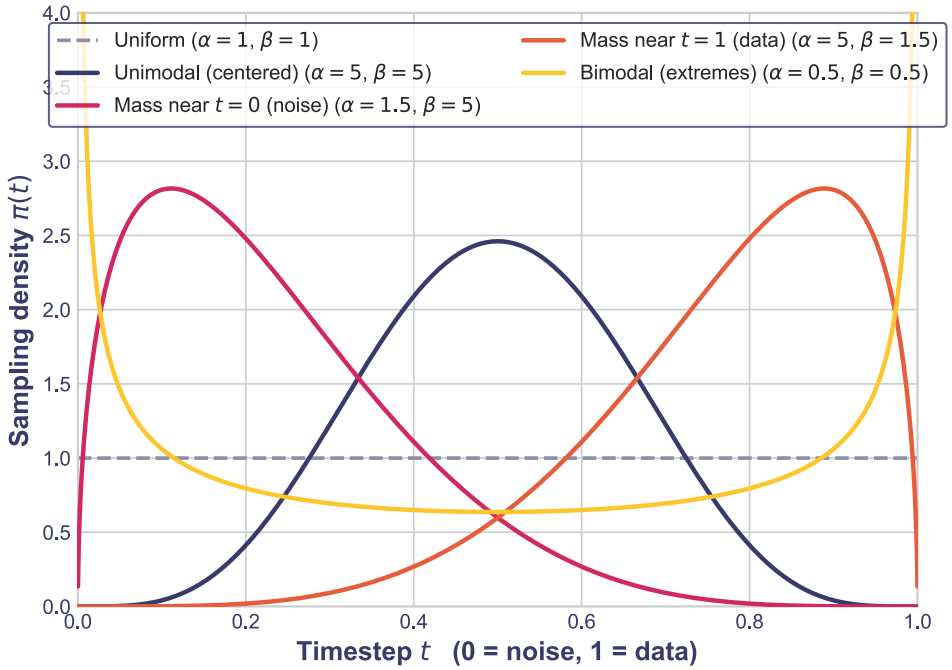


Figure 4.1: **Probability density functions of the Beta distribution.** The high flexibility of the Beta distribution enables the adaptive sampler to adopt a wide variety of timestep sampling strategies across the interval $t \in [0, 1]$. By dynamically updating the shape parameters α and β, the distribution can seamlessly transition between a uniform baseline ($\alpha = 1, \beta = 1$), unimodal distributions centered on intermediate timesteps ($\alpha = \beta > 1$), bimodal shapes emphasizing extremes ($\alpha < 1, \beta < 1$), or heavily skewed distributions favoring either early noise regimes ($\alpha < \beta$) or the data regime ($\alpha > \beta$).

where γ is the learning rate and θ_k represents the current model weights.

To determine the global average impact of the gradient update, Δ_k^t, one must average the local reductions $\delta_{k,\tau}^t$ over all possible discrete diffusion timesteps T, yielding

$$\Delta_k^t = \frac{1}{T}\sum_{\tau=1}^{T} \delta_{k,\tau}^t = \frac{1}{T}\sum_{\tau=1}^{T} \Big(\mathcal{L}_{\theta_k}(x_1,\tau) - \mathcal{L}_{\theta_k - \gamma\nabla_{\theta_k}\mathcal{L}(x_1,t)}(x_1,\tau)\Big). \tag{4.3}$$

However, naively computing this exact average would necessitate $2 \times T$ forward passes for every single policy update. This immense computational overhead would immediately offset any efficiency gains achieved by non-uniform sampling. To overcome this challenge, the authors introduce a second approximation: by leveraging the strong correlation of the losses $\mathcal{L}_t$ between adjacent timesteps, Δ_k^t can be accurately approximated using a significantly smaller subset of timesteps $S \subset \{1,\dots,T\}$ with $|S| \ll T$ that best explains the global loss reduction over all timesteps.

By evaluating the loss reduction only at the selected representative timesteps $\tau' \in S$, the computational burden shrinks to $2 \times |S|$ forward passes. The arithmetic mean of the loss reductions at these $|S|$ evaluation points acts as a highly efficient, tractable proxy, defined as $\tilde{\Delta}_k^t$, replacing the true global average across all $\tau \in \{1,\dots,T\}$. This final approximation is formally expressed as

$$\begin{aligned}\Delta_k^t &= \frac{1}{T}\sum_{\tau=1}^{T} \Big(\mathcal{L}_{\theta_k}(x_1,\tau) - \mathcal{L}_{\theta_k - \gamma\nabla_{\theta_k}\mathcal{L}(x_1,t)}(x_1,\tau)\Big) \\ &\approx \frac{1}{|S|}\sum_{\tau'\in S} \Big(\mathcal{L}_{\theta_k}(x_1,\tau') - \mathcal{L}_{\theta_k - \gamma\nabla_{\theta_k}\mathcal{L}(x_1,t)}(x_1,\tau')\Big) =: \tilde{\Delta}_k^t.\end{aligned} \tag{4.4}$$

Queue-Based Feature Selection. To dynamically identify this most informative subset of timesteps S, the framework employs a queue-based feature selection mechanism. In each optimization step, a single probe image x_{probe} is sampled from the current training batch. To evaluate the gradient impact across all noise levels, this probe image is replicated into a new evaluation batch of size T, where each replica is paired with a distinct discrete timestep $\tau \in \{1,\dots,T\}$.

The backbone loss $\mathcal{L}(x_{\text{probe}},\tau)$ is calculated for all T replicas both immediately before and after applying the gradient update derived from the sampled timestep t. The difference between these two states yields a complete vector of T local loss reductions. To build a robust statistical basis without excessive per-step computation, these vectors are accumulated in a rolling memory queue of a fixed capacity $|Q|$. In each iteration, the vector of local reductions $\delta_{k,\tau}^t$ is appended as a new row to a feature matrix $X \in \mathbb{R}^{|Q|\times T}$, while their exact mean, the true global loss reduction Δ_k^t, is appended to the queue's target vector $y \in \mathbb{R}^{|Q|}$. Explicitly, for a full queue containing $|Q|$ past evaluations, these structures take the form

$$X = \begin{bmatrix} \delta_{1,1} & \delta_{1,2} & \dots & \delta_{1,T} \\ \delta_{2,1} & \delta_{2,2} & \dots & \delta_{2,T} \\ \vdots & \vdots & \ddots & \vdots \\ \delta_{|Q|,1} & \delta_{|Q|,2} & \dots & \delta_{|Q|,T} \end{bmatrix}, \quad y = \begin{bmatrix} \Delta_1 \\ \Delta_2 \\ \vdots \\ \Delta_{|Q|} \end{bmatrix}, \tag{4.5}$$

where each row index $q \in \{1, \ldots, |Q|\}$ represents a distinct training step stored in the memory buffer. By maintaining this sliding window, the queue ensures that the feature selection relies on a sufficiently large sample size and up-to-date representation of the training dynamics, effectively compensating for the estimation error introduced by evaluating only a single probe image per iteration.

Once the queue is sufficiently populated, a univariate F-regression is executed between the feature matrix X and the target vector y from the buffer. This yields an F-statistic for each evaluation timestep τ, computed as

$$F_\tau = \frac{r_\tau^2}{1 - r_\tau^2} \cdot (|Q| - 2), \tag{4.6}$$

where r_τ denotes the Pearson correlation coefficient between the local loss reductions at timestep τ (the τ-th column of X) and the global reductions (the vector y). Finally, the algorithm selects the $|S|$ timesteps that exhibit the highest F-statistics to form the subset S. Based on the recent history captured in the queue, the local loss reductions $\delta_{k,\tau'}^t$ at these specifically selected timesteps act as the optimal proxies for the true global loss reduction Δ_k^t.

REINFORCE Update of the Policy Network. The theoretical objective of the adaptive sampler is to find the parameters ϕ_k^* that maximize the expected global loss reduction at training step k. By substituting the true global loss reduction Δ_k^t with the tractable proxy $\tilde{\Delta}_k^t$ established above, the objective is formulated as

$$\begin{aligned} \phi_k^* &= \arg\max_{\phi_k} \mathbb{E}_{x_1, x_0, t \sim \pi_{\phi_k}(\cdot|x_1)} \left[\Delta_k^t \right] \\ &\approx \arg\max_{\phi_k} \mathbb{E}_{x_1, x_0, t \sim \pi_{\phi_k}(\cdot|x_1)} \left[\tilde{\Delta}_k^t \right]. \end{aligned} \tag{4.7}$$

In conventional neural networks, parameters are iteratively refined via standard backpropagation. However, in Equation (4.7), the specific value of the sampled timestep t inherently depends on the policy parameters ϕ_k, as it is drawn stochastically from a Beta distribution predicted by the network. Consequently, for the network to learn, the gradient ∇_{ϕ_k} would need to flow backward through this stochastic sampling operation ($t \sim \pi_{\phi_k}$). Because standard gradients cannot propagate through non-deterministic nodes in a computational graph, direct optimization via backpropagation is impossible. Note that this bottleneck strictly affects the t sampling operation; the selection of the input image x_1 and the prior Gaussian noise x_0 are entirely independent of the policy parameters.

To bypass this non-differentiable sampling bottleneck, Kim et al. (2025a) frame the optimization as a reinforcement learning problem and apply the REINFORCE algorithm, introduced in Section 2.3.2. In this context, the approximated expected loss reduction acts as the empirical reward signal ($R = \tilde{\Delta}_k^t$). Because the backbone Flow Matching model evaluates this loss reduction independently of the current policy parameters ϕ_k, the fundamental REINFORCE assumption, $\nabla_{\phi_k} \tilde{\Delta}_k^t = 0$, is satisfied. In this vanilla ATS formulation, the reinforcement baseline is trivially set to zero ($b = 0$).

To prevent the predicted Beta distribution from prematurely collapsing into a deterministic delta function, entropy regularization (see Section 2.3.2) must be incorporated. Originally, Kim et al.

(2025a) implemented this by adding the analytical differential entropy $\mathcal{H}[\pi_{\phi_k}]$ directly to the empirical reward signal. While this continuous mathematical formulation is analytically sound, substituting it directly into a PyTorch REINFORCE update equation violates the core assumption of a parameter-independent reward, since $\nabla_\phi\big(\tilde{\Delta}_k^t + \eta\mathcal{H}[\pi_\phi]\big) \neq 0$. Because the analytical entropy depends on the policy parameters, automatic differentiation mechanically applies the product rule. This unintentionally scales the true entropy gradient by a state-dependent weighting factor. Although this warped gradient still pushes the policy in the correct general direction, it yields a more unstable gradient estimator that injects unnecessary noise into the policy adaptation trajectory. A detailed mathematical proof of this discrepancy and its structural consequences is provided in Appendix A.1.

To resolve this mathematical instability and ensure sustained exploration, we explicitly decouple the entropy from the environmental reward. Instead of treating it as a stochastic environmental signal, we implement it as a structural, analytically differentiable additive term, aligning with established regularized policy optimization procedures (Mnih et al., 2016). The overall optimization objective $J(\phi_k)$ is thus to maximize the expected reward alongside the policy entropy, defined as

$$J(\phi_k) = \underbrace{\mathbb{E}_{x_1,x_0,t\sim\pi_{\phi_k}(\cdot|x_1)}\left[\tilde{\Delta}_k^t\right]}_{\text{Expected reward}} + \underbrace{\eta\cdot\mathbb{E}_{x_1}\big[\mathcal{H}[\pi_{\phi_k}(\cdot|x_1)]\big]}_{\text{Entropy regularization}}, \tag{4.8}$$

where η is a hyperparameter scaling the exploration bonus.

To optimize this objective, we apply the Leibniz integral rule and the log-derivative trick established in Section 2.3.2. This elegantly transforms the intractable gradient of the expected reward into a tractable expectation of the gradient of the log-Beta distribution, yielding

$$\nabla_{\phi_k} J(\phi_k) = \mathbb{E}_{x_1,x_0,t\sim\pi_{\phi_k}}\left[\tilde{\Delta}_k^t \cdot \nabla_{\phi_k}\log\pi_{\phi_k}(t|x_1)\right] + \eta\,\mathbb{E}_{x_1}\big[\nabla_{\phi_k}\mathcal{H}[\pi_{\phi_k}(\cdot|x_1)]\big]. \tag{4.9}$$

In practice, computing this exact expectation over the typically high-dimensional continuous data and noise distributions is intractable. Instead, we estimate it via Monte Carlo simulation. By sampling a batch of B clean training images $x_1^{(i)}$, prior Gaussian noise terms $x_0^{(i)}$, and timesteps $t^{(i)} \sim \pi_{\phi_k}$, we calculate the empirical batch mean gradient as

$$\widehat{\nabla}_{\phi_k} J(\phi_k) = \frac{1}{B}\sum_{i=1}^{B}\Big(\tilde{\Delta}_k^{t^{(i)}} \cdot \nabla_{\phi_k}\log\pi_{\phi_k}(t^{(i)}|x_1^{(i)}) + \eta\nabla_{\phi_k}\mathcal{H}\big[\pi_{\phi_k}(\cdot|x_1^{(i)})\big]\Big). \tag{4.10}$$

As guaranteed by the core REINFORCE theorem, this Monte Carlo approximation provides an unbiased estimate of the true performance gradient. This allows us to define the stochastic parameter update rule as

$$\phi_{k+1} = \phi_k + \gamma_\pi \widehat{\nabla}_{\phi_k} J(\phi_k), \tag{4.11}$$

where γ_π represents the learning rate of the policy network. This REINFORCE update shifts the policy weights on average in the direction of higher expected loss reduction.

Fundamentally, the REINFORCE algorithm is formulated as a reward maximization problem. However, modern deep learning frameworks are architected to perform SGD via loss minimization. To execute our Monte Carlo simulation within a standard backpropagation pipeline using

the sampled training batch, the empirical optimization objective is reframed as

$$\arg\min_{\phi_k} L_\pi(\phi_k) \equiv \arg\max_{\phi_k} \widehat{J}(\phi_k). \tag{4.12}$$

To achieve this, we define a negative surrogate policy loss whose analytical gradient exactly matches the negative empirical REINFORCE estimate, ensuring $-\nabla_{\phi_k} L_\pi(\phi_k) = \widehat{\nabla}_{\phi_k} J(\phi_k)$. This surrogate loss is given by

$$L_\pi(\phi_k) = -\frac{1}{B}\sum_{i=1}^{B}\Big(\tilde{\Delta}_k^{t^{(i)}} \cdot \log \pi_{\phi_k}(t^{(i)}|x_1^{(i)}) + \eta \cdot \mathcal{H}\big[\pi_{\phi_k}(\cdot|x_1^{(i)})\big]\Big). \tag{4.13}$$

By defining the problem this way, the computation graph of the policy network becomes fully deterministic. Minimizing this surrogate policy loss via standard backpropagation automatically executes the intended REINFORCE policy update directly through the sampled SGD batch.

Finally, to further limit the computational overhead of the adaptive timestep sampling, the calculation of the proxy $\tilde{\Delta}_k^t$ and the subsequent REINFORCE policy update are not performed at every iteration. Instead, the policy network is updated only once every f_S backbone gradient steps. The complete algorithmic sequence of a policy update iteration is summarized in Figure 4.2.

4.1.2 Transfer to Flow Matching

To apply the ATS framework developed for diffusion models to Flow Matching, we must translate it from its original discrete diffusion formulation into the continuous-time paradigm of Flow Matching models. This transition requires addressing two fundamental structural differences: the shift in the underlying backbone objective and the transition from a discrete timestep index to a continuous time interval.

Accommodating the Flow Matching Objective. In the original diffusion-based ATS, the evaluated loss $\mathcal{L}_{\theta_k}(x_1, \tau)$ typically represents the mean squared error in predicting the added Gaussian noise x_0 at a discrete noise scale. Because the ATS framework is fundamentally loss-agnostic, we simply swap the underlying objective without altering the algorithm itself. For Flow Matching, the backbone model is instead trained to minimize the mean squared error of the approximation of a continuous velocity vector field defined by a linear interpolant between prior noise and the image data distribution. Consequently, the interpretation changes and the local loss reduction $\delta_{k,\tau}^t$ simply evaluates the change in this vector-field matching error caused by a gradient step taken at the sampled continuous time $t \in [0, 1]$.

Continuous-Time Formulation of Expected Loss Reduction. The more significant theoretical adjustment arises from the time domain itself. Because Flow Matching operates over a continuous interval $t \in [0, 1]$, the global average impact of a gradient update across all possible noise levels can no longer be expressed as a finite sum over T discrete steps (as defined in Equation (4.3)). Instead, the true global loss reduction Δ_k^t becomes an integral over the continuous time domain:

$$\Delta_k^t = \int_0^1 \Big(\mathcal{L}_{\theta_k}(x_1, \tau) - \mathcal{L}_{\theta_k - \gamma\nabla_{\theta_k}\mathcal{L}(x_1,t)}(x_1, \tau)\Big) d\tau. \tag{4.14}$$

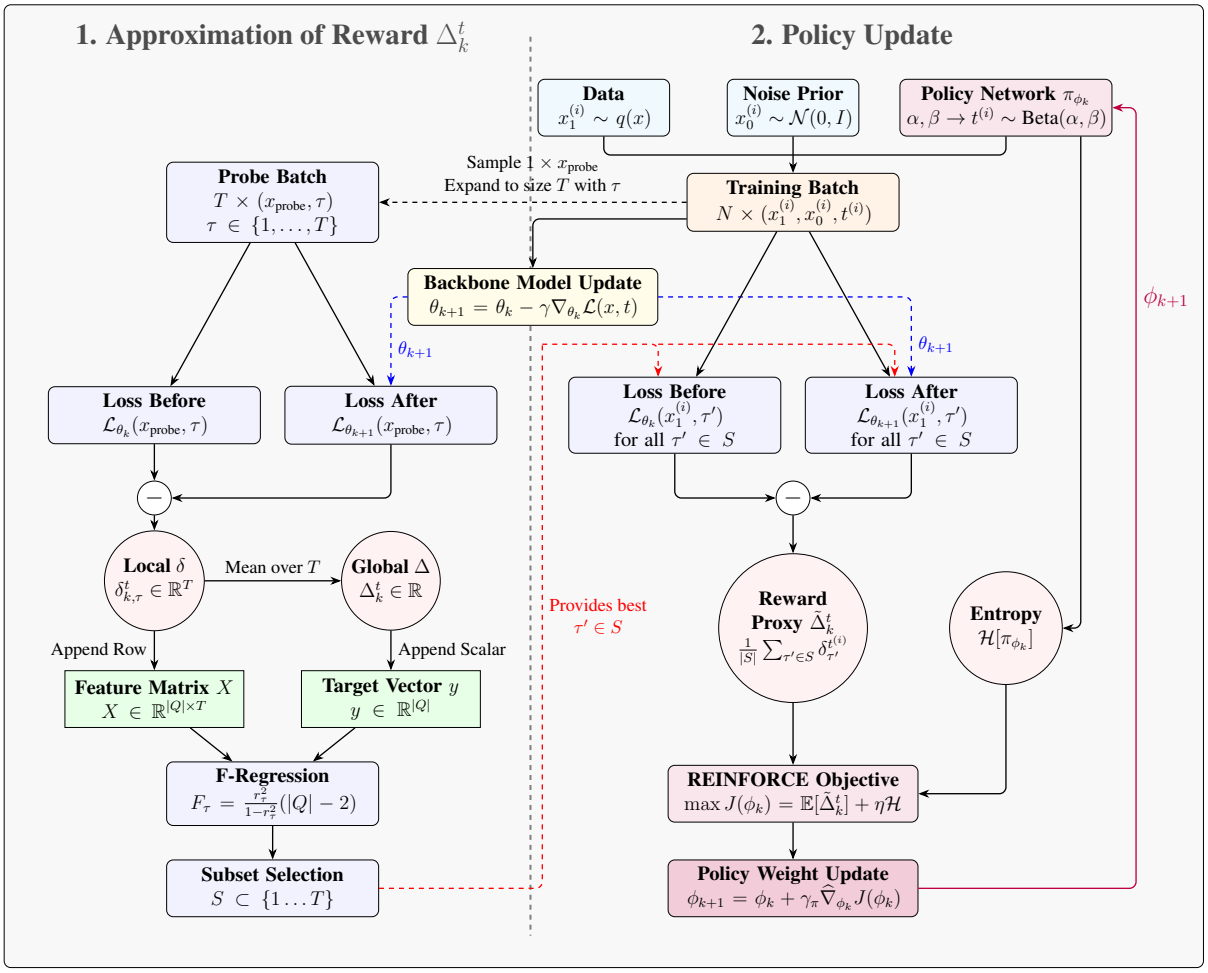


Figure 4.2: **Overview of the Adaptive Timestep Sampling Process.** The diagram illustrates the interplay between the backbone Flow Matching model and the policy network. The policy network generates a Beta distribution from which a timestep is sampled, while the backbone loss reduction over a subset of timesteps provides the reward to update the policy via REINFORCE.

However, just as computing the sum over T evaluation points was computationally intractable for diffusion models, analytically evaluating or numerically integrating this continuous function at every optimization step is practically infeasible.

To overcome this, we discretize the continuous interval $[0, 1]$ into a uniform grid of T diffusion-like discrete evaluation points. This allows us to reapply the queue-based feature selection mechanism to track the local loss reductions at these T anchor points. Subsequently, we can run the F-regression to select a highly informative subset S (where $|S| \ll T$). The integral is thus approximated by the arithmetic mean of the loss reductions at these $|S|$ proxy timesteps:

$$\begin{aligned}\Delta_k^t &= \int_0^1 \Big(\mathcal{L}_{\theta_k}(x_1, \tau) - \mathcal{L}_{\theta_k - \gamma \nabla_{\theta_k} \mathcal{L}(x_1,t)}(x_1, \tau)\Big) d\tau \\ &\approx \frac{1}{|S|} \sum_{\tau' \in S} \Big(\mathcal{L}_{\theta_k}(x_1, \tau') - \mathcal{L}_{\theta_k - \gamma \nabla_{\theta_k} \mathcal{L}(x_1,t)}(x_1, \tau')\Big) =: \tilde{\Delta}_k^t. \end{aligned} \tag{4.15}$$

While the initial discretization of the continuous interval into T bins introduces a numerical integration error, we expect this to be negligible in practice. The restriction of the loss landscape evaluation to the drastically reduced subset S dominates the approximation error. Therefore, treating the discretized points identically to discrete diffusion steps is not expected to meaningfully worsen the quality of the proxy $\tilde{\Delta}_k^t$.

Once the tractable proxy $\tilde{\Delta}_k^t$ is calculated, the remainder of the framework requires no further modification. The REINFORCE policy update, the entropy regularization, and the parameterization of the Beta distribution $\pi_{\phi_k}(t|x_1)$ proceed exactly as formulated for diffusion models. The Flow Matching framework simply serves as the continuous environment that provides the empirical reward signal for the adaptive timestep sampler.

4.2 Advantage Actor-Critic Adaptive Timestep Sampling

Building upon the vanilla ATS framework, this section introduces an architectural refinement designed to stabilize and accelerate the policy adaptation during training. We first argue that heterogeneous sample difficulty injects avoidable variance into the policy gradient. We then propose an A2C formulation that centers the reward signal with a learned value baseline, and show that this modification leaves the gradient estimate unbiased while reducing its variance. Finally, we sketch why a lower-variance policy gradient should translate into faster convergence of both the policy and the generative backbone.

Sample Difficulty as a Confounder of the Reward. In the vanilla ATS formulation, the policy network is updated without a reinforcement baseline ($b = 0$), so the policy gradient is scaled by the absolute magnitude of the loss-reduction proxy $\tilde{\Delta}_k^t$. We conjecture that this magnitude is driven to a substantial extent by the inherent learning difficulty of the individual training sample rather than by the quality of the chosen timestep. A low-complexity image dominated by uniform regions, such as a clear sky or a homogeneous texture, plausibly induces velocity targets that the backbone can fit within few updates, so a single gradient step may already produce a large measurable loss drop and hence a large reward. A structurally complex image, in contrast, may improve only incrementally, yielding a small reward regardless of which timestep the policy

selected. If this conjecture holds, the policy is rewarded or penalized for the difficulty of the image it happened to draw rather than for its timestep choice. Such a dynamic could drive the network to prefer timestep distributions that facilitate learning easy images while neglecting noise regimes critical for learning complex data.

From a statistical perspective, this manifests as additional policy gradient variance. Let $\hat{g}$ denote the policy gradient estimator with respect to parameters ϕ_k. Because $\hat{g}$ is a vector, its variance is technically a covariance matrix. For convergence analysis, we investigate its trace, $\mathrm{Tr}(\mathrm{Cov}(\hat{g})) = \mathbb{E}[\|\hat{g}\|^2] - \|\mathbb{E}[\hat{g}]\|^2$. Applying the law of total covariance and the linearity of the trace operator, this metric decomposes across the batch of images x_1 as

$$\mathrm{Tr}(\mathrm{Cov}(\hat{g})) = \underbrace{\mathbb{E}_{x_1}\left[\,\mathrm{Tr}(\mathrm{Cov}_{t,x_0}(\hat{g} \mid x_1))\right]}_{\text{stochastic variance}} + \underbrace{\mathrm{Tr}\left(\,\mathrm{Cov}_{x_1}\left(\mathbb{E}_{t,x_0}[\hat{g} \mid x_1]\right)\right)}_{\text{batch variance}} \tag{4.16}$$

into two structurally different components. The *stochastic variance* arises from the random draws of the scalar timestep t and the source noise vector x_0 for a fixed image. The *batch variance* instead captures how strongly the expected true gradient fluctuates across the different images of a batch. While the batch variance is a property of the objective itself, the massive disparity in reward magnitudes between easy and hard images inflates the uncentered second moment of the gradient for fixed images, drastically inflating the stochastic variance. A common remedy from the reinforcement learning literature is to center the reward with a state-dependent baseline (Williams, 1992; Konda and Tsitsiklis, 1999), a concept introduced in Section 2.3.2. The A2C-ATS formulation instantiates exactly this remedy for the ATS setting.

Value Network and Advantage Formulation. We introduce an auxiliary *value network* (the Critic), parameterized by ψ and denoted $V_\psi(x_1)$. Its purpose is to predict the expected reward for a given image, marginalizing out the stochasticity of the sampled timestep and the source noise, such that $V_\psi(x_1) \approx \mathbb{E}_{t,x_0}\left[\tilde{\Delta}_k^t \mid x_1\right]$. The Critic is trained concurrently with the policy network by minimizing the mean squared error against the empirically observed loss reductions of the training batch, given by

$$\mathcal{L}_{\text{Value}}(\psi) = \frac{1}{B}\sum_{i=1}^{B}\left(V_\psi(x_1^{(i)}) - \tilde{\Delta}_k^{t^{(i)}}\right)^2. \tag{4.17}$$

Using the Critic's prediction as a dynamic, state-dependent baseline $b = V_\psi(x_1)$, the policy network (the Actor) is optimized with the advantage $A(t, x_1) = \tilde{\Delta}_k^t - V_\psi(x_1)$ in place of the raw reward. Integrating this into the entropy-regularized surrogate objective established in Section 4.1, the A2C update rule for the policy parameters becomes

$$\phi_{k+1} = \phi_k + \gamma_\pi\Big(\tilde{\Delta}_k^t - V_\psi(x_1)\Big)\nabla_{\phi_k}\log\pi_{\phi_k}(t|x_1) + \eta\nabla_{\phi_k}\mathcal{H}[\pi_{\phi_k}]. \tag{4.18}$$

Unbiasedness of the A2C Policy Gradient. Replacing the raw reward with the advantage does not distort the learning signal. Because the Critic conditions only on the image and never on the sampled timestep, the baseline satisfies the conditional-independence assumption of the

REINFORCE guarantee established by Williams (1992). Applied to the ATS setting, the baseline contribution to the expected gradient factors out of the expectation and vanishes, since

$$\mathbb{E}_{t\sim\pi_{\phi_k}}\Big[V_\psi(x_1)\nabla_{\phi_k}\log\pi_{\phi_k}(t|x_1)\Big] = V_\psi(x_1)\nabla_{\phi_k}\underbrace{\int_0^1 \pi_{\phi_k}(t|x_1)\,dt}_{=1} = 0. \tag{4.19}$$

The expected conditional gradient for any specific image remains identical: $\mathbb{E}_{t,x_0}[\hat{g}_{\mathrm{A2C}} \mid x_1] = \mathbb{E}_{t,x_0}[\hat{g}_{\mathrm{ATS}} \mid x_1]$. Consequently, the overall expectation is preserved ($\mathbb{E}[\hat{g}_{\mathrm{A2C}}] = \mathbb{E}[\hat{g}_{\mathrm{ATS}}]$) and the policy gradient remains perfectly unbiased.

Reduction of the Policy Gradient Variance. Because the conditional expectations are identical, the state-dependent baseline leaves the *batch variance* term of Equation (4.16) mathematically untouched. The variance reduction occurs entirely within the *stochastic variance* term. Letting $S_k(t) = \nabla_{\phi_k}\log\pi_{\phi_k}(t|x_1)$ represent the score function, the trace of the stochastic covariance for a fixed image x_1 under the A2C estimator is thus given by

$$\mathrm{Tr}\left(\mathrm{Cov}_{t,x_0}(\hat{g}_{\mathrm{A2C}} \mid x_1)\right) = \mathbb{E}_{t,x_0}\Big[\big(\tilde{\Delta}_k^t - V_\psi(x_1)\big)^2\|S_k(t)\|^2 \mid x_1\Big] - \big\|\mathbb{E}_{t,x_0}[\hat{g}_{\mathrm{ATS}} \mid x_1]\big\|^2. \tag{4.20}$$

As the second term is identical for A2C-ATS and ATS, reducing the total trace of the gradient covariance relies entirely on reducing the uncentered second moment represented by the first term (Weaver and Tao, 2001).

Under the vanilla ATS estimator, this uncentered second moment is simply $\mathbb{E}_{t,x_0}\big[(\tilde{\Delta}_k^t)^2\|S_k(t)\|^2 \mid x_1\big]$. For an inherently "easy" image, the raw reward magnitude $(\tilde{\Delta}_k^t)^2$ is massive, forcing the stochastic variance for that specific image draw to explode. By introducing the baseline, the A2C formulation absorbs this inherent image-dependent magnitude in the advantage. Assuming the reward is largely determined by the image difficulty, a perfect value approximation $V_\psi(x_1) \approx \tilde{\Delta}_k^t$ collapses the advantage, driving the uncentered second moment toward zero:

$$\mathbb{E}_{t,x_0}\Big[\big(\tilde{\Delta}_k^t - V_\psi(x_1)\big)^2\|S_k(t)\|^2 \mid x_1\Big] \approx 0. \tag{4.21}$$

By eliminating the scaling factor induced by easy images, the baseline shrinks the stochastic variance of the gradient estimator. In expectation, this directly yields a smaller total gradient covariance trace for the A2C batch estimator compared to vanilla ATS:

$$\mathrm{Tr}(\mathrm{Cov}(\hat{g}_{\mathrm{A2C}})) < \mathrm{Tr}(\mathrm{Cov}(\hat{g}_{\mathrm{ATS}})). \tag{4.22}$$

We emphasize that this reduction is conditional: it materializes only to the degree that the Critic successfully attains the approximation $V_\psi(x_1) \approx \mathbb{E}_{t,x_0}[\tilde{\Delta}_k^t \mid x_1]$, a requirement that is not inherently guaranteed under the non-stationary reward scale of Flow Matching training and will be investigated in Section 5.2.

Accelerated Backbone Convergence via Precise Policy Tracking. The practical value of this variance reduction is that less noisy policy updates yield a better timestep sampling distribution at every backbone update step. Because the backbone parameters θ_k change at each step, the reward landscape $\Delta(t;\theta_k)$ drifts and the policy must continuously track a moving optimal

distribution $\pi^*_{\theta_k} = \arg\max_\pi \mathbb{E}_{x_1,x_0,t}[\Delta(t;\theta_k)]$. How closely a stochastic gradient method can follow this optimum is limited by the noise of its gradient estimate. Convergence analyses of constant-stepsize SGD show that the iterates do not settle on the optimum but fluctuate around it within a neighborhood whose radius grows with the gradient variance (Garrigos and Gower, 2024), and when the optimum additionally drifts the same mechanism bounds how tightly it can be tracked. A policy with lower gradient variance therefore hovers closer to the momentarily optimal distribution and samples timesteps that are, on average, better aligned with the current high-reward region. Since the ATS reward is the immediate loss reduction of the backbone, these better-aligned timesteps extract larger loss reductions per update, and the accumulation of these per-step gains accelerates the decrease of the Flow Matching training loss. We investigate empirical differences in backbone convergence of the A2C-ATS and vanilla ATS in Section 5.2.

4.3 Curvature-Aware Adaptive Timestep Sampling

Both ATS and A2C-ATS optimize a purely *training-oriented* objective, since their reward is defined as the reduction of the Flow Matching training loss. This reward formulation, however, disregards a decisive aspect of generative modeling. At inference time, sample quality is driven largely by the error that accumulates while numerically integrating the learned vector field. At a fixed budget of function evaluations, this integration error is governed by the geometry of the learned probability path, because ODE solvers accumulate error wherever the trajectory is curved.

This observation motivates a second, *inference-oriented* reward. Instead of biasing training toward the timesteps with the highest expected loss reduction, we propose a policy that allocates training resources to the temporal regimes in which the generative trajectory is geometrically hardest to integrate. The remainder of this section derives such a curvature signal from numerical analysis, shows how it is estimated in practice during training, and integrates it as a new reward into the actor-critic sampling framework of Section 4.2.

The Local Truncation Error as a Cheap Curvature Proxy. In numerical analysis, the local truncation error denotes the deviation a solver introduces during a single integration step under the assumption that the previous state lay exactly on the true analytic trajectory. Since the analytic probability path of the generative ODE is intractable, the true LTE cannot be computed. Following standard practice for adaptive step-size ODE solvers (Karras et al., 2022; Iserles, 2009), we therefore approximate it by the discrepancy between a lower-order and a higher-order integration step. Concretely, we compare the linear Euler predictor $\tilde{x}_{t+\Delta t}$ with the curvature-aware Heun corrector $x_{t+\Delta t}$, both introduced in Section 2.1.3. Wherever the two steps disagree strongly, the trajectory violates the local linearity assumption of the Euler step, so their discrepancy acts as a direct scalar measure of the local geometric curvature of the generative trajectory. This construction is formalized in the following definition and illustrated in Figure 4.3.

Definition: Local Truncation Error (LTE) Proxy

For a neural vector field $v_\theta(x, t)$ integrated with step size Δt, the LTE proxy at state x_t and time t is defined as the squared L_2 distance between the second-order Heun corrector step $x_{t+\Delta t}$ and the first-order Euler predictor step $\tilde{x}_{t+\Delta t}$, given by

$$\widehat{\mathrm{LTE}}_\theta(x_t, t) = \|x_{t+\Delta t} - \tilde{x}_{t+\Delta t}\|_2^2 = \left\| \frac{\Delta t}{2} \Big(v_\theta(\tilde{x}_{t+\Delta t}, t + \Delta t) - v_\theta(x_t, t) \Big) \right\|_2^2. \quad (4.23)$$

This scalar approximates the true local truncation error and thereby the instantaneous curvature of the generative trajectory at time t.

A decisive practical advantage of this LTE proxy is its computational cost. Evaluating it requires exactly two forward passes of the backbone per policy update, one for each velocity evaluation, and no backward pass. This makes the LTE proxy considerably less expensive than the approximation of the loss-reduction in standard ATS. We confirm this cost advantage empirically in Chapter 5.

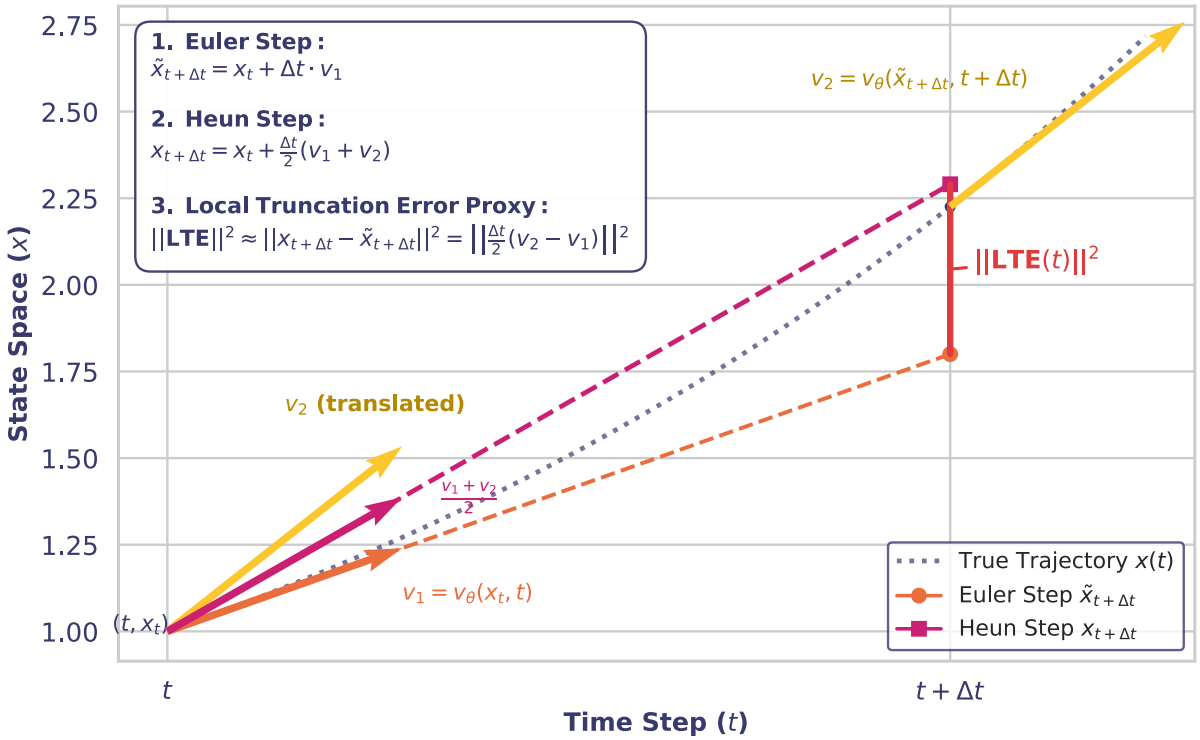


Figure 4.3: **Geometric interpretation of the Local Truncation Error (LTE) proxy.** The first-order Euler predictor $\tilde{x}_{t+\Delta t}$ extrapolates the state using the instantaneous velocity $v_\theta(x_t, t)$, assuming a strictly linear trajectory. The second-order Heun corrector evaluates the vector field at this predicted future state to compute a curvature-aware final update $x_{t+\Delta t}$. The squared magnitude of the spatial discrepancy between these two predictions yields the LTE proxy $\widehat{\mathrm{LTE}}_\theta$, a quantifiable scalar metric for the underlying geometric curvature of the ODE trajectory.

Curvature as Reinforcement Signal. We propose an adaptive timestep sampling strategy that uses this curvature proxy as the reward for the policy adaptation and term it *LTE-ATS*. LTE-ATS retains the complete actor-critic machinery of Section 4.2, including the Beta policy, the entropy regularization, and the value-network baseline, and only replaces the loss-reduction proxy $\tilde{\Delta}_k^t$ by the curvature reward

$$R^{\mathrm{LTE}}(t, x_1) = \widehat{\mathrm{LTE}}_\theta(x_t, t), \tag{4.24}$$

evaluated with the current backbone v_θ at the policy-sampled timestep t. Under this reward, the REINFORCE update shifts the sampling mass of the policy toward the timesteps at which the learned trajectory is currently most curved.

Practical Estimation on the Training Batch. The reward defined above still contains an intractable quantity. The state x_t lies on a generative trajectory that would have to be produced by explicitly integrating the ODE from the prior, which would incur a prohibitive computational overhead at every policy update. Our implementation therefore estimates the reward directly on the current training batch. For every image $x_1^{(i)}$ with policy-sampled timestep $t^{(i)}$, we draw fresh prior noise $x_0^{(i)} \sim \mathcal{N}(0, I)$ and form the interpolant point $z^{(i)} = \big(1 - t^{(i)}\big)x_0^{(i)} + t^{(i)}x_1^{(i)}$, at which the Euler predictor and the Heun corrector of the proxy are evaluated. The step size is tied to the inference configuration as $\Delta t = 1/N$, where N denotes the number of integration steps used by the sampler at inference time, so the proxy probes the curvature at exactly the temporal resolution at which the solver will later traverse the trajectory. To keep the corrector evaluation inside the time domain, the sampled timestep is clamped to the interval $[0, 1 - \Delta t]$. The empirical per-sample reward entering the policy update is thus

$$\widehat{R}^{\mathrm{LTE}}\big(t^{(i)}, x_1^{(i)}\big) = \widehat{\mathrm{LTE}}_\theta\big(z^{(i)}, t^{(i)}\big), \tag{4.25}$$

and each REINFORCE update averages over the B rewards of the current batch.

The Straightening Hypothesis. The fundamental hypothesis underlying LTE-ATS is that additional training in high-curvature temporal regimes reduces the curvature precisely there. If the backbone fits its velocity targets more accurately in the regimes where the trajectory is currently hardest to integrate, the learned marginal path should straighten, easing numerical integration and improving sample quality. Whether this hypothesis holds, that is, whether the additional *conditional* training reduces the *marginal* trajectory curvature, is examined empirically in Section 5.2.

5 Experiments

Having established the theoretical framework of Flow Matching and introduced our extensions to the ATS methodology, we now conduct a systematic empirical evaluation of the presented methods. The primary question of this chapter is whether the ATS framework (Kim et al., 2025a), originally designed for standard diffusion models, transfers to Flow Matching with CondOT probability paths, where it would have to accelerate convergence or improve output fidelity.

We approach this question from two sides. We first measure whether adaptive sampling delivers the promised gains over static and uniform baselines. We then turn from the sampler to the backbone and characterize the per-timestep loss and gradient landscapes of the CondOT objective, how quickly they form, and whether biasing the timestep distribution reshapes them or only the compute a sampler spends on each regime. A final analysis asks whether these training-time differences leave any measurable imprint on the geometry of the trajectories the trained models integrate at generation time. Our empirical analysis is accordingly structured into three targeted experiments.

First, however, Section 5.1 establishes the foundational experimental design. This section details the benchmark image dataset and underlying generative architecture employed, the training configurations, and the quantitative metrics utilized to evaluate performance. Following this setup, the empirical findings are presented across three targeted investigations:

- **E1 – Generation Quality and Training Efficiency** (Section 5.2). We analyze the three adaptive timestep sampling strategies and compare their generation quality and training cost against static non-uniform strategies and the standard uniform baseline.
- **E2 – Insights Into the Flow Matching Learning Process** (Section 5.3). Shifting the analytical focus from the sampler to the backbone, we read the per-timestep-bin loss and gradient statistics that training logs at every epoch. This characterizes where along the timestep axis the CondOT Flow Matching objective is hard, how quickly that structure is established, and whether biasing the timestep sampler toward the noise or the data regime reshapes it.
- **E3 – Impact of Timestep Distribution on Inference Trajectories** (Section 5.4). Finally, we investigate the geometric consequences of training with biased temporal distributions. This experiment analyzes whether biasing the training-time timestep distribution alters the geometry of the learned marginal vector field, measured through trajectory curvature, kinetic transport energy, and the numerical ODE integration error.

5.1 Experimental Setup

To systematically address the objectives of all three experiments, we conduct a single, unified comparison suite. The resulting training runs are subsequently analyzed through the three distinct analytical lenses defined by Experiments 1, 2, and 3.

This comparison suite benchmarks our adaptive sampling methods (ATS, A2C-ATS, and LTE-ATS) against the standard uniform baseline. Furthermore, we compare our dynamic approaches against the established static non-uniform strategies detailed in Section 3.2.3: the Bernoulli (Zheng et al., 2024a), Beta (Zheng et al., 2024b), and logit-normal (Esser et al., 2024) samplers. Table 5.1 summarizes the complete set of evaluated strategies. For all static baselines, we strictly employ the optimal distribution hyperparameters empirically identified in their respective original publications. Crucially, every strategy is trained using an identical generative backbone, optimizer, and training schedule; the timestep-sampling mechanism acts as the sole independent variable. This rigorous control ensures that all observed variations in performance or training dynamics are strictly attributable to the respective sampling strategy, isolating the results from any confounding architectural effects.

The remainder of this section details the specific training configurations, network architectures, and quantitative evaluation metrics used to conduct this suite. An exhaustive overview of the hyperparameters for every component is provided in Appendix A.2.

Table 5.1: **Timestep-sampling strategies evaluated in the comparison suite, together with the exact hyperparameter configurations tested across our runs.** All share an identical backbone, optimizer, and schedule and differ only in how $t \in [0, 1]$ is drawn during training. *Static* samplers use a fixed density; *adaptive* samplers learn the density online via a policy (REINFORCE), and the *actor–critic* variants additionally learn a value baseline. Every adaptive variant shares the same policy backbone (a Beta head initialized to $\mathrm{Beta}(1, 1) = \mathcal{U}[0, 1]$, updated every 40 optimizer steps, actor/critic learning rates $10^{-2}/10^{-3}$).

Strategy	Sampling density	Tested configuration(s)	Reference
Static samplers (fixed density)			
Uniform	$\mathcal{U}(0, 1)$	baseline (no parameters)	Lipman et al. (2023)
Logit-normal	$t = \sigma(z),\ z \sim \mathcal{N}(m, s^2)$	$m = 0,\ s = 1$	Esser et al. (2024)
Bernoulli (data)	Piecewise: $f(t) \propto p$ on $(t_\delta, 1]$, else $1 - p$	t_δ=0.2, $p \in \{0.8, 0.9\}$	Zheng et al. (2024a)
Bernoulli (noise)	Piecewise: $f(t) \propto p$ on $(t_\delta, 1]$, else $1 - p$	t_δ=0.2, p=0.2	Zheng et al. (2024a)
Beta (data)	$\mathrm{Beta}(\alpha, \beta)$	(α, β)=$(1, 0.8)$	Zheng et al. (2024b)
Beta (noise)	$\mathrm{Beta}(\alpha, \beta)$	(α, β)=$(0.8, 1)$	Zheng et al. (2024b)
Adaptive samplers (learned Beta policy, REINFORCE)			
ATS	Learned Beta	$R = \Delta_k^t$	Kim et al. (2025a)
A2C-ATS	Learned Beta + critic	$R = \Delta_k^t - V_\psi(x_1)$	This work
LTE-ATS	Learned Beta + critic, LTE reward	$R = \widehat{\mathrm{LTE}}_\theta(x_t, t) - V_\psi(x_1)$	This work

5.1.1 Dataset and Preprocessing

All experiments are conducted in the context of image generation and use the CIFAR-10 dataset (Krizhevsky, 2009) at the native resolution of 32×32. CIFAR-10 comprises $60{,}000$

natural color images evenly distributed over ten mutually exclusive object classes: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship*, and *truck* (see Figure 5.1). Clean images x_1 are scaled to $[-1, 1]$ by per-channel normalization with mean and standard deviation 0.5. The standard $50{,}000/10{,}000$ train/test split is used to provide the held-out data for the validation loss (Section 5.1.4).



Figure 5.1: **The CIFAR-10 dataset.** One representative example from each of the ten CIFAR-10 classes at the native 32×32 resolution.

5.1.2 Model Architecture

Generative backbone. The flow vector field $v_t(x_t; \theta)$ is parameterized by a U-Net architecture, illustrated in Figure 5.2. The network uses a base width of 128 channels with channel multipliers $(1, 2, 2, 2)$, giving four resolution stages ($32 \to 16 \to 8 \to 4$) with two residual blocks each, and self-attention applied at both the 16×16 spatial stages and the 4×4 bottleneck. Dropout of 0.1 is applied inside the residual blocks. The timesteps $t \in [0, 1]$ are injected through the standard sinusoidal timestep embedding. Since this embedding was designed for integer DDPM timesteps, t is rescaled by 1000 before embedding. This representational convenience allows using the same U-Net backbone for Flow Matching without the need for architectural adjustments. In total, the backbone has approximately 35.7M trainable parameters.

ATS policy and value networks. The Adaptive Timestep Sampler uses an Actor (policy) network and, for the Actor-Critic variants, additionally a Critic (value) network. Both share a small convolutional encoder ($3 \to 32 \to 64 \to 128$ channels with 3×3 convolutions and max-pooling) terminated by an `AdaptiveAvgPool2d(4)` that yields a fixed $128 \times 4 \times 4 = 2048$-dimensional feature regardless of input resolution, followed by a two-layer MLP of width 256 (≈ 0.68M parameters each, negligible against the backbone). The Actor emits two outputs passed through a softplus to produce the strictly positive shape parameters (α, β) of a Beta distribution on $[0, 1]$, from which the continuous timestep t is sampled; its final-layer bias is initialized so the policy starts at $\mathrm{Beta}(1, 1) = \mathcal{U}[0, 1]$ (no initial temporal bias). The Critic emits a single scalar estimating the expected reward used as the policy-gradient baseline.

ATS feature selection and policy update. Following the ablation study of Kim et al. (2025a), the adaptive samplers evaluate a probe grid of 1000 candidate timesteps, from which an F-regression feature selector retains the 3 most informative candidates per policy update. The policy is updated every 40 optimizer steps, with the rewards aggregated over a replay buffer of length 5.

Flow Matching specifics. We use the CondOT path with the straight-line interpolant $x_t = (1 - t)\, x_0 + t\, x_1$ and target velocity $u_t = x_1 - x_0$, where $t = 0$ is pure noise and $t = 1$ is clean

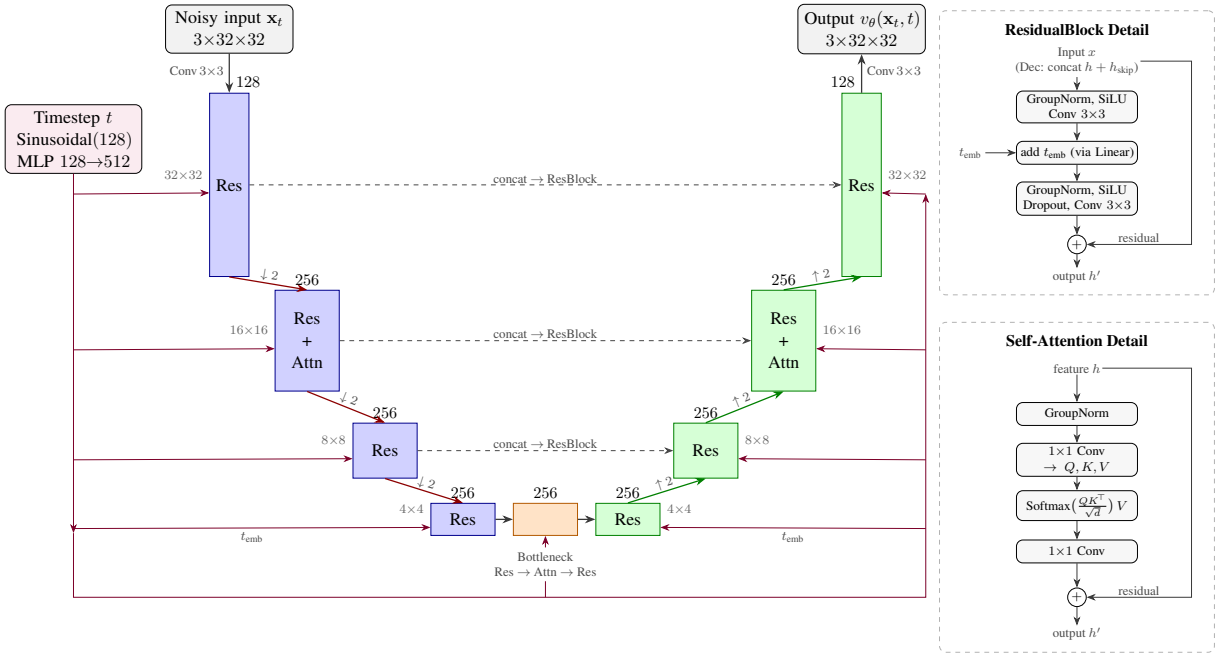


Figure 5.2: **The shared U-Net backbone used for all CIFAR-10 runs** (≈ 35.7**M parameters).** The architecture follows Kim et al. (2025b). *Left/center:* The contracting (encoder, blue) and expanding (decoder, green) paths. Each block's *height* encodes its spatial resolution ($32^2 \to 4^2$) and its *width* encodes the channel count ($128 \to 256$). The bottleneck operates at the 4×4 resolution and consists of two ResidualBlocks surrounding a Self-Attention layer. Red/green arrows indicate strided-convolution downsampling ($\downarrow 2$) and nearest-neighbor upsampling ($\uparrow 2$). Dashed gray arrows represent skip connections that *concatenate* encoder feature maps onto the decoder inputs. *Right:* Internal block details. The decoder ResidualBlocks fuse the concatenated features. Self-Attention is applied at both the 16×16 spatial stages and the 4×4 bottleneck. The sinusoidal timestep embedding t_{emb} is broadcast along the purple rail and added inside every residual block via a block-specific linear projection to match changing channel dimensions.

data. As in the standard Flow Matching formulation of Lipman et al. (2023), the noise x_0 and the data x_1 are drawn independently. At inference, the learned ODE is integrated with 50 uniform Euler steps.

5.1.3 Training Configuration and Hyperparameters

We summarize here only the settings that materially affect the comparison. The exhaustive listing of every backbone, actor, critic, optimizer, and ATS-probe hyperparameter is deferred to Table A.2 in Appendix A.2.

Optimization. All networks are trained with the Adam optimizer (Kingma and Ba, 2015), which combines momentum through an exponential moving average of the gradients (first moment) with adaptive learning rates derived from an exponential moving average of the squared gradients (second moment). Writing g_k for the gradient at step k, Adam maintains

$$\begin{aligned} m_k &= \beta_1\, m_{k-1} + (1-\beta_1)\, g_k, \qquad & \hat{m}_k &= \frac{m_k}{1-\beta_1^k}, \\ v_k &= \beta_2\, v_{k-1} + (1-\beta_2)\, g_k^2, \qquad & \hat{v}_k &= \frac{v_k}{1-\beta_2^k}, \\ \theta_k &= \theta_{k-1} - \eta\, \frac{\hat{m}_k}{\sqrt{\hat{v}_k}+\epsilon}, \end{aligned} \tag{5.1}$$

where η is the learning rate and $\epsilon = 10^{-8}$ prevents division by zero. The decay rates β_1 and β_2 control the memory of the first- and second-moment estimates. Specifically, $\beta_1 = 0.9$ averages gradients over roughly the last ten optimization steps, stabilizing the update direction, whereas $\beta_2 = 0.999$ accumulates squared gradients over a much longer horizon to adapt the effective step size of each parameter via $1/(\sqrt{\hat{v}_k}+\epsilon)$, reducing updates for parameters with consistently large or highly variable gradients. The bias-correction terms $\hat{m}_k$ and $\hat{v}_k$ compensate for the zero initialization of m_0 and v_0, preventing both estimates from being biased toward zero during the initial optimization steps. We use the standard default values $(\beta_1, \beta_2) = (0.9, 0.999)$.

Learning rates differ by role: We employ 2×10^{-4} for the U-Net backbone, 1×10^{-2} for the Actor—matching the policy learning rate used by Kim et al. (2025a)—and 1×10^{-3} for the Critic. Gradients are clipped to a global ℓ_2 norm of 1.0 for all three networks to cap the occasional large-norm updates common early in training and stabilize the optimization. The backbone learning rate is additionally warmed up linearly from 0 to 2×10^{-4} over the first 5000 steps, which avoids large, poorly conditioned updates while Adam's moment estimates are still based on too few gradients to be reliable.

Weights are tracked with an Exponential Moving Average (EMA) at decay 0.9999, and all sampling and evaluation use the EMA weights.

Hardware and batching. Each run trains for 300 epochs with a global batch size of 256, split as 64 samples across $4\times$ NVIDIA Titan X (Pascal) GPUs and synchronized via PyTorch DDP, which averages the per-GPU gradients into a single update per optimizer step.

All computation is carried out in single precision (FP32). We evaluated FP16 automatic mixed precision (AMP) but observed no training speed-up on the Pascal-generation Titan X hardware,

which lacks efficient half-precision tensor support, so mixed precision was disabled for all reported runs.

5.1.4 Evaluation Metrics

Fréchet Inception Distance (FID). We assess generation quality with the FID, introduced by Heusel et al. (2017). Generated and real images are embedded into the 2048-dimensional activations of the final average-pooling layer of an Inception-v3 network, and each set of activations is modeled as a multivariate Gaussian. The FID is then the squared 2-Wasserstein distance between these two Gaussians. Because the Wasserstein-2 distance between Gaussians admits the closed form

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\Big(\Sigma_r + \Sigma_g - 2\big(\Sigma_r\Sigma_g\big)^{1/2}\Big), \tag{5.2}$$

it coincides with the Fréchet distance between the two distributions, hence the name of this metric. Here (μ_r, Σ_r) and (μ_g, Σ_g) are the mean and covariance of the Inception features of the real and generated distributions, respectively. The first term penalizes a shift in feature means and the second a mismatch in feature covariances, so a lower FID indicates that generated samples are statistically closer to real data in feature space. Because it compares the full feature distributions rather than individual images, FID is sensitive to both sample fidelity *and* diversity: mode collapse, which shrinks the generated covariance, inflates the score just as low-fidelity samples do.

Samples are generated with the EMA model and compared against precomputed CIFAR-10 statistics. Both the training-time monitoring and the final reported scores use $10{,}000$ generated samples.

Recent work, however, cautions that the metric must be interpreted carefully, as its estimator is approximate in several respects. First, the Gaussian model captures only the first two moments of the feature distribution and discards all higher-order structure, so distributions that differ in skew or multimodality can receive identical scores (Bińkowski et al., 2018; Jayasumana et al., 2024). Second, the Inception-v3 embedding is trained on ImageNet classification and applied here to low-resolution CIFAR-10, a domain mismatch that makes the feature space only a proxy for perceptual similarity and can misalign FID rankings with human judgment (Stein et al., 2023; Jayasumana et al., 2024). Third, FID is positively biased at finite sample size (Chong and Forsyth, 2020; Bińkowski et al., 2018): our use of $10{,}000$ samples rather than the full $50{,}000$ keeps training-time evaluation affordable but inflates the absolute score relative to large-sample literature values.

Since the same sample count and reference statistics are used for every strategy we compare, this bias is shared across all runs. The *relative* comparison between strategies therefore remains valid, even though our absolute FIDs are not directly comparable to externally reported numbers.

Neural Function Evaluations (NFE). We report the NFE of the *backbone* only; the policy and value networks are comparatively negligible at ≈ 0.68M versus 35.7M parameters (see Section 5.1.2). NFE is a hardware-independent measure of computational expenditure: unlike wall-clock time, which is confounded by GPU model, kernel efficiency, and system load, a single

backbone evaluation costs the same NFE on any machine, making it the fair theoretical metric for comparing strategies. One *forward* pass ($\mathrm{NFE}_{\text{fwd}}$) is a single evaluation of $v_\theta(x_t, t)$ and one *backward* pass ($\mathrm{NFE}_{\text{bwd}}$) is the corresponding gradient computation through the backbone. A backward pass is not free relative to a forward one: empirically it costs $\approx 3\times$ as much (Li et al., 2020). To summarize the total training cost in a single hardware-independent scalar that reflects this asymmetry, we follow Kim et al. (2025a) and express it in *forward-pass equivalents*, charging each backward pass at that $3\times$ cost:

$$\mathrm{NFE}_{\text{fwd-eq}} = \mathrm{NFE}_{\text{fwd}} + 3\,\mathrm{NFE}_{\text{bwd}}. \tag{5.3}$$

This is the quantity reported in Table 5.2 and decomposed in Figure 5.4. Tracking the forward and backward counts explicitly matters because the adaptive strategies do not only differ in their training step: the (A2C-)ATS and LTE-ATS variants issue *additional forward-only* backbone passes for reward estimation (loss-reduction probes or local-truncation-error probes) that the static samplers never incur, so their extra cost lands entirely in $\mathrm{NFE}_{\text{fwd}}$ and must be charged against the strategy. NFE nonetheless remains a *proxy* rather than an exact cost: it is a unit count of backbone evaluations, not a FLOP measurement.

Wall-clock time. We additionally report the absolute training duration in seconds on the fixed $4\times$ NVIDIA Titan X (Pascal) hardware. While NFE captures theoretical compute, wall-clock time captures the practical, system-level overhead of the reinforcement-learning loops (policy and value updates, probe evaluations, and feature selection) layered on top of backbone training, which NFE alone does not reflect. Being hardware- and load-dependent, these numbers are not portable across machines and are reported only as a practical complement to the hardware-independent NFE.

Validation loss. The training loss is *not* a strategy-independent quality measure: because each sampler draws t from a different density, a sampler that concentrates on high-loss temporal regions (where the loss-reduction signal is potentially largest) will report a systematically higher training loss even at equal model quality, which makes cross-strategy comparison of training loss misleading. We therefore compute a validation loss once per epoch on the held-out test split, always with *uniform* temporal sampling $t \sim \mathcal{U}(0, 1)$, EMA weights, and a fixed seed, irrespective of the training sampler. Uniform t is the natural choice because it directly estimates the global Flow Matching objective over the entire trajectory,

$$\begin{aligned} \mathcal{L}(\theta) &= \mathbb{E}_{t\sim\mathcal{U}(0,1)}\,\mathbb{E}_{x_0,x_1}\left[\left\|v_\theta(x_t,t) - (x_1 - x_0)\right\|_2^2\right] \\ &= \int_0^1 \mathbb{E}_{x_0,x_1}\left[\left\|v_\theta(x_t,t) - (x_1 - x_0)\right\|_2^2\right] dt, \end{aligned} \tag{5.4}$$

which weights every timestep equally. This uniform weighting mirrors deployment: at inference each particle traverses the full trajectory and the ODE integrator queries the velocity field at every $t \in [0, 1]$ with equal weight (Section 5.1.2), so a metric that treats all timesteps equally is exactly the one aligned with the generative process. The reported value is, however, a Monte Carlo estimate of the continuous expectation in Equation (5.4), computed from a fixed set of ten held-out mini-batches with a *single* timestep drawn per image. Fixing the random seed makes the estimator consistent across epochs and strategies.

Per-timestep binning. A single scalar metric aggregated over the entire timestep domain hides how the training behavior of Flow Matching varies across noise regimes. To diagnose this variation across timesteps we partition $t \in [0, 1]$ into 10 uniform bins and, per epoch, log three per-bin quantities that the experiments below use directly. The mean *training* loss is the CondOT objective accumulated over the samples the training sampler places in the bin. The mean *validation* loss is measured with uniform-in-bin t on the held-out split, so it is independent of the training sampler and provides the sampler-invariant loss landscape analyzed in Experiment E2. The per-bin gradient statistics are described separately below. The discretization of the continuous interval into ten bins represents an approximation of the actual per-timestep metrics.

Per-t-bin gradient statistics. For Experiment E2 we additionally estimate two per-bin gradient statistics, the per-sample gradient variance and the mean gradient, and combine them into a gradient signal-to-noise ratio. Neither statistic can be measured exactly, because a backward pass returns only the summed gradient of a mini-batch and never the individual per-sample gradients, so we estimate each from micro-batch probes and mark the estimators with a hat. Every tenth epoch, for each bin b, the probe draws $K = 8$ micro-batches of $m = 64$ real training images, the two most recent mini-batches serving as the image pool, each image paired with a fresh noise draw $x_0 \sim \mathcal{N}(0, I)$ and a timestep drawn uniformly *within* the bin, and computes the mean gradient $\bar{g}_b^{(k)}$ of each micro-batch in a separate backward pass.

The *per-sample gradient variance* is the scalar $\operatorname{tr}\operatorname{Cov}(g) = \mathbb{E}\big\|g - \bar{g}\big\|^2$, where the per-example gradient is

$$g^{(i)} = \nabla_\theta \ell\big(\theta;\, x_1^{(i)}, x_0^{(i)}, t^{(i)}\big) \in \mathbb{R}^P,$$

a vector over the P network parameters. It measures how much the gradient of a single random example (x_1, x_0, t) drawn from the bin scatters around the bin's mean gradient, an intrinsic property of the objective and the data in the bin rather than of any particular batch, and its imbalance across timesteps is what motivates non-uniform timestep sampling. Because the variance of a micro-batch mean gradient is the per-sample variance shrunk by m, we estimate it from the Bessel-corrected sample variance of the K micro-batch gradients, rescaled by m,

$$\widehat{\operatorname{tr}\operatorname{Cov}}(g)_b \;=\; \frac{m}{K-1}\sum_{k=1}^{K}\big\|\bar{g}_b^{(k)} - \hat{g}_b\big\|^2, \qquad \hat{g}_b = \frac{1}{K}\sum_{k=1}^{K}\bar{g}_b^{(k)}, \tag{5.5}$$

which is unbiased for any $K \geq 2$. We report the trace rather than the full $P \times P$ covariance, which is intractable.

The *mean gradient* $\bar{g}_b = \mathbb{E}[g]$ is the systematic learning direction of the bin, estimated by the micro-batch average $\hat{g}_b$. We record its norm and its de-biased square,

$$\big\|\hat{g}_b\big\| = \Big(\textstyle\sum_{p\in\theta}(\hat{g}_b)_p^2\Big)^{1/2}, \qquad \widehat{\big\|\bar{g}_b\big\|^2} = \big\|\hat{g}_b\big\|^2 - \tfrac{\widehat{\operatorname{tr}\operatorname{Cov}}(g)_b}{Km}, \tag{5.6}$$

where the correction removes the finite-sample inflation $\mathbb{E}\big\|\hat{g}_b\big\|^2 = \big\|\bar{g}_b\big\|^2 + \operatorname{tr}\operatorname{Cov}(g)_b/(Km)$ of the squared norm of a Km-sample average (McCandlish et al., 2018, Appendix A.1).

The *gradient signal-to-noise ratio* read in Experiment E2 is the ratio $\widehat{\|\bar{g}_b\|^2}/\widehat{\operatorname{tr}\operatorname{Cov}}(g)_b$ of these two estimates, the share of a bin's gradient that is systematic signal rather than per-sample noise.

Reusing recent training images rather than loading a fresh batch is only an implementation convenience, and since the mini-batches are themselves iid data draws it does not bias the estimate, while the fresh noise and in-bin timestep are what the per-bin statistic requires. The probe runs outside the optimizer and before gradient clipping, and because it draws its own timesteps uniformly within each bin, its precision does not depend on how rarely the training sampler visits that bin. Unbiasedness holds for any K, but the precision of a single probe does not, as a variance estimated from only $K = 8$ micro-batches carries a relative standard error of about 50% that the micro-batch size m cannot reduce. We therefore average the last four probes and read orderings and orders of magnitude rather than precise per-bin values. The instrumented reruns match the E1 configuration in every hyperparameter and differ only in length (100 rather than 300 epochs, which suffices because the per-bin landscape is stationary after the first few epochs, Section 5.3.1), add a $\approx 1\%$ compute overhead, and feed only the E2 gradient statistics, so all E1 comparisons continue to rest on the original runs.

Policy optimization metrics. For the adaptive samplers we additionally log a set of policy diagnostics that verify the REINFORCE updates behave as intended and that the actor, and for the A2C variants the critic, converge stably rather than collapsing or diverging. We describe each in turn.

The *actor gradient norm* is the global pre-clipping ℓ_2 norm over all policy parameters ϕ,

$$\|g_\phi\|_2 = \Big(\textstyle\sum_{p \in \phi} \|\nabla_p \mathcal{L}_{\text{actor}}\|_2^2 \Big)^{1/2}, \tag{5.7}$$

and serves as a magnitude and stability signal (see Appendix A.1). A large norm does not by itself separate signal from noise, since it can reflect either a strong systematic update or a noisy, high-variance one, so we read a sustained or erratic norm as a flag for potential instability rather than as a direct measure of gradient noise.

The *policy-gradient variance* is the empirical within-batch variance of the per-sample REINFORCE surrogate $\log \pi_\phi(t \mid x_1)\, A(t, x_1)$, with A the advantage, and measures the noise of the Monte Carlo policy-gradient estimator directly. For the head-to-head comparison in Section 5.2.2 we report the step-wise log-ratio $\log_{10}(\mathrm{Var}_{\mathrm{pg,A2C}}/\mathrm{Var}_{\mathrm{pg,ATS}})$ of the two policy-gradient variances at matched optimization steps, which removes the shared decaying reward scale and exposes the sign of the baseline's effect at each update.

The *policy entropy* $\mathcal{H}[\pi_\phi]$ and its entropy-loss term track how strongly the entropy bonus holds the policy near the uniform distribution, and their behavior under the coupled and decoupled entropy formulations is analyzed in Appendix A.1.

For the A2C variants we additionally log the *critic loss* $\mathcal{L}_{\text{value}} = \mathbb{E}[(R - V)^2]$, the batch mean-squared error of the value prediction $V = V_\psi(x_1)$ against the reward $R = \tilde{\Delta}_k^t$, which doubles as the critic's approximation error, since a baseline can only cancel reward variance it can predict. Because R and V are per-image *scalars*, the variance-budget terms used to test the baseline in Section 5.2.2, namely the reward variance $\mathrm{Var}(R)$, the baseline variance $\mathrm{Var}(V)$, and their covariance $\mathrm{Cov}(R, V)$, are the ordinary empirical variance and covariance of those scalars over the batch and need no micro-batch estimation.

To track the learned sampling distribution we log the *batch-averaged Beta parameters* $\bar{\alpha} = \mathbb{E}[\alpha]$ and $\bar{\beta} = \mathbb{E}[\beta]$ together with the induced mean timestep $\mathbb{E}[\alpha/(\alpha+\beta)]$, the expectation taken over the per-image policy outputs in the batch. Because the policy is conditioned on x_1, these are averages over a batch of distinct Beta distributions, so the pair $(\bar{\alpha}, \bar{\beta})$ need not correspond to any single Beta that is actually sampled from and $\mathrm{Beta}(\bar{\alpha}, \bar{\beta})$ is not the mean density. We report them only as trend indicators of whether the policy drifts toward noise-biased ($t \to 0$) or data-biased ($t \to 1$) sampling.

These diagnostics are within-batch population statistics logged at the policy-update cadence, so we read them as per-update trend indicators rather than exact expectations and summarize the reward and variance-budget series by medians and the paired log-ratio, since a single early spike can dominate their mean. Every approximation is applied identically to all strategies, so it perturbs absolute values while preserving the relative ordering on which the comparative conclusions of this chapter rest.

Inference trajectory diagnostics. The inference-side analysis of Section 5.4 characterizes the geometry of the learned velocity field along generation trajectories with three diagnostics. All three are motivated by the same principle: straight, constant-speed trajectories are exactly the ones a first-order ODE solver integrates without error and in few steps, which is why straightening the learned flow is the explicit design goal of rectified flow and optimal-transport Flow Matching (Liu et al., 2023; Tong et al., 2024).

The *path deviation* $1 - \cos\big(v_\theta(x_t, t),\, x_t - x_0\big)$ measures the angle between the instantaneous velocity and the chord traveled so far. It vanishes for a perfectly straight ray from x_0 and grows with the accumulated curvature of the trajectory. It is a directional proxy for the rectified-flow straightness $\int_0^1 \mathbb{E}\,\|(x_1 - x_0) - v_\theta(x_t, t)\|_2^2\,\mathrm{d}t$ of Liu et al. (2023), retaining its geometric content—zero if and only if the path is straight—while remaining invariant to the velocity magnitude. Because the endpoint x_1 is unavailable while the ODE is integrated, the diagnostic substitutes the chord $x_t - x_0$ to the current state for that ideal displacement and reads the deviation pointwise at each t rather than accumulating it along the path.

The *kinetic transport energy* $W_{\mathrm{kin}} = \int_0^1 \mathbb{E}\,\|v_\theta(x_t, t)\|_2^2\,\mathrm{d}t$ integrates the squared velocity magnitude over the schedule. Through the dynamic formulation of optimal transport by Benamou and Brenier (2000), the squared Wasserstein-2 distance between two distributions is the minimal kinetic energy over all velocity fields that transport one into the other. The kinetic energy of a learned flow consequently upper-bounds the squared W_2 distance between its noise and output distributions, and any excess over that minimum is transport suboptimality spent on curved or non-constant-speed paths—the same energy that flow-based optimal-transport methods minimize to obtain straighter, cheaper-to-integrate flows (Tong et al., 2024).

The *local truncation error* is the Euler–Heun discrepancy $\widehat{\mathrm{LTE}}_\theta$ defined in Section 4.3, the same proxy that serves as the LTE-ATS reward, here evaluated along the free Euler trajectory. It measures directly the per-step error a first-order solver incurs from the trajectory's curvature and thus translates the geometric quantities above into integrator cost.

5.2 E1 – Generation Quality and Training Efficiency

We first provide an overview of the final sample quality and training metrics of all evaluated sampling strategies after the full 300-epoch training run in Table 5.2.

Table 5.2: **Final comparison of t-sampling strategies after 300 training epochs.** FID quantifies the sample quality; wall-clock time and backbone forward-pass-equivalent NFE (Equation (5.3)) serve as metrics for the required training compute. As the training loss is biased by the t-sampling strategy, we additionally report the validation loss (evaluated with *uniform-t* sampling, hence comparable across samplers, see Section 5.1.4). Adaptive strategies are grouped at the bottom; best value per comparable column in **bold**. All non-adaptive samplers perform *identical* per-step compute; their $<1\%$ wall-clock spread reflects shared-cluster scheduling noise, not algorithmic cost. The meaningful compute gap is static (≈ 5.4 h) versus ATS/A2C-ATS (5.93 h).

Sampler	FID ↓	Train loss	Val. loss ↓	Wall-clock (h) ↓	$\mathrm{NFE}_{\text{fwd-eq}}$ ↓
Non-adaptive					
Uniform	13.83	0.1747	0.1709	5.41	**236,910**
Logit-normal	13.80	0.1373	0.1738	**5.39**	**236,910**
Bernoulli (data, 0.8)	**13.60**	0.1697	0.1697	**5.39**	**236,910**
Bernoulli (data, 0.9)	13.84	0.1663	0.1699	5.40	**236,910**
Beta (data)	13.70	0.1915	0.1707	5.41	**236,910**
Beta (noise)	14.62	0.1747	0.1704	5.44	**236,910**
Bernoulli (noise, 0.2)	17.34	0.1855	0.1704	5.42	**236,910**
Adaptive					
ATS	14.18	0.1749	**0.1694**	5.93	248,910
A2C-ATS	14.21	0.1738	0.1709	5.93	248,910
LTE-ATS	395.2	0.5835	1.1194	5.44	239,910

Based on these results, the remainder of this section conducts the investigation at the core of this thesis: the evaluation of the adaptive timestep sampling strategies. We perform this evaluation along four hypotheses, which first assess each of the adaptive strategies presented in the method chapter on its own terms (H1.1–H1.3) and then benchmark them against the static non-uniform timestep samplers (H1.4):

- **H1.1.** Adaptive Timestep Sampling reaches a given held-out loss, or a given sample quality, with *less* compute than uniform sampling—or reaches a better value at equal compute.
- **H1.2.** Adding the A2C value-network baseline lowers the variance of the policy gradient relative to baseline-free ATS, and this translates into better final models.
- **H1.3.** The curvature-aware LTE reward yields a valid t-sampling policy and induces straighter inference trajectories.
- **H1.4.** Adaptive Timestep Sampling outperforms the static non-uniform sampling strategies.

5.2.1 Does Adaptive Timestep Sampling Improve Flow Matching Training Efficiency?

Hypothesis. If some timesteps carry a more useful learning signal like a higher expected loss reduction than others, a policy biased toward those timesteps should reduce the validation loss faster per unit of compute than uniform sampling and should improve sample quality. We therefore expect ATS to dominate uniform in the loss-vs-compute and FID-vs-compute planes, and A2C-ATS to at least match it.

Equal-step comparison after 300 epochs. We first compare the final metrics after a fixed number of 58.5k optimizer steps (300 epochs, Table 5.2). The validation loss, which is not confounded by the sampler's own t-density (Section 5.1.4), is 0.1709, 0.1694, and 0.1709 for uniform, ATS, and A2C-ATS. The nominal advantage of ATS lies within a spread of under 1% and is not distinguishable from run-to-run noise. Sample quality gives the same picture. Final FID scores are 13.83, 14.18, and 14.21, and the FID-versus-optimizer-step curves overlap throughout training, with uniform sampling marginally ahead in the final third (Figure 5.3, left). At equal optimizer steps, adaptive sampling therefore already yields no measurable benefit. This view, however, does not yet account for the overhead of the policy updates that ATS and A2C-ATS perform every 40 optimizer steps.

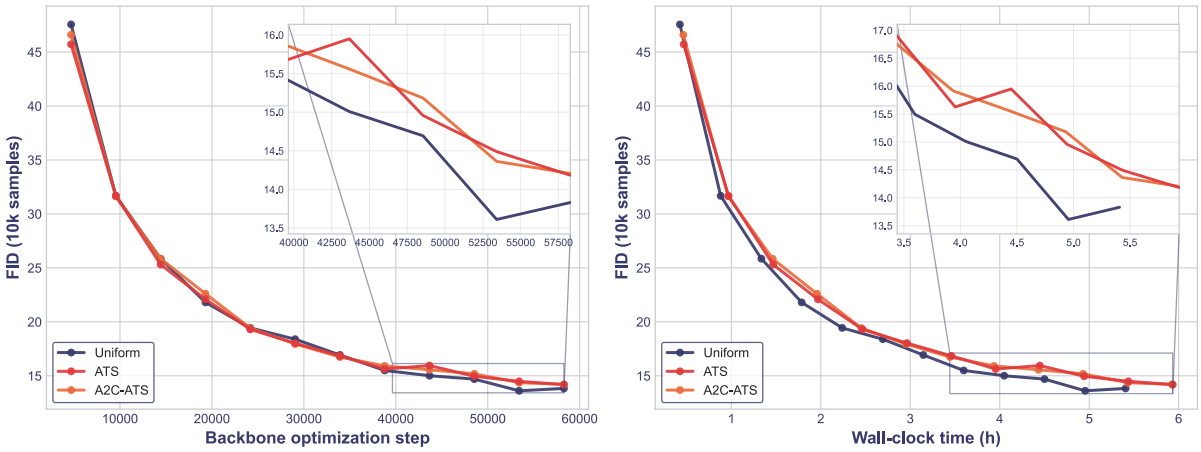


Figure 5.3: **Uniform sampling matches or slightly beats adaptive FID at every budget.** FID over training ($10{,}000$ EMA samples). *Left:* FID versus optimizer steps. The three strategies overlap, with uniform sampling marginally ahead in the final-third zoom. *Right:* FID versus wall-clock time. The adaptive strategies spend $\approx 10\%$ longer per epoch, so the uniform curve lies on or slightly below both the ATS and A2C-ATS curves at every compute budget and reaches its final FID notably sooner.

Accounting for the adaptive overhead. A fair comparison must charge the adaptive methods for the compute they spend outside the plain optimization step. Under uniform sampling, as under the static non-uniform strategies, drawing the timestep is effectively free, and each optimization step costs one backbone forward pass for the loss and one backbone backward pass for the gradient. ATS adds a forward pass through the ≈ 0.68M-parameter actor to draw t, together with

an actor backward pass at every policy update. Both are negligible relative to the backbone and are excluded from the backbone compute (Section 5.1.2). The material cost lies in the reward estimation. At every policy update, performed once per 40 optimization steps, the backbone is queried by loss-reduction probes before and after the weight update, so that the measured loss difference isolates the effect of that update (Section 5.1.2). A2C-ATS additionally trains a critic, whose passes are likewise small and excluded, which is why ATS and A2C-ATS carry identical backbone compute.

Each policy update contributes eight backbone forward-pass-equivalents. Two arise from the 1000-point τ-grid probe that maps the loss profile, evaluated as a single batched backbone call before and after the update, and six from the reward estimation on the three selected timesteps, again evaluated before and after. With five policy updates per epoch over 300 epochs (1500 updates in total), the probes add $1500 \times 8 = 12{,}000$ forward-pass-equivalents.

Aggregated over training (Figure 5.4, left), this raises the backbone cost from $236{,}910$ to $248{,}910$ forward-pass-equivalents (Equation (5.3)), an increase of 5.1% for both ATS and A2C-ATS. On our hardware the theoretical overhead, combined with the actor and critic updates, probe bookkeeping, and feature selection, produces a wall-clock time increase of 9.6%, from 5.41 to 5.93 hours (Figure 5.4, right). Adaptive sampling is therefore strictly more expensive, and for H1.1 to hold it must recover this premium through faster convergence.

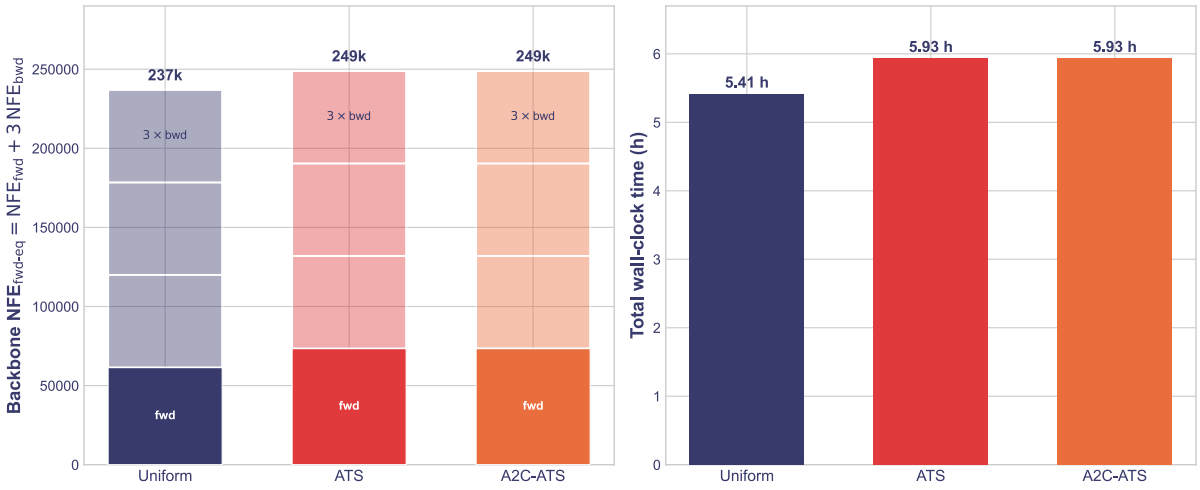


Figure 5.4: **Adaptive sampling adds a $\approx 10\%$ compute premium.** Training cost of uniform vs. adaptive sampling. *Left:* theoretical backbone compute in forward-pass equivalents (Equation (5.3)); each bar places the forward passes at the bottom (solid) and stacks the backward passes on top at their $3\times$ cost, drawn as three translucent tiers. The adaptive methods' extra forward-only probes are included in the forward block, so the NFE gap over the static samplers stems solely from the additional probe and reward-estimation forward passes. *Right:* the corresponding measured wall-clock time on $4\times$ Titan X, where the untracked reinforcement-learning bookkeeping widens the gap to $+9.6\%$.

Training comparison at equal compute. Once the overhead is charged and the curves are read against wall-clock time rather than optimizer steps, adaptive sampling fares no better.

The training and validation losses are effectively indistinguishable across uniform, ATS, and A2C-ATS (Figure 5.5). Because the adaptive curves are shifted right by their $\approx 10\%$ overhead, they at best match uniform and never lead it, so neither adaptive strategy shows a convergence advantage in loss.

Sample quality separates the strategies more clearly. Plotted against wall-clock time, the uniform FID curve lies on or below both adaptive curves at every compute budget after the first reported scores (Figure 5.3, right), and uniform reaches its final FID of 13.83 roughly half an hour before ATS reaches the worse final value of 14.18. Adaptive sampling therefore pays a $\approx 10\%$ compute premium for no quality gain, and if anything a slight regression. How the static non-uniform baselines, which carry no such overhead, compare with uniform is taken up in the suite-wide benchmark (Section 5.2.4, H1.4).

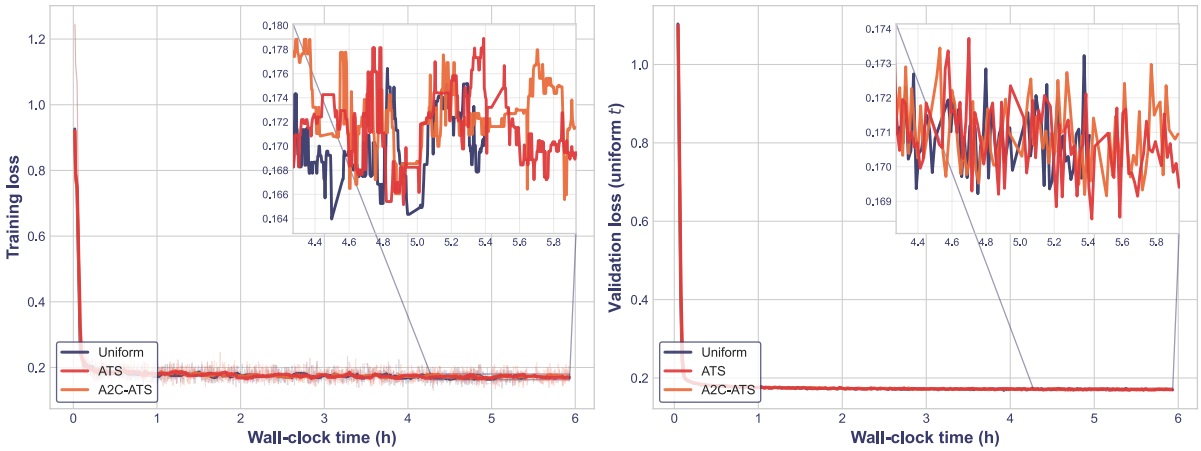


Figure 5.5: **Adaptive sampling does not accelerate convergence at equal compute.** Backbone loss versus wall-clock time under uniform, ATS, and A2C-ATS sampling. *Left:* training loss, logged every 40 optimizer steps (raw in faint, 25-point rolling median bold); its high-frequency fluctuation reflects the mini-batch estimation error. *Right:* the sampler-independent validation loss per epoch. The strategies track one another, and the adaptive curves, shifted right by their $\approx 10\%$ overhead, never lead uniform, as the final-quarter zoom insets confirm.

Qualitative samples. The FID verdict is mirrored in the generated images. Figure 5.6 shows unconditional CIFAR-10 samples from the final (e=299) EMA model of each strategy. Consistent with the sub-1 FID spread, the three panels are perceptually interchangeable: all render the same mix of recognizable-but-imperfect 32×32 objects with no systematic difference in fidelity or diversity attributable to the sampler. Adaptive sampling here impacts neither the quantitative nor the qualitative output.

The policy becomes approximately uniform after an initial warm-up. The failure of ATS to improve over uniform sampling does not stem from an immobile policy that never leaves its uniform initialization. Within the first ~ 10 epochs the mean sampled timestep $\mathbb{E}[t] = \alpha/(\alpha+\beta)$ swings between 0.34 and 0.60 and the shape parameters oscillate visibly (Figure 5.7). Thereafter the oscillations damp and the policy mean-reverts, ending training near $\mathrm{Beta}(1.00, 1.00)$, which

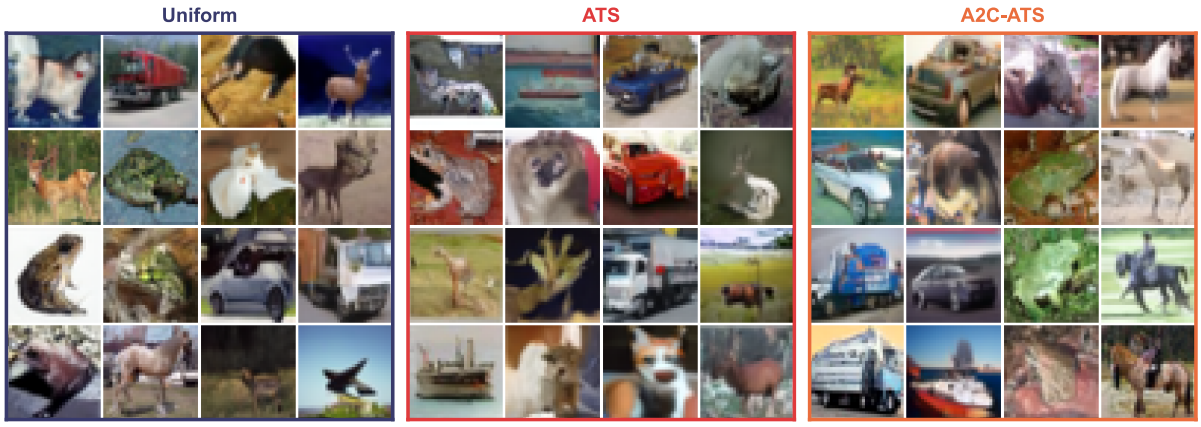


Figure 5.6: **No visible difference in image quality between sampling strategies.** Sixteen unconditional CIFAR-10 samples (4×4 grid) from the final EMA model of each strategy (Euler, $N{=}50$; each panel drawn from its run's own noise). At this resolution the outputs are perceptually indistinguishable, matching the <1 FID spread of Table 5.2.

corresponds to a uniform distribution. A2C-ATS behaves identically once it survives a critic-noise-driven transient in its first epoch ($\mathbb{E}[t] \to 0$ with β briefly ≈ 10). The A2C critic is analyzed in the next subsection.

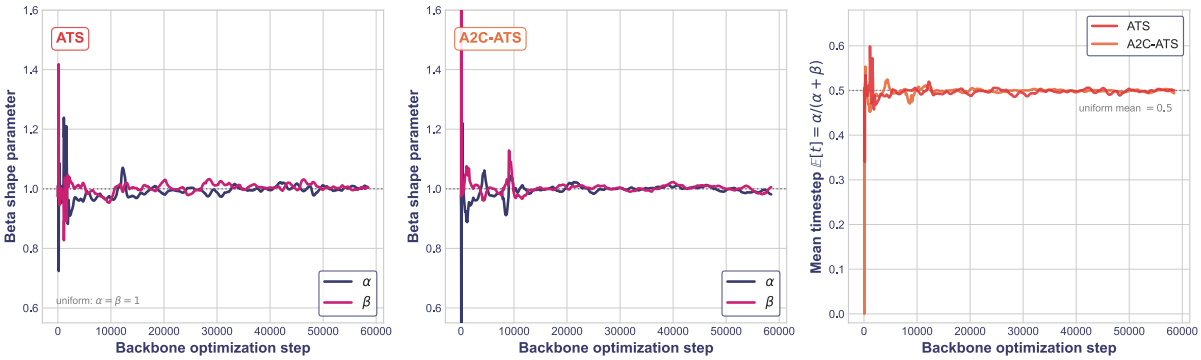


Figure 5.7: **The learned policy explores, then reverts to uniform.** Policy dynamics of ATS and A2C-ATS over training. *Left, center:* the batch-averaged Beta shape parameters α (navy) and β (magenta) for ATS and A2C-ATS separately; both oscillate during the first ~ 10 epochs, then settle at $\alpha \approx \beta \approx 1$ (the A2C first-epoch transient reaching $\beta \approx 10$ runs off-scale). *Right:* the induced mean timestep $\mathbb{E}[t] = \alpha/(\alpha + \beta)$ swings between 0.34 and 0.60 early (A2C transiently to ≈ 0), then holds within $\pm 2\%$ of the uniform value 0.5 for the remaining ~ 290 epochs.

A likely cause of this mean-reversion is apparent in the reward itself (Figure 5.8, left). The per-update loss-reduction reward $R = \mathcal{L}_{\text{before}} - \mathcal{L}_{\text{after}}$ on the straight CondOT interpolant is minuscule and shrinks as the backbone converges. The batch-mean reward $\bar{R}$ starts at $\approx 4 \times 10^{-3}$ during the mobile phase and decays toward $\approx 10^{-4}$, fluctuating around zero for the remainder of training (median $|\bar{R}| \approx 7 \times 10^{-4}$). With almost no reward signal, the entropy term, which is maximal for a Beta head exactly at $\mathrm{Beta}(1, 1)$, becomes the dominant force on the policy and anchors it at uniform (Figure 5.8, right). Whether this near-zero reward is also *flat across* t, and

therefore offers no non-uniform optimum even in principle, is examined through the per-bin loss and gradient statistics in Section 5.3.

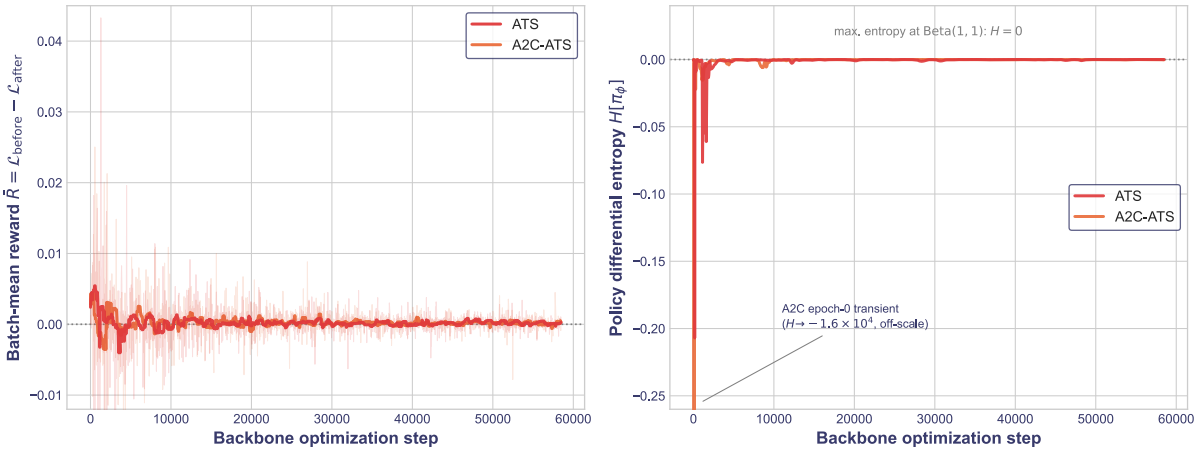


Figure 5.8: **A vanishing reward lets the entropy term restore uniform.** *Left:* the batch-mean loss-reduction reward $\bar{R}$ per policy update (raw faint, rolling median bold) is on the order of 10^{-3} during the early mobile phase and decays toward $\approx 10^{-4}$, fluctuating around zero thereafter. *Right:* the policy differential entropy $H[\pi_\phi]$, maximal ($= 0$) at $\mathrm{Beta}(1, 1)$; with a near-vanishing reward the entropy term dominates and drives the policy back to the maximum-entropy uniform distribution (the A2C epoch-0 critic transient drives H far off-scale and is annotated).

H1.1 does not hold. Adaptive timestep sampling yields no significant convergence-speed or sample-quality gain over uniform, yet adds a $\approx 10\%$ compute overhead. A possible explanation is that the reducible part of the CondOT per-timestep loss landscape is exhausted within the first few epochs and the landscape is stationary thereafter (Section 5.3), leaving the loss-reduction reward too weak and too uniform for an online policy to exploit for the remainder of training.

5.2.2 Does the A2C Baseline Reduce Policy-Gradient Variance?

Hypothesis. The A2C critic is a learned value network $V_\psi(x_1)$ that predicts the expected reward for a given image (Section 4.2). Subtracting this baseline from the loss-reduction reward $\tilde{\Delta}_k^t$ should center the reward and reduce the variance of the policy-gradient estimator. This lower-variance signal should in turn stabilize policy learning and yield a better final generative model.

Analysis of policy-gradient variance. We compare the policy-gradient variance of A2C-ATS and baseline-free ATS on a training run instrumented to log the reward and baseline statistics at every policy update, and write $R = \tilde{\Delta}_k^t$ for the loss-reduction reward and $V = V_\psi(x_1)$ for the critic prediction. The step-wise log-ratio $\log_{10}(\mathrm{Var}_{\mathrm{pg,A2C}}/\mathrm{Var}_{\mathrm{pg,ATS}})$ isolates the effect of the baseline at each update (Figure 5.9). Rather than lowering the variance, the baseline mildly raises it. A2C-ATS is the higher of the two on 67% of updates, by a median factor of about 2.9 (median log-ratio $+0.46$), and the two medians differ in the same direction (1.3×10^{-8} for A2C-ATS against 4.1×10^{-9} for ATS). The critic therefore does not reduce the policy-gradient variance and, if anything, adds to it.

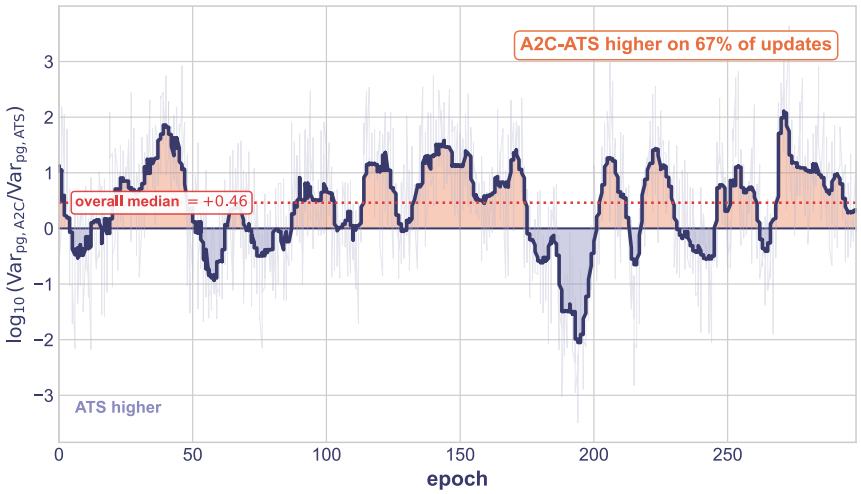


Figure 5.9: **The learned critic does not reduce the policy-gradient variance.** Step-wise log-ratio $\log_{10}(\mathrm{Var}_{\mathrm{pg,A2C}}/\mathrm{Var}_{\mathrm{pg,ATS}})$ over training (raw faint, 25-point rolling median bold). Positive values (orange) mark updates where A2C-ATS has the higher variance, negative values (blue) where baseline-free ATS does. The ratio is positive on 67% of updates, so the baseline raises rather than lowers the policy-gradient variance.

Testing the variance-reduction condition. The variance-reduction argument of Section 4.2 is conditional on the critic approximating the conditional-mean reward well, $V_\psi(x_1) \approx \mathbb{E}_{t,x_0}[\tilde{\Delta}_k^t \mid x_1]$. We test this assumption directly. The variance of the centered reward that drives the update decomposes as

$$\mathrm{Var}(R - V) = \mathrm{Var}(R) + \mathrm{Var}(V) - 2\,\mathrm{Cov}(R, V), \tag{5.8}$$

so the baseline reduces variance only when $2\,\mathrm{Cov}(R, V) > \mathrm{Var}(V)$, that is when the critic is accurate enough that its correlation with the reward outweighs the variance it introduces. The instrumentation additionally logs the three terms at every policy update (Table 5.3). The critic–reward covariance is effectively zero, so the normalized correlation $\rho = \mathrm{Cov}(R, V)/(\sigma_R \sigma_V)$ vanishes and the variance-reduction gain $2\,\mathrm{Cov}(R, V) - \mathrm{Var}(V)$ is negative on 89% of updates. Subtracting the baseline therefore *raises* the centered-reward variance above $\mathrm{Var}(R)$ throughout training, by a median factor of about 1.5 (Figure 5.10). Both variances shrink by more than an order of magnitude as the backbone converges, so we summarize the check with the scale-free correlation and variance ratio, which are stable across training. Binned into ten equal windows, ρ stays within ± 0.02 of zero and $\mathrm{Var}(R - V)/\mathrm{Var}(R)$ stays above one throughout (3.3 early, 1.2 late), so the baseline fails the condition over the entire run, not only on average. This is consistent with the weak reward established for the efficiency hypothesis (Section 5.2.1). The CondOT loss-reduction reward is minuscule ($|R| \sim 10^{-4}$, $\mathrm{Var}(R) \sim 10^{-5}$), leaving little reward structure for the critic to fit, so its prediction ends up uncorrelated with the reward it should track.

Possible reasons for the baseline's failure. A plausible explanation for the critic's inability to reliably track the actor's reward is the *non-stationarity* of that reward. Unlike the fixed reward of a standard reinforcement-learning problem, here the reward is the per-sample loss reduction of the backbone, which shifts at every optimization step as the U-Net updates. A timestep that yields a high reward at one point in training may therefore yield a low one shortly after. A critic

Table 5.3: **Variance-decomposition terms for A2C-ATS (Eq. (5.8)).** Per-update within-batch statistics from the instrumented run (median and interquartile range over the 1500 policy updates). Variances and covariance are in units of 10^{-5}; the correlation ρ is dimensionless. The variance-reduction gain $2\,\mathrm{Cov}(R,V) - \mathrm{Var}(V)$ is negative on 89% of updates, so the condition $2\,\mathrm{Cov}(R,V) > \mathrm{Var}(V)$ fails throughout training and $\mathrm{Var}(R-V)$ exceeds $\mathrm{Var}(R)$ (median ratio $\approx$1.5).

	Median	25%	75%
Variances and covariance ($\times 10^{-5}$)			
Reward variance $\mathrm{Var}(R)$	2.8	1.0	8.6
Baseline variance $\mathrm{Var}(V)$	1.0	0.4	3.4
Covariance $\mathrm{Cov}(R,V)$	0.0	−0.2	0.2
Gain $2\,\mathrm{Cov}(R,V) - \mathrm{Var}(V)$	−0.9	−3.4	−0.3
Centered-reward variance $\mathrm{Var}(R-V)$	4.5	1.8	14.0
Correlation $\rho = \mathrm{Cov}(R,V)/(\sigma_R\sigma_V)$	−0.003	−0.12	0.11

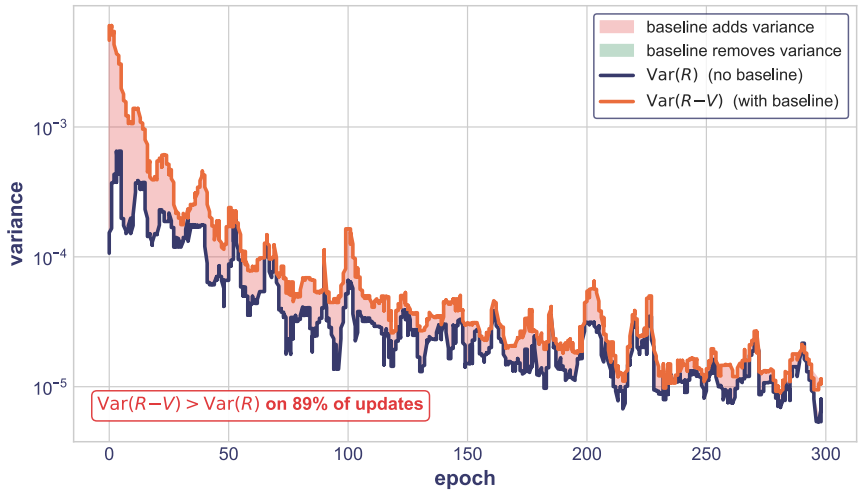


Figure 5.10: **Subtracting the baseline increases the centered-reward variance.** Per-update reward variance $\mathrm{Var}(R)$ (no baseline) and centered-reward variance $\mathrm{Var}(R-V)$ (with baseline), shown as 25-point rolling medians. The shaded gap is the variance-reduction gain, red where $\mathrm{Var}(R-V) > \mathrm{Var}(R)$ (the baseline adds variance) and green where it removes it. Both variances decay by more than an order of magnitude as the backbone converges, yet $\mathrm{Var}(R-V)$ stays above $\mathrm{Var}(R)$ on 89% of updates across the whole run.

regressing on such a moving target is expected to lag, fitting its baselines to outdated network states and injecting additional noise as observed.

H1.2 is refuted. The learned baseline does not reduce policy-gradient variance and mildly increases it (Figure 5.9). The decomposition explains why. The critic–reward covariance is effectively zero, so the correlation is too weak to satisfy $2\,\mathrm{Cov}(R, V) > \mathrm{Var}(V)$ (Eq. (5.8)), and subtracting an uncorrelated baseline injects its own variance rather than canceling the reward's. The highly non-stationary, near-zero-reward regime is a plausible reason for that approximation failure.

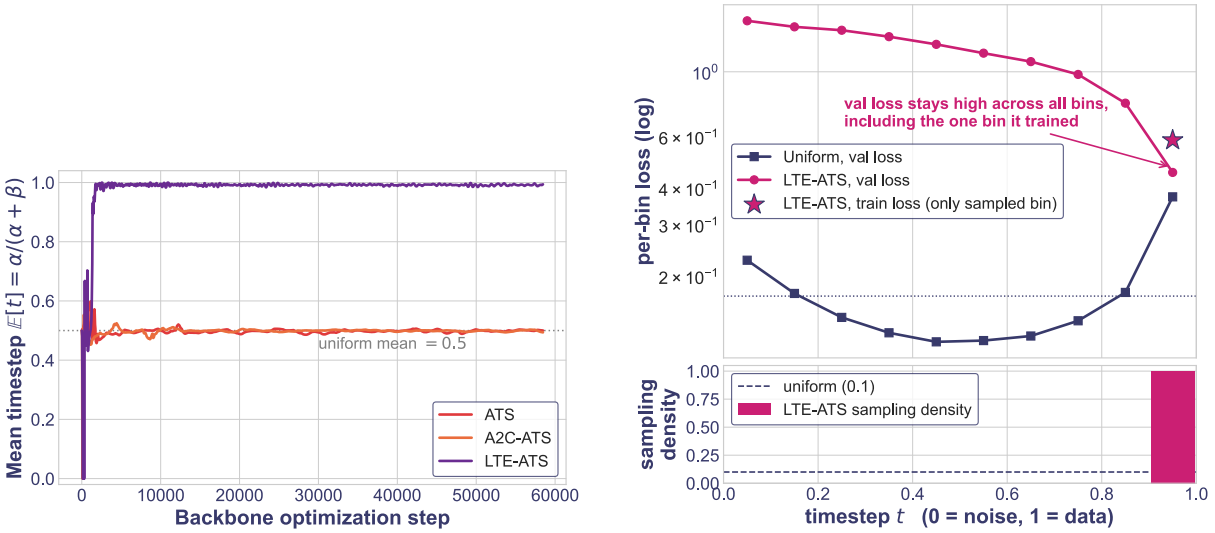
This absence of variance reduction does not translate into a drastic difference in sample quality, plausibly because both policies converge to the same, approximately uniform distribution for most of training. The FID scores of A2C-ATS (14.21) and vanilla ATS (14.18) lie within run-to-run noise and remain worse than uniform.

5.2.3 Is the LTE Curvature Reward a Valid Training Signal?

Hypothesis. The LTE reward assigns high value to the timesteps whose inference trajectories are most curved. The motivating idea is corrective. Biasing training toward these high-curvature timesteps should lead the model to devote more capacity to the part of the trajectory where a first-order ODE integrator struggles most. This in turn should reduce the measured curvature there in inference. If that feedback loop closes, curvature is a useful reward for guiding the timestep sampler.

LTE policy analysis. The policy collapses onto a single mode. The mean sampled timestep rises from 0.5 to $\mathbb{E}[t] > 0.9$ within the first $\sim$1400 optimizer steps (about seven epochs) and then freezes near $\mathbb{E}[t] = 0.994$ (Figure 5.11a), where the Beta density has become almost degenerate with $\alpha \approx 675$ and $\beta \approx 4.1$ and places nearly all of its mass at $t \approx 1$. The backbone is consequently starved of every timestep below $t = 0.9$, as the bottom nine t-bins receive no samples at all (Figure 5.11b). Its uniform-t validation loss rises to 1.119, against 0.171 for every non-collapsed sampler, and its FID reaches 395.2 (Table 5.2), so it no longer produces coherent images. The collapse is self-reinforcing but not self-correcting. The LTE reward is largest near $t = 1$ (Section 5.4), the policy concentrates its mass there, and training at $t \approx 1$ does not lower that reward. The reward therefore never decays and the policy never leaves the mode.

The conditional–marginal disconnect. Training at the high-curvature timesteps fails to reduce their inference curvature because of a mismatch between the *conditional* vector field learned in the objective and the *marginal* vector field that governs inference. Minimizing the *conditional* Flow Matching loss at $t \approx 1$ pushes the model's velocity field toward the straight noise–data interpolant of each *individual* training pair. The curvature that actually shapes generation, however, is that of the *marginal* velocity field, the expectation over all noise–data couplings that the ODE integrates at inference. Straightening every conditional path does not necessarily straighten this marginal average, so the curvature in inference the reward measures never falls. Concentrating all sampling at $t \approx 1$ thus reduces only the conditional training loss in the top bin and leaves the remainder of the trajectory ($t < 0.9$) untrained (Figure 5.11b), which eventually collapses generative quality.



(a) Mean sampled timestep over training

(b) Per-bin loss and sampling density

Figure 5.11: **The LTE reward collapses the sampler onto the highest-curvature mode.** *(a)* The LTE mean sampled timestep collapses to $t \approx 1$ within $\sim$1400 steps and never returns, while ATS and A2C-ATS revert to uniform after their exploratory transients. *(b) Top:* per-t-bin loss. LTE-ATS has a training loss only in the single bin it samples (star), yet its held-out uniform-t validation loss (magenta) stays high across every bin and remains well above the uniform reference (navy) even in that trained bin, so concentrating conditional training at $t \approx 1$ never straightens the marginal field. *Bottom:* final per-bin sampling density; LTE-ATS places all of its mass in the top bin ($t \geq 0.9$) against the uniform 0.1, so the backbone is never trained below $t = 0.9$.

H1.3 is refuted. The curvature reward is a valid signal only in the narrow sense that the policy responds to it strongly. It is not a useful training objective, because minimizing conditional curvature at a single timestep neither straightens the marginal field nor trains the rest of the trajectory. The sampler collapses onto the highest-curvature mode, starves the backbone, and produces unusable samples.

5.2.4 Benchmarking Adaptive against Static Timestep Sampling

Hypothesis. H1.4 asserts that adaptive timestep sampling outperforms the static non-uniform strategies drawn from the related work. The preceding subsections already established that none of the learned samplers improves on uniform sampling, which reduces H1.4 to a statement about the static priors alone, because any static prior that improves on uniform sampling thereby also surpasses the adaptive samplers. We therefore test whether the static non-uniform priors outperform uniform sampling and, if they do, which strategy of the full suite performs best overall.

Static non-uniformity carries no overhead. All static samplers incur the same backbone compute as uniform sampling, namely $236{,}910$ forward-pass-equivalents and ≈ 5.4 hours of wall-clock time, because drawing t from a fixed density is no more expensive than drawing it uniformly. Any quality a static prior gains over uniform sampling is therefore obtained at no additional cost. The adaptive samplers, in contrast, must first offset the $\approx 10\%$ overhead of their online policy (Section 5.2.1) before any improvement counts as a net benefit.

Bias direction is the dominant pattern. The clearest signal in Table 5.2 is that *where* a static prior places its probability mass matters more than the family it is drawn from. Reading the FID column down the static block, a bias toward the data-proximal regime ($t \to 1$) improves output quality and a bias toward the noise regime ($t \to 0$) is detrimental to it, in both sampler families. Measured against uniform sampling (13.83), the data-biased priors reach 13.60 for Bernoulli (data, 0.8) and 13.70 for Beta (data), whereas the noise-biased priors fall back to 14.62 for Beta (noise) and 17.34 for Bernoulli (noise, 0.2). Logit-normal sampling (13.80) falls in with the data-biased group, and Bernoulli (data, 0.9) at 13.84 is level with uniform sampling.

These margins should not be overstated, however. With the single exception of Bernoulli (noise, 0.2), the whole suite is contained in a band of about one FID point, from 13.60 to 14.62, and the six strategies drawn in Figure 5.12 span only 0.58. Bernoulli (noise, 0.2) alone is worse by a margin that is unambiguous. What the suite offers is thus a consistent direction rather than a decisive separation, and within that direction the best strategy is also among the cheapest. Bernoulli (data, 0.8) attains the lowest FID of the suite (13.60) at no cost beyond uniform sampling, ahead of the logit-normal density adopted as the default in Stable Diffusion 3 by Esser et al. (2024), though by a margin comparable to the run-to-run variation between neighboring evaluations of a single run.

The FID trajectories show that the lead of the data-biased and logit-normal priors is not an artifact of the final checkpoint but is visible over most of training (Figure 5.12). They lead uniform sampling and ATS up to a budget of roughly $40{,}000$ optimizer steps, by which point the curves have largely converged and the strategies become almost indistinguishable in FID. Reading

FID against wall-clock time rather than optimizer step widens the margin over ATS further, because its $\approx 10\%$ overhead shifts the curve to the right without ever letting it overtake a zero-cost static prior. Within the leading group the order does change once. Logit-normal sampling is the strongest strategy of the early phase and leads for roughly the first $17{,}000$ optimizer steps, after which both data-biased Bernoulli priors overtake it and remain at or below it for most of the remainder. A cautious reading is that the mid-range, on which logit-normal sampling concentrates its mass, is the regime the backbone fits first, and that once it is learned, further capacity is better spent near the data manifold, where the sharp and nearly noise-free images are resolved. The final margins are small enough that this remains a weak hint.

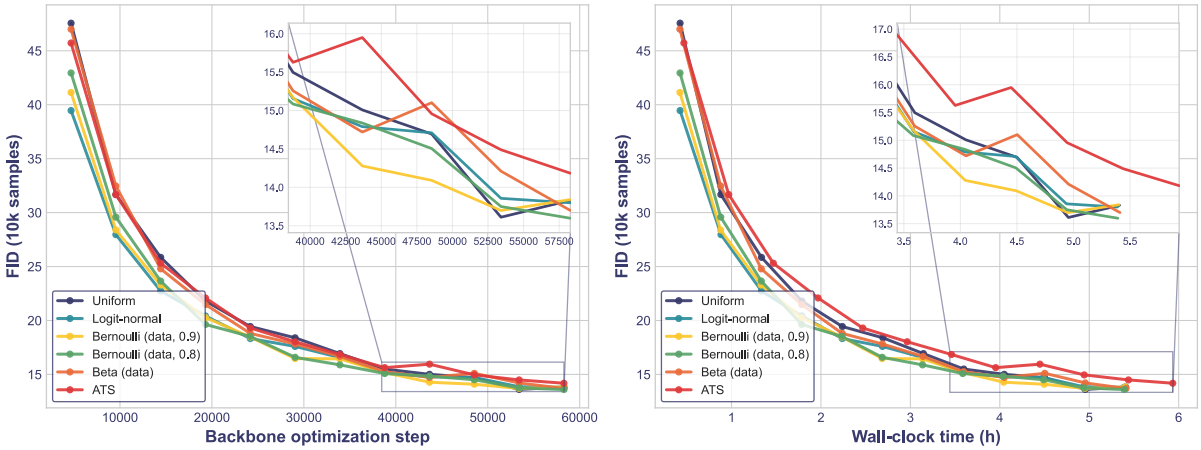


Figure 5.12: **Static data-biased priors lead adaptive sampling over most of training.** FID over training ($10{,}000$ EMA samples) for six representative strategies. *Left:* FID versus optimizer step. *Right:* FID versus wall-clock time. The final-portion zoom insets show the strategies converging into a sub-1 FID band, with ATS remaining above the data-biased priors and, on the wall-clock axis, penalized further by its $\approx 10\%$ overhead.

The training–validation loss mismatch. The two panels of Figure 5.13 disagree with one another systematically. In the epoch training loss the strategies spread over a wide band. Logit-normal sampling settles far below the field at 0.137, while Beta (data) sits at the top with 0.192, Bernoulli (noise, 0.2) reaches 0.186, and uniform sampling reaches 0.175. In the validation loss the same strategies collapse onto a common band near 0.170, and logit-normal sampling is now the highest of the suite at 0.174. The strategy with by far the lowest training loss is therefore the weakest on the validation loss.

This discrepancy can be explained by the fact that these two quantities are not measured against the same distribution of t. The epoch training loss is averaged over each epoch under the sampler's *own* t-density, whereas the validation loss is always evaluated at uniform t, so only the latter is comparable across samplers. Measured at uniform t, the per-timestep loss is U-shaped, low in the mid-range and rising toward both boundaries (Section 5.3.1). Each sampler's epoch loss is that same profile weighted by its own empirical sampling frequency over t, which reproduces the ordering of the left panel. Logit-normal sampling concentrates its mass on the low-loss mid-range and reports the smallest value, whereas Beta (data) and Bernoulli (noise, 0.2) weight the expensive boundaries and report the largest. The epoch training loss is thus a

readout of the shared U-shaped loss landscape combined with the sampling density, and only the validation loss and the FID assess model quality independently of it.

Because only the validation loss is comparable across samplers, it is notable that logit-normal sampling is consistently the highest of the suite on it. Its 0.174 lies above the ≈ 0.170 of every other strategy, and it does so over almost the entire run rather than only at the end. A plausible explanation is the severity with which logit-normal sampling neglects the two boundary regimes, from which it draws only about 1% of its samples each. Those regimes must still be integrated at inference, so underfitting them raises the loss at uniform t unless the mid-range is fitted substantially better in return. E2 finds no sign of such compensation, since every sampler reaches the same mid-range loss (Section 5.3.2).

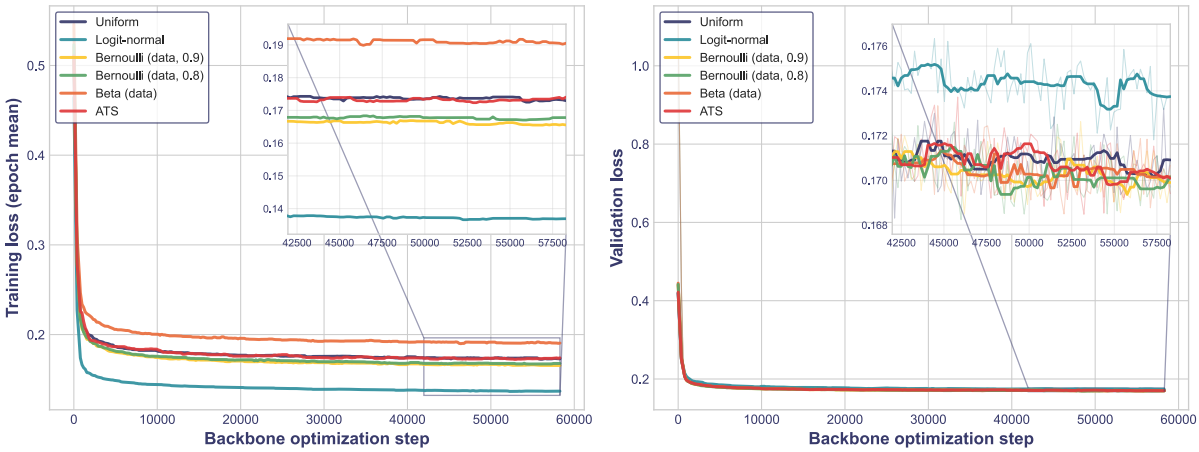


Figure 5.13: **The epoch training loss tracks the sampling density, not model quality.** *Left:* epoch-mean training loss under each sampler's own t-density (raw values with a 9-point rolling median); logit-normal sampling, concentrated on the low-loss mid-range timesteps, posts by far the lowest value. *Right:* the sampler-independent validation loss at uniform t, on which logit-normal is instead the highest while the remaining strategies collapse onto a common band near 0.170.

Qualitative samples. The FID ranking corresponds to almost no visible difference in the generated images (Appendix A.2.2, Figure A.3). The nine non-collapsed strategies, spanning the static priors together with ATS and A2C-ATS, render the same mixture of recognizable 32×32 objects and are perceptually interchangeable, which is consistent with the narrow FID band they occupy. Even Bernoulli (noise, 0.2), the one strategy whose FID sets it apart from the field, produces images that are at most marginally less clear to the eye. LTE-ATS is the sole visible exception and produces incoherent noise, the sample-space signature of the policy collapse analyzed for H1.3.

H1.4 is refuted. Adaptive timestep sampling does not outperform static non-uniform sampling on CondOT Flow Matching. ATS (14.18) and A2C-ATS (14.21) are worse in FID than uniform sampling and than every data-biased static prior while costing $\approx 10\%$ more compute, and the sole nominal edge of ATS, its validation loss of 0.1694, lies within the run-to-run spread of the static values and does not carry over to sample quality. LTE-ATS is off the scale (FID 395.2). Across

the whole static block the only samplers the adaptive methods surpass are the two noise-biased priors, and they reach none of the priors biased toward the data-proximal regime, which is the bias direction consistently associated with an improvement over uniform sampling in this setting. The best strategy of the suite is therefore a static data-biased prior like Bernoulli (data, 0.8) that costs nothing beyond uniform sampling, even though its margin over the rest of the field is small.

5.2.5 E1 Synthesis

A consistent thread links the four hypotheses of E1. The learned policy fails to help (H1.1) and the A2C critic only hurts (H1.2) in the same regime, because the loss-reduction reward on the straight CondOT interpolant is weak ($|R| \sim 10^{-4}$), decays further as the backbone converges, and admits no stable non-uniform optimum, so the policy explores briefly and then reverts to uniform. The one reward that is *not* weak, curvature (H1.3), is non-decaying and mode-peaked and collapses the policy outright. Adaptivity consequently never earns back its $\approx 10\%$ overhead in compute (H1.4).

Non-uniformity itself remains useful, provided it points in the right direction. A fixed prior biased toward the data-proximal regime can yield a small FID gain at no cost, whereas a noise-biased prior is harmful relative to uniform sampling. This is in line with the data-weighted timestep schedules advocated for diffusion models by Zheng et al. (2024b) and Zheng et al. (2024a) and equally with the logit-normal density of Esser et al. (2024), which performs on par with the data-biased priors in our suite. Our results support the claim these works share, that a non-uniform emphasis improves on uniform sampling, without separating their preferred densities from one another.

The ATS framework as ported therefore does not transfer to CondOT Flow Matching in this setting. A hand-set prior delivers, at no cost, what the online policy never found. This could hint that the exploitable temporal structure on CondOT is largely static rather than something a policy must track online. In this case an adaptive strategy could only add overhead on top of what a well-chosen prior already delivers. Experiment E2 takes this question up directly and measures whether the per-timestep structure of the learning process is in fact stationary (Section 5.3). Our finding does not rule out an adaptive method whose reward is reformulated for the CondOT objective, but such a method would still have to earn back its overhead against a practically compute-free baseline. Whether the benefit of ATS in discrete diffusion is instead contingent on the curved probability paths that the CondOT formulation in Flow Matching removes is an open question that we leave to future work.

5.3 E2 – Insights into the Flow Matching Learning Process

E1 assessed the timestep samplers by their outcomes and left two observations unexplained. The learned policies of ATS and A2C-ATS depart from uniform only during a short warm-up and then return to it (Section 5.2.1), which suggests that whatever per-timestep structure the CondOT objective offers is mostly static rather than evolving, so that a fixed prior can capture it as well as an online policy and without the overhead. A fixed data-biased prior indeed improves FID at no additional cost (Section 5.2.4), yet E1 offers no mechanism for why this particular bias helps. E2 examines both questions from the backbone's side, through the two quantities

that Kim et al. (2025a) use to build the case for adaptive timestep sampling in the DDPM setting, the per-timestep training loss and the per-timestep gradient variance. Since the realized densities of ATS and A2C-ATS coincide with uniform for all but the first few epochs, their Beta densities carry no further information, and we instead measure the learning process directly, combining the per-timestep-bin statistics of the E1 runs with three instrumented reruns, the uniform configuration and the two opposite Bernoulli priors, that estimate the per-bin gradient variance every tenth epoch (Section 5.1.4). We test two hypotheses.

- **H2.1.** The per-timestep landscape of CondOT Flow Matching is not uniform. The difficulty of the regression problem varies along the timestep axis, in the loss it attains and in the variance of the gradients it produces, analogous to the non-uniform denoising difficulty that motivates adaptive timestep sampling in the DDPM setting (Kim et al., 2025a).
- **H2.2.** Biasing the timestep sampler toward the noise or the data regime reshapes these per-timestep loss and gradient-variance landscapes.

5.3.1 Is the CondOT Loss Landscape Uniform across Timesteps?

Hypothesis. If the CondOT objective posed an equally hard regression problem at every timestep, it would attain a comparable loss and emit a comparably noisy gradient across the schedule. Kim et al. (2025a) document that the DDPM ϵ-objective is strongly non-uniform in t, with a per-timestep gradient variance that spans orders of magnitude and concentrates at the data-proximal, low-noise end of the schedule (Kim et al., 2025a, Figure 2), and with its per-timestep loss largest there as well.[1] Because the number of SGD iterations a subproblem needs to converge grows with its gradient variance (Garrigos and Gower, 2024), Kim et al. (2025a) read the high-variance timesteps as the bottleneck of uniformly sampled training and use it to motivate adaptive timestep sampling. H2.1 asks whether Flow Matching with the CondOT interpolant reproduces the same imbalance. We answer it on the uniform configuration, where every bin receives the same number of samples and the per-bin statistics are directly comparable, reading the loss profile off the uniform E1 run and the gradient variance off an instrumented rerun (Section 5.1.4).

The loss profile starts flat and settles into a fixed U-shape within a few epochs. At initialization the objective is nearly uniform in t, with all ten bins between 1.17 and 1.24 (Figure 5.14, left). The U-shape is already visible by the second epoch, deepens over the next few, and reaches its final spread of about $3\times$ within roughly five epochs. After that the shape barely changes while the overall level keeps drifting down slowly, so that the epoch-100 and epoch-300 profiles almost coincide. Averaged over the last twenty epochs the loss runs from a mid-range minimum of 0.120 up to 0.369 in the data-proximal bin, with the noise-proximal bin at 0.231 (Table 5.4).

The variance of the CondOT target shapes the loss profile. For a mean-squared objective the achievable per-bin loss splits exactly into a reducible and an irreducible part. Adding and subtracting the conditional mean $\mathbb{E}[u \mid x_t, t]$ inside the norm and expanding the square, the cross

[1] Note: Diffusion timestep notation is inverse to the Flow Matching convention, so their data-proximal timesteps $t \to 0$ correspond to our $t \to 1$.

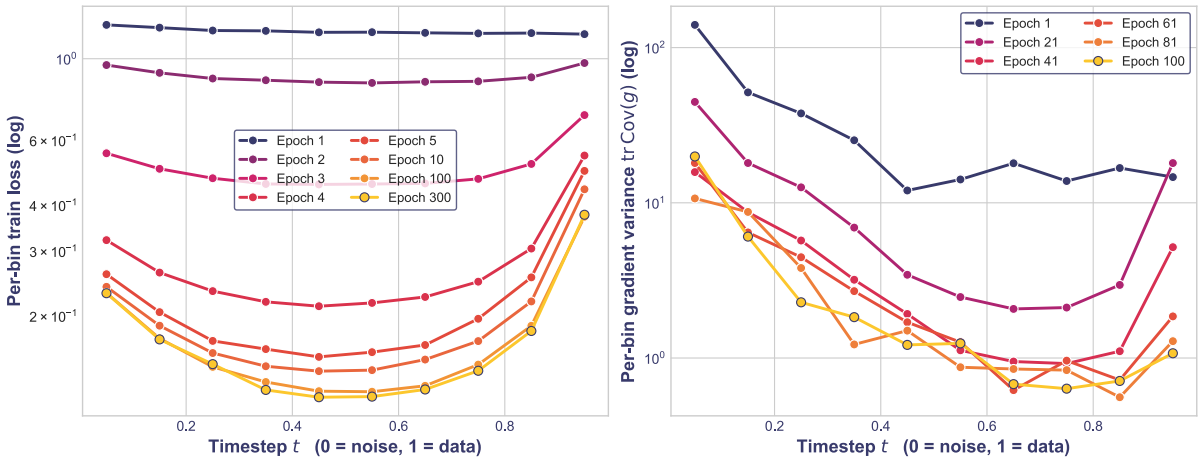


Figure 5.14: **The per-timestep loss and gradient variance are both strongly non-uniform in t, both U-shaped, and stationary in shape after the first few epochs, but their peaks sit at opposite ends of the schedule.** *Left:* per-bin training loss of the uniform run at selected epochs. From a nearly flat start the profile shows the U-shape by the second epoch, reaches its final $\approx 3\times$ spread within about five epochs and its final shape by epoch 10, after which it barely moves (epochs 100 and 300 almost coincide), with the data-proximal bin carrying the largest loss throughout. *Right:* per-sample gradient variance $\mathrm{tr}\,\mathrm{Cov}(g)$ per bin (log scale), estimated every tenth epoch on the instrumented rerun of the same configuration (Equation (5.5)). This profile is U-shaped as well but dominated by the noise boundary, so the loss and the gradient variance place the difficulty at opposite ends. The noise boundary is the highest-variance region at every probe and the shape is preserved, while the overall level, including an elevation at the data boundary during the first training epochs, decays as the fit improves and the interior bins draw together.

Table 5.4: **Per-timestep training statistics of the uniform run at four representative bins.** Each row is a single timestep bin of width 0.1, identified by its center t: the two boundary bins, the bin attaining the smallest loss, and the bin attaining the smallest gradient variance. Loss is the mean over the last twenty epochs; the gradient quantities are the mean over the last four probes, since a single $K{=}8$ probe carries roughly 50% relative error that the averaging suppresses (Section 5.1.4). $\mathrm{tr}\,\mathrm{Cov}(g)$ is the per-sample gradient variance, $\|\bar{g}\|^2$ the squared norm of the mean gradient (corrected for the estimator's finite-sample inflation), and the final column their ratio, the gradient signal-to-noise ratio. Across t the loss spans $3.1\times$, the gradient variance about $20\times$, and the squared mean-gradient norm less than $2\times$.

Regime	t	Loss $\mathcal{L}_t$	$\mathrm{tr}\,\mathrm{Cov}(g)$	$\|\bar{g}\|^2$	SNR $\|\bar{g}\|^2/\,\mathrm{tr}\,\mathrm{Cov}(g)$
Noise boundary	0.05	0.231	13.56	0.236	0.017
Loss minimum	0.55	0.120	1.19	0.128	0.108
Gradient-variance minimum	0.85	0.181	0.61	0.152	0.249
Data boundary	0.95	0.369	1.31	0.152	0.116

term has zero expectation because $\mathbb{E}\big[u - \mathbb{E}[u \mid x_t, t] \mid x_t, t\big] = 0$, which leaves

$$\mathcal{L}_t(\theta) = \mathbb{E}\big\|u - v_\theta(x_t, t)\big\|^2 = \underbrace{\mathbb{E}\big\|\mathbb{E}[u \mid x_t, t] - v_\theta(x_t, t)\big\|^2}_{\text{reducible by training}} + \underbrace{\mathbb{E}\big\|u - \mathbb{E}[u \mid x_t, t]\big\|^2}_{\text{irreducible}}. \tag{5.9}$$

The first term is the model's squared distance from the conditional mean, which proper training lowers. The second is the conditional variance of the target $u = x_1 - x_0$ given the input (x_t, t), which is irreducible. Conditioning on x_t imposes one linear constraint on the pair (x_1, x_0), so each of the two is an affine function of the other and the target can be written either as $u = (x_1 - x_t)/(1 - t)$ or as $u = (x_t - x_0)/t$. Since x_t is held fixed under the conditional covariance, this gives the exact identity $\mathrm{Cov}(u \mid x_t) = \mathrm{Cov}(x_1 \mid x_t)/(1-t)^2 = \mathrm{Cov}(x_0 \mid x_t)/t^2$, so the irreducible term is the posterior variance of the data, equivalently of the noise, rescaled by the interpolation weight. Each boundary can then be read off through whichever of the two variables x_t leaves free, and requires nothing beyond their independence. At the data end the noise enters x_t with weight $(1-t) \to 0$, so x_t carries no information about it, $\mathrm{Cov}(x_0 \mid x_t) \to \mathrm{Cov}(x_0)$, and the loss rises toward the full per-coordinate noise variance $\mathrm{Var}(x_0) = 1$. At the noise end the data enters with weight $t \to 0$, so $\mathrm{Cov}(x_1 \mid x_t) \to \mathrm{Cov}(x_1)$ and the loss rises toward the per-coordinate data variance $\mathrm{Var}(x_1) = \sigma^2$. Both ends therefore expose a full marginal variance, but of different variables, which also fixes their order: normalized images carry a lower per-coordinate variance than standard normal noise, $\sigma^2 < 1$, so the data boundary is the taller arm, as measured (Table 5.4). The interior minimum is not settled by this argument. It requires the posterior variance of the data to fall with t fast enough to outrun the $1/(1-t)^2$ amplification, which is a property of the data distribution rather than of the interpolant, and we do not pursue it here.

The converged U-shape therefore shows the irreducible term almost on its own. Training spends the reducible part within the first few epochs, which is the gap between the near-flat epoch-1 profile and the final shape, and once that part is exhausted the per-bin loss stops moving. This leaves an online loss-reduction policy only a brief early window in which the reward carries exploitable per-timestep structure, matching the short exploratory phase and mean reversion of ATS in Section 5.2.1.

The gradient variance is also U-shaped, but dominated by the opposite arm. The gradient statistics order the timesteps differently from the loss. On the instrumented uniform rerun we estimate, every tenth epoch, the per-sample gradient variance $\mathrm{tr}\,\mathrm{Cov}(g)$ and the mean gradient of each bin from micro-batch probes (Equation (5.5)). The mean gradient $\bar{g}_b = \mathbb{E}[g \mid t \in b]$ is the systematic direction a bin would pull in under infinite averaging, estimated at each probe by the micro-batch average $\hat{g}_b$, and its squared norm $\|\bar{g}_b\|^2$ is the learnable signal the bin carries. Across t this signal varies by less than a factor of two (Table 5.4), so the imbalance lies in the noise of the gradient rather than its direction. The variance profile is U-shaped as well, but its dominant arm sits at the noise boundary (Figure 5.14, right). Averaged over the last four probes it reaches $\sim$14 in the noise-proximal bin, more than twenty times the interior trough of $\sim$0.6, and rises again only to $\sim$1.3 at the data boundary. Near the noise boundary the mean gradient stays small while the per-sample spread around it is largest, because x_t then carries almost no information about x_1 and the target $u = x_1 - x_0$ points toward whichever data example the pair happens to draw, scattering across the data distribution and canceling in the mean.

To weigh both metrics at once we define a per-bin gradient signal-to-noise ratio $\|\bar{g}\|^2 / \operatorname{tr}\operatorname{Cov}(g)$, the share of a bin's gradient that is systematic signal rather than per-sample noise. It is lowest at the noise boundary at ~ 0.02, against ~ 0.1 to ~ 0.25 over the interior and ~ 0.12 at the data boundary (Table 5.4), and the uniform curve of Figure 5.16 in Section 5.3.2 traces the same profile across bins. The noise boundary is the highest-variance region at every probe and the U-shape of the gradient variance holds throughout training, while the overall level falls with the fit until the residual shape variation sits within probe noise. The one contrast with the DDPM setting is where the peak sits. Under ϵ-prediction the variance concentrates at the *data* boundary (Kim et al., 2025a, Figure 2), under CondOT velocity prediction at the *noise* boundary, in each case the end where the regression target enters the input with the smallest coefficient and is least identifiable from it. For the ϵ-objective that end is the low-noise, data-proximal one, where ϵ enters $x_t = \sqrt{\bar{\alpha}_t}\, x_1 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$ with weight $\sqrt{1-\bar{\alpha}_t} \to 0$ and its conditional variance returns to the prior, $\operatorname{Var}(\epsilon \mid x_t) \to \operatorname{Var}(\epsilon) = 1$. For CondOT it is the noise end, where the data x_1 enters x_t with weight $t \to 0$.

H2.1 holds. Across the timestep axis the loss varies by about $3\times$ and the per-sample gradient variance by about $20\times$, so the imbalance that motivates adaptive timestep sampling in the DDPM setting is present under CondOT Flow Matching as well, in both quantities on which that motivation rests. Its structure differs in one respect and is unfavorable in two. The loss agrees with the DDPM case in peaking at the data boundary, but the gradient variance peaks at the noise boundary, so under CondOT the two difficulty measures sit at opposite ends and disagree about where training is hardest, where under ϵ-prediction they coincide. For an adaptive t-sampling policy the landscape offers little to exploit, since the loss profile is set within the first few epochs and stationary thereafter. This could be explained by its maximum being dominated by the irreducible conditional variance of the target rather than a reducible deficit the policy could chase.

5.3.2 Does Biasing the Timestep Distribution Reshape the Loss and Gradient Landscapes?

Hypothesis. H2.2 asks whether a biased sampler changes the per-bin loss and gradient-variance landscapes, or only how often each bin is visited during training. The distinction is explanatory. If a biased sampler reshaped the landscapes, each prior would train on a measurably different per-timestep objective, and the FID differences among the static priors of E1 could be traced to differences in their per-bin training statistics. If it does not, a static prior can act only through its *compute allocation*, the fraction of each batch it spends in each timestep regime, and any FID difference it produces must be priced by the fixed landscapes of Section 5.3.1 rather than by a landscape the sampler reshaped. We test both landscapes for sampler dependence on uniform sampling and the two opposite Bernoulli priors, Bernoulli (data, 0.8) and Bernoulli (noise, 0.2), whose densities differ by almost an order of magnitude at the boundaries (Figure 5.15, bottom panel).

The loss landscape is invariant to the sampler. The clean test uses the per-bin *validation* loss, measured under uniform-in-bin t on held-out data, which is independent of how the training sampler places its own draws. On the uniform run this validation profile matches the training

profile of Section 5.3.1 to within 0.004 on average, so the two report the same landscape, and for a biased sampler only the validation loss stays free of the sampler's within-bin draw placement. Across the three samplers the per-bin validation loss is the same U to within 0.007 at every bin (Figure 5.15, left), even though Bernoulli (data, 0.8) draws under 3% of its samples from each noise bin while Bernoulli (noise, 0.2) commits a quarter of its batch to each of them. Starving a regime does not raise its held-out loss and flooding it does not lower it. The per-bin loss is a property of the objective and the data, not of the sampler that visits it, in line with the finding of Section 5.3.1 that the converged per-bin loss is essentially the irreducible floor.

The gradient-variance landscape keeps its shape. The gradient-variance landscape is nearly as rigid. All three reruns reproduce the same noise-dominated U, with the noise boundary an order of magnitude above the interior trough for every sampler (Figure 5.15, right). What moves is the overall level. Bernoulli (noise, 0.2) sits roughly a quarter below uniform across the whole schedule, 9.8 against 13.6 at the noise boundary and 0.50 against 0.61 at the interior trough, while Bernoulli (data, 0.8) sits slightly above uniform at 15.9 at the noise boundary. Each per-bin offset is about the size of the averaged probe's 25% precision, so no single bin resolves it, but the sign of the offset is consistent across all ten bins, which points to a small genuine difference between the three trained models rather than probe noise. Because the offset is roughly uniform in t instead of localized to the bins whose visitation changed, it is a level shift of the whole model and not a per-bin response to starving or flooding, and it leaves the shape, the bin ordering, and the order-of-magnitude gap between the noise boundary and the interior intact for every sampler.

The landscape prices the bias directions asymmetrically. With reshaping ruled out, a static prior acts only through where it allocates the fixed sample budget of each batch, and the gradient SNR prices that allocation (Figure 5.16). The SNR of a bin's gradient, defined as $\|\bar{g}\|^2 / \operatorname{tr} \operatorname{Cov}(g)$, is the fraction of its gradient that is systematic signal rather than per-sample noise, so a low-SNR bin contributes mostly noise to the shared batch gradient and buys little learning per update spent there. All three samplers agree that the two noise-proximal bins are the low-SNR regime, about an order of magnitude below the interior. The two bias directions therefore face very different prices. Bernoulli (noise, 0.2) places a quarter of every batch in each of the two noise bins, so half of every update lands in the low-SNR regime, and it posts the worst FID once the collapsed LTE-ATS is excluded (17.34 against 13.83 for uniform). Uniform sampling already spends only about a fifth of each batch there, so a data-biased prior that reclaims that fifth can gain at most a little (13.60 for Bernoulli (data, 0.8)). The shape of the SNR profile is thus consistent with the asymmetry of the E1 ranking, a small reward for avoiding the noise boundary against a large penalty for oversampling it, although the training-side SNR alone does not finally prove that gradient noise is the operative mechanism.

H2.2 is refuted. Static priors whose densities differ by almost an order of magnitude leave the per-bin loss landscape unchanged and the gradient-variance landscape unchanged in shape, moving only its overall level. What separates the samplers of E1 therefore does not seem to be the landscape they train on but the allocation they choose on it. The gradient SNR profile of that fixed landscape prices their allocations in a way that is consistent with the observed FID ranking.

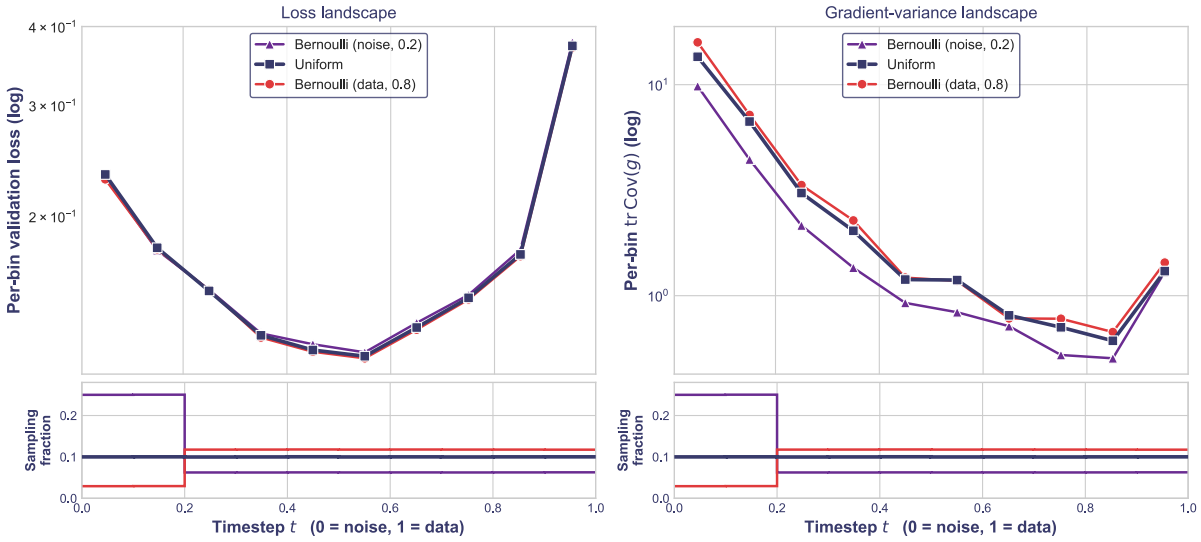


Figure 5.15: **Biasing the sampler leaves the loss landscape unchanged and the gradient-variance landscape unchanged in shape.** Uniform sampling against the two opposite Bernoulli priors, whose sampling densities (bottom panel, per-bin mean fraction drawn as step lines that are flat within a bin and jump at the bin edge, uniform flat at the reference value 0.1, both Bernoulli densities piecewise-uniform with their break at $t = 0.2$) differ by almost an order of magnitude at the boundaries. *Left:* per-bin validation loss (uniform-in-bin t, held-out data, mean over the last 20 epochs, log scale), the sampler-independent loss landscape, where the three curves coincide to within 0.007 everywhere. *Right:* per-bin gradient variance $\mathrm{tr}\,\mathrm{Cov}(g)$ (log scale; mean over the last four probes, which suppresses the $\approx 50\%$ single-probe estimation noise), where the three share the same noise-dominated U shape and differ only by a modest overall level shift.

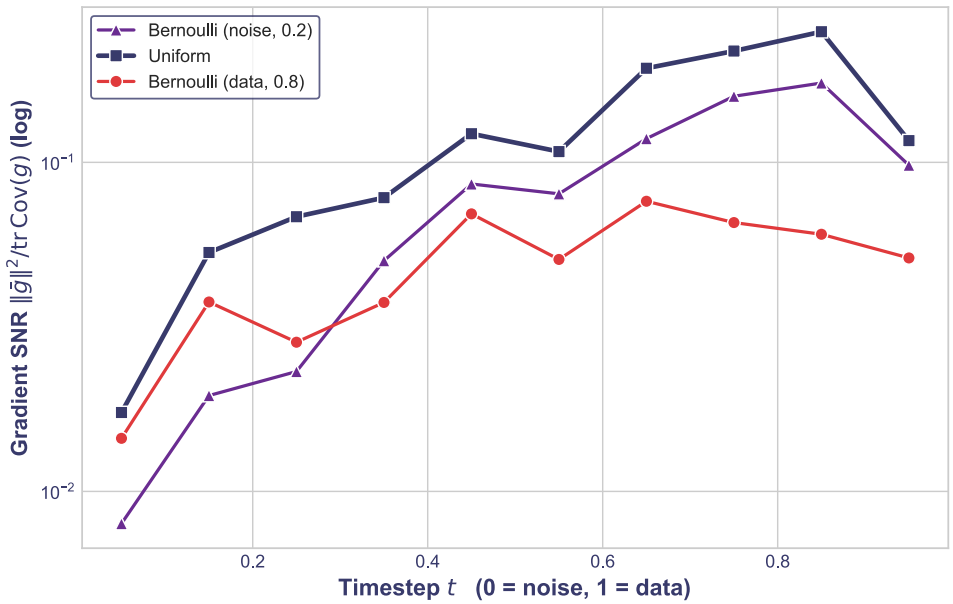


Figure 5.16: **The gradient SNR marks the noise boundary as the low-value regime for every sampler.** Per-bin gradient signal-to-noise ratio $\|\bar{g}\|^2 / \mathrm{tr}\,\mathrm{Cov}(g)$ (log scale) for uniform sampling and the two opposite Bernoulli priors. All three place the two noise-proximal bins about an order of magnitude below the interior, so a training draw spent there contributes mostly per-sample noise to the batch gradient and advances the fit little. This prices the two bias directions asymmetrically (Section 5.3.2), and the cross-sampler differences in the interior are within the probe's estimation noise.

5.3.3 E2 Synthesis

H2.1 holds and H2.2 is refuted. The timestep axis of CondOT Flow Matching is genuinely non-uniform in the two quantities whose imbalance motivates adaptive timestep sampling in the DDPM setting (Kim et al., 2025a). The loss is U-shaped with its maximum at the data boundary, and the gradient variance is a second, opposite U, spanning about $20\times$ with its maximum at the noise-proximal end, over a mean gradient whose squared norm varies by less than a factor of two. Both landscapes are fixed properties of the objective. The loss profile is set within the first few epochs and stationary thereafter, and the variance profile keeps its noise-boundary maximum and overall shape from the first probe onward, with only noisy reordering among the interior bins late in training. The data- and noise-biased Bernoulli samplers, whose densities differ by almost an order of magnitude, leave the per-bin loss unchanged and the variance unchanged in shape, moving only its overall level. A sampler chooses where to spend its updates, and has no impact on how hard or how noisy the objective is at a timestep.

This offers an answer to both questions E1 left open. First, the premise of ATS survives the transfer to CondOT Flow Matching, but every property an *online* policy needs does not. The loss-reduction reward decays to zero once the reducible part of the landscape is exhausted after the first few epochs, the stationary profile offers nothing to track thereafter, and the one axis along which timesteps most differ, the gradient variance, is not one a loss-reduction reward directly observes. This could explain from the backbone's side why ATS explores briefly and then reverts to uniform (Section 5.2.1), and why a static prior captures whatever the temporal structure offers without the adaptive overhead. Second, because the landscapes are invariant, a static prior acts only through its allocation, and the SNR profile prices the two bias directions asymmetrically. Avoiding the low-SNR noise boundary frees only the small share of the batch that uniform sampling wastes there, a small gain, while oversampling it spends half of every update in the low-SNR regime on near-pure gradient noise that advances the fit little, a large and avoidable cost. This is consistent with the E1 ranking and its magnitudes. Whether the reallocated timesteps leave an imprint on the learned flow that the sampling process follows at generation time is the complementary, generation-side question that Section 5.4 takes up.

5.4 E3 – Impact on Inference Trajectories

The timestep samplers act exclusively during training. E1 compared the generative performance of the different t-sampling strategies, and E2 analyzed how the resulting per-t-bin training statistics, namely the loss landscape and its gradient signal, shape the learning dynamics.

E2 priced the sampling biases from the training side alone and left open whether the reallocated updates also leave a notable imprint on the learned velocity field that the ODE integrates at generation time (Section 5.3.3). E3 therefore examines whether the samplers differ not only in their training statistics but also in inference, in the geometry of the trajectories produced by the fully trained models. We test one joint hypothesis.

- **H3.** The timestep sampling distribution applied during training impacts the inference trajectories of the trained model.

We probe this imprint with three geometric diagnostics of the learned marginal flow (Sec-

tion 5.1.4).

Methodology. All diagnostics are computed post-training with the final models (300 epochs) held fixed. The sampler only ever shaped the training distribution, so any geometric difference at inference is a property that the training bias left in the learned marginal field. For each strategy we integrate the probability-flow ODE $\dot{x}_t = v_\theta(x_t, t)$ from $t = 0$ to $t = 1$ with a uniform-step Euler solver ($N = 50$) from a set of 256 noise draws shared across all strategies and record the three trajectory diagnostics defined in Section 5.1.4, namely the *path deviation*, the time-integrated *kinetic transport energy* W_{kin}, and the Euler *local-truncation-error proxy* of Section 4.3. Because every strategy integrates the same noise set, the comparison is paired, and each metric is reported as its mean over the 256 trajectories together with the standard error of that mean, so the tabulated $\pm$ is the uncertainty of the mean estimate and separates a genuine between-strategy difference from the far larger trajectory-to-trajectory scatter. Figures 5.18 and 5.19 and Table 5.5 summarize the results for all ten strategies of the suite.

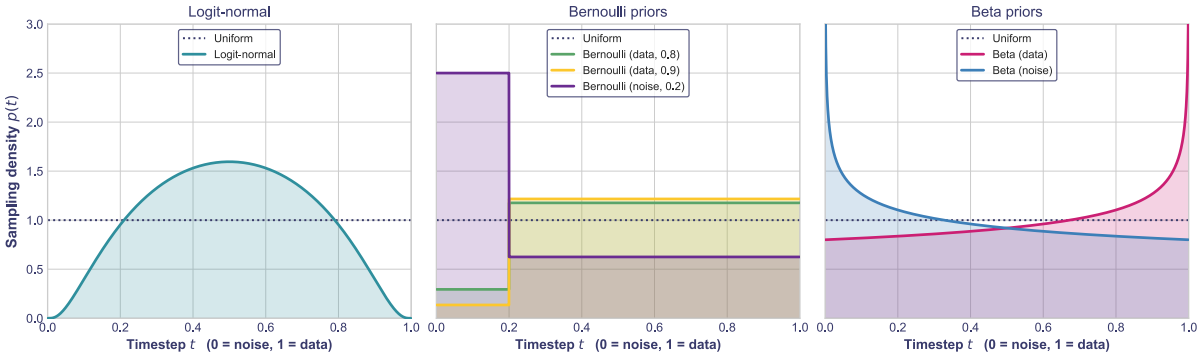


Figure 5.17: **Training-time sampling densities, grouped by family on a shared density axis with the uniform density (dotted) as the reference in every panel.** Every trajectory launches from the noise manifold at $t = 0$, so the density at $t \to 0$ measures how much training mass a prior withdraws from the launch regime. *Left:* logit-normal drives its density to zero at both boundaries. *Middle:* the data-biased Bernoulli priors drop the noise-boundary density to 0.29 (data, 0.8) and 0.14 (data, 0.9), while Bernoulli (noise, 0.2) raises it to 2.5. *Right:* Beta (data), though nominally data-biased, keeps a near-uniform density of 0.8 at the noise boundary, far above the data-biased Bernoullis, whereas Beta (noise) concentrates there. The Beta densities diverge at their bias boundary and are clipped at the panel ceiling.

The timestep prior leaves no imprint on the inference geometry. The trajectories of a single model vary far more among themselves than the models vary from one another, so the training timestep distribution does not shape the learned geometry in any way that matters. The trajectory-to-trajectory standard deviation of the path deviation is 7.0×10^{-3}, about a third of the mean and roughly $13\times$ the 0.5×10^{-3} spread of the nine converged model means, and the kinetic energy shows the same 12-to-1 ratio. An order of magnitude of variation in the training density (Figure 5.17) thus collapses onto one shared inference geometry. Every converged model sits in the same narrow band, with a path deviation between 20.7 and 22.5×10^{-3}, a kinetic-energy spread of 0.8%, and a truncation error within a few percent for eight of the nine (Table 5.5), and the per-t profiles superimpose almost exactly (Figures 5.18 and 5.19). The three near-uniform

strategies, uniform, ATS, and A2C-ATS (Section 5.2.1), agree most tightly of all, their means separated by no more than a standard error on every diagnostic, as their realized densities are effectively identical and the residual differences are run-to-run noise.

Logit-normal is the one resolvable outlier. Logit-normal is the only prior that starves both ends of the schedule, driving its training density to almost zero at the noise and the data boundary alike (Figure 5.17), and it is the only strategy whose inference geometry separates from the pack. The separation splits into a velocity-magnitude effect and a curvature effect that point in opposite directions but share one cause. Its kinetic energy is the highest of the suite at every step budget, 3364 ± 21 against uniform's 3290 ± 19 at $N = 50$ (standard errors of the mean over the 256 paired trajectories, next to a per-trajectory spread near 338 that hides any single trajectory), because the starved noise boundary is underfit. Its reducible loss there is not yet exhausted (Section 5.3.3), so the launch velocity has not settled to the minimal value the other models reach, the conditional mean $\mathbb{E}[x_1 - x_0 \mid x_0] = -x_0$ of norm $\|x_0\|_2 \approx \sqrt{3072}$, and a non-converged field still carries part of the larger per-sample target $x_1 - x_0$ and takes a slightly longer step. Curvature moves the other way. At the full budget logit-normal is the straightest field of the suite, lowest in both path deviation and truncation error, because the interior it does train is where a well-resolved trajectory spends most of its length. A fast flow and a straight one are not in conflict, since the kinetic energy measures how quickly the flow moves and the curvature diagnostics measure how much it turns.

The same boundary starvation governs its behavior as the step budget shrinks (Figure 5.21). At the coarsest budgets its path deviation, at five steps, and its truncation error, at a single step, are the highest of the converged strategies, the reverse of their full-budget ranking. A handful of large Euler steps is dominated by the first step out of the anomalous noise boundary rather than by the well-fit interior, and only once the grid resolves the schedule, from about ten steps on, does the straight interior show through and return logit-normal to the lowest curvature. Its high launch velocity is an advantage in this regime, carrying samples toward the data manifold in fewer steps, so its FID is the best of the suite at five steps (32.4 against 35 to 39) before it rejoins the pack by ten and settles mid-field at fifty. None of this reaches sample quality at the standard budget, where its FID is competitive with uniform (Table 5.2). We read the whole pattern as a boundary artifact peculiar to the one prior that removes almost all density from both ends, not as a geometric imprint of the timestep distribution.

The curvature diagnostics share a late-schedule signature. Although the strategies are indistinguishable in aggregate, the shape that each diagnostic traces over t is systematic, identical across all of them, and informative in its own right (Figure 5.18). The path deviation begins near zero at $t \to 0$, holds close to 0.02 through the interior of the schedule, and rises steeply to about 0.12 only as $t \to 1$. The metric mixes a cumulative reference with an instantaneous reading. The reference chord $x_t - x_0$ is the path integrated up to t, so it carries the whole history of the trajectory, whereas the value reported at t is the angle between that chord and the current velocity at that instant, not a running sum of the earlier angles. The near-zero value at the launch regime is in part a property of this construction, because over the first steps the chord is dominated by the nearly constant initial velocity and stays almost collinear with it. The flat interior then reflects a genuinely near-straight field in which the velocity remains aligned with the net displacement. The steep late rise, by contrast, is a property of the velocity field itself.

As the trajectory approaches the data manifold the field must turn to commit to an individual mode, so the velocity rotates away from a chord that has by then integrated a mostly straight interior, which is the same irreducible late conditional-target variance that E2 located in the loss landscape (Section 5.3.1).

The truncation error traces a U over t that agrees with this reading. It is elevated near the noise boundary, falls to a minimum near $t \approx 0.45$, and rises to a sharp maximum at $t \to 1$. Because the proxy measures the change in velocity between consecutive steps, its interior minimum marks the regime where the field is most nearly constant and its late spike marks the same sharp turning toward the data manifold that drives the path deviation. The smaller elevation at $t \to 0$ has a different origin and we read it more cautiously. The mean field is smooth there, since $\mathbb{E}[x_1 - x_0 \mid x_0] = -x_0$ varies gently with the initial noise, so the bump is not directional turning. It reflects instead the velocity magnitude rising as the flow accelerates off the noise manifold, which a second-difference proxy registers in part, together with the one-sided finite difference available at the first step. The late spike, by contrast, is genuine turning of the field. Both curvature diagnostics therefore agree that the learned marginal flow is near-straight through its interior and does almost all of its directional turning in the final tenth of the schedule. This describes the geometry of the path, not the speed along it. The kinetic power follows a separate profile, rising from the launch value to a mid-schedule maximum and easing as the field settles onto the data manifold (Figure 5.19), so the direction of motion is nearly fixed through the interior while its magnitude is not, and the truncation-error minimum near $t \approx 0.45$ marks where both change most slowly. The kinetic power also reads as a measure of conditional agreement. The marginal velocity is the average $\mathbb{E}[x_1 - x_0 \mid x_t]$ of the per-sample targets, so its magnitude falls below the average target length in proportion to how much those targets disagree, and the flow accordingly slows at both boundaries, most of all late, where the conditional-target variance that E2 identified is largest (Section 5.3.1).

The late truncation-error maximum accounts for the LTE-ATS collapse. This late maximum of the truncation error is also what explains the failure of LTE-ATS from the inference side, since the proxy plotted here is the very reward that policy optimized (Section 4.3). The reward is largest at $t \to 1$ and cannot be reduced there, because training shapes only the conditional residual and leaves the marginal curvature mostly intact (Section 5.2.3). The late mode is therefore absorbing, and the policy collapsed onto it ($\mathbb{E}[t] = 0.994$, Section 5.2.3), so the collapse was a correct response to a real and stationary feature of the field rather than optimization noise.

Cutting the step budget costs every model the same. A budget of fifty Euler steps is generous at this resolution, so a difference could in principle remain hidden until the trajectory is integrated with far fewer function evaluations, where discretization error dominates. We therefore recomputed both FID and the trajectory diagnostics over a sweep $N \in \{1, 5, 10, 20, 50\}$ on the same epoch-300 models, with noise paired across budgets and strategies (Figures 5.20 and 5.21). The two views agree that the nine converged models fall together at every budget. Their FID lies within $[17.3, 18.8]$ at ten steps for all but Bernoulli (noise, 0.2), and every arm inflates by about the same factor as the budget drops, roughly $3.5\times$ its fifty-step score at five steps, while their path deviation and truncation error stay superimposed, the truncation error growing as N^{-2} and the path deviation shrinking as a coarser grid under-samples the late turn. Reducing the step count therefore exposes no hidden geometric gap and degrades the well-trained samplers in step,

Table 5.5: **Trajectory diagnostics of the learned marginal fields.** Euler solver, $N{=}50$, with every strategy integrating the same 256 noise draws and metric definitions given in Section 5.1.4. Cells give each mean $\pm$ its standard error, that is, the per-trajectory standard deviation over the 256 paired draws divided by $\sqrt{256} = 16$, averaged over t unless noted; the $\pm$ therefore measures how precisely each mean is estimated and is what the between-strategy comparison rests on, while the trajectory-to-trajectory scatter is $16\times$ larger and is shown by the interquartile bands of Figure 5.18. W_{kin} is the time-integrated squared velocity magnitude and upper-bounds the squared W_2 transport cost. Across the nine converged strategies every diagnostic is constant to within run-to-run variation, with a path-deviation coefficient of variation of 2.4%, a W_{kin} spread of 0.8%, and a truncation error within a few percent for eight of the nine, so no column admits a meaningful "best" value. The LTE-ATS row (†) is reported for completeness but is not comparable to the others. Its backbone collapsed and defines no generative flow (Section 5.2.3), so its favorable-looking low W_{kin} and low truncation error describe a nearly constant, low-magnitude field that never reaches the data manifold rather than an efficient sampler, and it is excluded from Figure 5.18.

Sampler	Path dev. ($\times 10^{-3}$)	Path dev. at $t{\to}1$ ($\times 10^{-3}$)	W_{kin}	LTE ($\times 10^{-3}$)
Non-adaptive				
Uniform	22.26 ± 0.44	126 ± 2.7	3290 ± 19	3.97 ± 0.038
Logit-normal	20.71 ± 0.42	116 ± 2.5	3364 ± 21	3.07 ± 0.056
Bernoulli (data, 0.8)	21.73 ± 0.44	121 ± 2.7	3302 ± 19	4.00 ± 0.036
Bernoulli (data, 0.9)	21.55 ± 0.43	120 ± 2.7	3299 ± 19	3.99 ± 0.036
Beta (data)	22.09 ± 0.43	124 ± 2.7	3289 ± 19	3.96 ± 0.034
Beta (noise)	22.29 ± 0.43	126 ± 2.7	3286 ± 19	3.94 ± 0.037
Bernoulli (noise, 0.2)	22.52 ± 0.44	130 ± 2.6	3276 ± 18	4.01 ± 0.041
Adaptive				
ATS	21.94 ± 0.44	124 ± 2.7	3293 ± 19	3.98 ± 0.037
A2C-ATS	21.95 ± 0.44	122 ± 2.6	3301 ± 19	3.96 ± 0.036
LTE-ATS†	50.22 ± 0.10	121 ± 0.3	1341 ± 2	1.56 ± 0.004

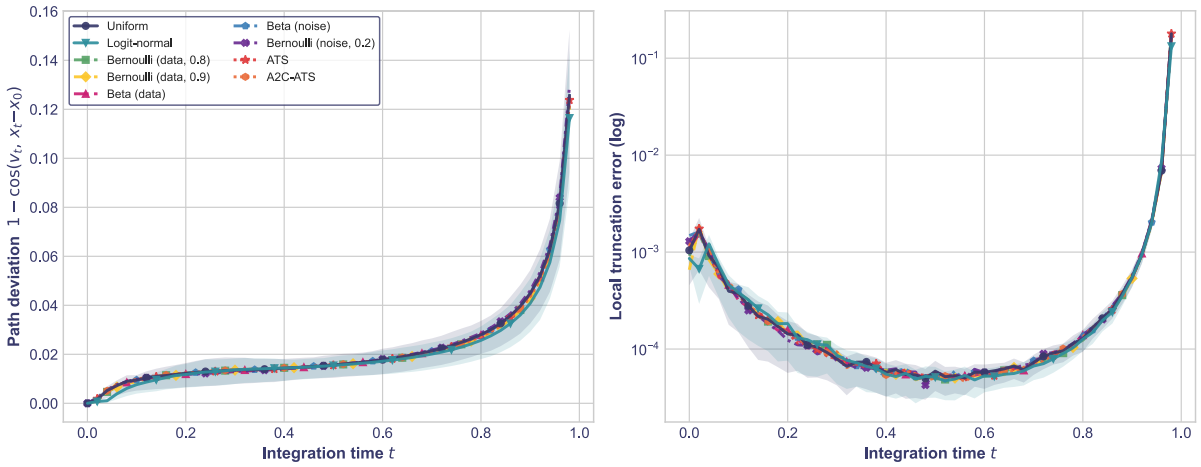


Figure 5.18: **Curvature diagnostics of the fully trained models along the generation trajectory (Euler, N=50, 256 trajectories).** *Left:* path deviation, near 0.02 through the interior and rising to about 0.12 as $t \to 1$. *Right:* the Euler–Heun truncation-error proxy on a log scale, a U in t minimal near $t \approx 0.45$ and spiking at $t \to 1$. Line patterns group the sampler families (solid: uniform and logit-normal; dashed: data-biased; dash-dotted: noise-biased; dotted: adaptive) and markers identify the strategies; LTE-ATS is tabulated only (Table 5.5). Shaded bands give the interquartile range over the 256 paired trajectories for uniform and logit-normal, whereas Table 5.5 reports each mean with its standard error. The kinetic power is shown separately in Figure 5.19. The nine converged strategies superimpose on both panels.

so few-step quality is governed by the discretization error shared by an equally straight family of flows. The two departures are the ones E1 already isolated, the collapsed LTE-ATS sitting apart on every diagnostic and Bernoulli (noise, 0.2) riding a constant three to four FID above the pack, a fixed loss of final fidelity rather than a low-N cliff. The only arm that moves against the pack is logit-normal, which leads at the coarsest budgets and reverses its curvature ranking as the grid changes. Both follow from its boundary starvation and are treated above, and neither disturbs the full-budget ranking of E1.

5.4.1 E3 Synthesis

H3 is refuted in effect. The timestep distribution used during training leaves no imprint on the inference geometry of the trained model large enough to matter. An order of magnitude of variation in the training density collapses onto a single shared inference geometry: across the nine converged strategies the path-deviation, kinetic-energy, and truncation-error profiles superimpose almost exactly, and the spread of the model means is roughly an order of magnitude below the trajectory-to-trajectory scatter within any one model. The paired design over 256 shared noise draws pins each mean tightly enough to resolve a few of these residual gaps as statistically significant, most visibly logit-normal's elevated kinetic energy, but each is a small fraction of a single trajectory's variation and none moves sample quality, logit-normal's FID remaining competitive despite carrying the strongest mid-schedule bias in the suite (Table 5.2). That one resolvable trace is moreover an artifact rather than a geometric imprint, an elevated launch velocity at the noise boundary its density starves, where an underfitted field has not yet

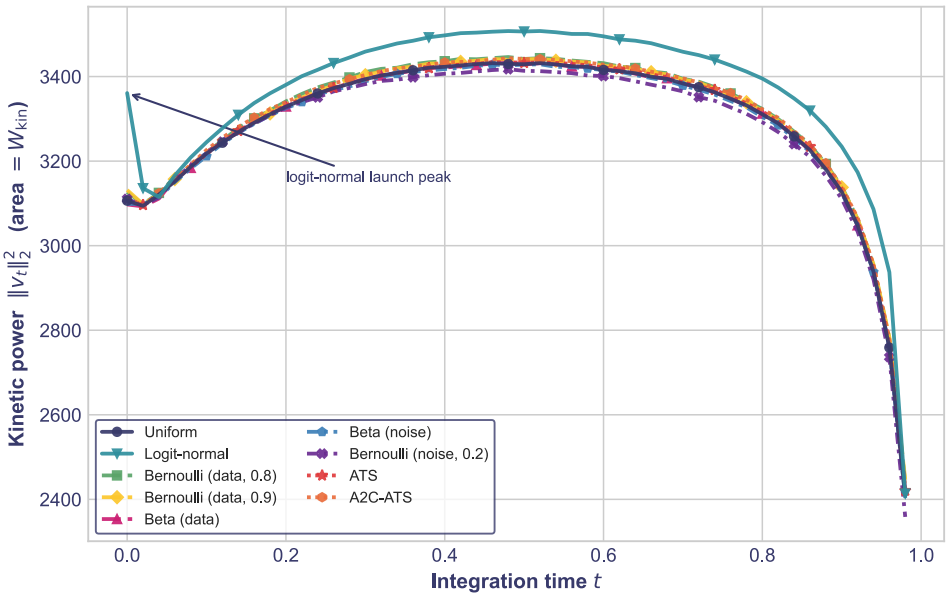
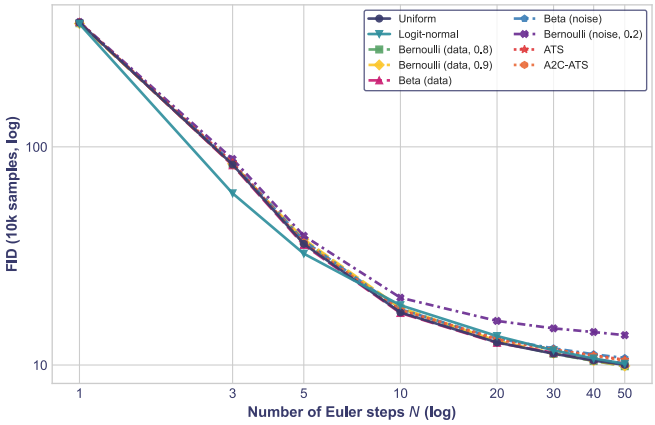


Figure 5.19: **Kinetic power $\|v_t\|_2^2$ along the generation trajectory (Euler, N=50, 256 trajectories), whose area over t is W_{kin}.** The field starts near the minimum-norm launch value $\|x_0\|_2^2 \approx 3072$ at $t \to 0$, accelerates to a mid-schedule maximum, and decelerates as it settles onto the data manifold. Logit-normal alone sits systematically above the pack, with a sharp launch peak at the noise boundary its density starves, the clearest trace of a training bias on the learned field, a boundary artifact small against the trajectory-to-trajectory scatter. Curves are means over the 256 paired trajectories, and LTE-ATS is omitted for legibility.



Sampler	1	5	10	20	50
Uniform	371.1	35.9	17.4	12.7	10.0
Logit-normal	365.8	32.4	18.8	13.6	10.1
Bernoulli (data, 0.8)	368.7	37.1	18.2	12.8	9.9
Bernoulli (data, 0.9)	367.4	37.7	18.3	12.9	9.9
Beta (data)	373.1	35.5	17.3	12.6	10.2
Beta (noise)	371.9	37.1	18.0	13.2	10.8
Bernoulli (noise, 0.2)	373.2	39.1	20.4	15.9	13.7
ATS	372.5	36.2	17.8	13.1	10.6
A2C-ATS	370.9	36.5	17.9	13.2	10.5
LTE-ATS†	404.7	398.0	397.2	395.4	395.1

Figure 5.20: **FID versus the number of Euler steps N.** 10,000 samples with noise paired across N and strategies. *Left:* log–log curves. *Right:* values at $N \in \{1, 5, 10, 20, 50\}$. The nine converged strategies collapse onto a shared curve, so cutting the budget costs each of them the same, while Bernoulli (noise, 0.2) rides a constant offset above the pack and the collapsed LTE-ATS (†) stays near 395 at every budget. The absolute scores are a dedicated recomputation on the epoch-300 checkpoints and lie ≈3.7 FID below the training-time values of Table 5.2, an offset of the separate generation draw rather than a finite-sample effect. It preserves what the comparison rests on, the isolation of Bernoulli (noise, 0.2) and the collapse of LTE-ATS; the re-ordering inside the sub-one-point band of the remaining strategies is itself evidence that those margins are draw noise.

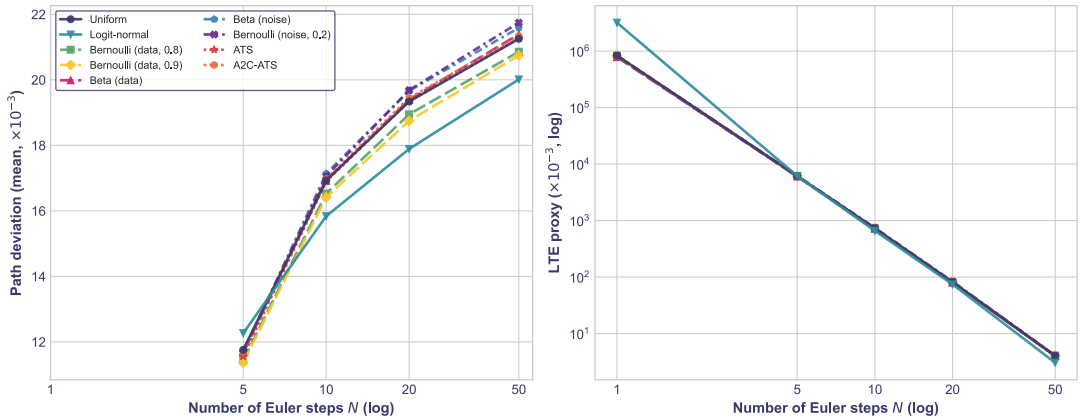


Figure 5.21: **Trajectory diagnostics versus the number of Euler steps N (log x-axis).** *Left:* path deviation. *Right:* the Euler–Heun truncation-error proxy (log scale). Both are recomputed on the epoch-300 models with noise paired across budgets and strategies. The nine converged strategies stay superimposed at every budget, mirroring the FID sweep of Figure 5.20; the truncation error grows as roughly N^{-2} and the path deviation falls as the coarse grid under-samples the late turn. The single-step budget has no interior and is omitted from both, and LTE-ATS is excluded for legibility.

reached the minimum-norm target $-x_0$ the objective prescribes.

The shared geometry is itself informative. Every field is near-straight through the interior of the schedule and does almost all of its directional turning in the final tenth, so both curvature diagnostics rise to a late maximum as the field commits to an individual data mode, the same irreducible late conditional-target variance E2 located in the loss landscape (Section 5.3.1). This late maximum is a stationary property of the learned field that training at $t \to 1$ cannot reduce, and it is precisely the truncation-error reward the LTE-ATS policy chased into its collapse (Section 5.2.3), so that collapse was correct reward-following rather than optimization noise. Cutting the Euler budget from fifty steps to as few as five costs every converged model the same, in both FID and the trajectory diagnostics, so the shared geometry exposes no hidden low-NFE gap either.

Taken with E1 and E2, this closes the case on the sampler from all three sides. A timestep prior acts only by reallocating where training updates are spent (E2), which the SNR profile prices into generation quality (E1) but which leaves the learned marginal field, and therefore the inference trajectories integrated from it, effectively untouched, once converged (E3). The adaptive policies, having reverted to uniform, hold no inference-side lever either. Whatever a timestep prior is worth is realized in the economics of training, not in a straighter or cheaper-to-integrate generative flow.

6 Conclusion

Training modern diffusion and Flow Matching generators is computationally expensive, and part of that expense stems from a suboptimal allocation of training compute across the timestep axis under the standard uniform t-sampling scheme. The per-timestep difficulty of the Flow Matching loss is non-uniform, yet a fixed sampling density cannot react to the changing capacity of the network as it learns. ATS, proposed by Kim et al. (2025a) for denoising diffusion models on the VP path, addresses this by learning the timestep distribution online through reinforcement learning, so that each update is spent where it promises the largest loss reduction. This thesis asked whether ATS transfers to the CondOT Flow Matching setting (Lipman et al., 2023; Liu et al., 2023), whose straight probability paths keep inference cheap. Three aspects were left open by the original formulation, namely its transposition from an ε-prediction to a velocity-regression objective, the stabilization of its REINFORCE policy-gradient estimator whose high variance is a known property of the algorithm, and the training-only focus of its reward, which ignores the geometry of the sampling trajectory at inference. Experiments E1 to E3 address these questions. The adaptive methods we study do not improve on a well-chosen static prior in this setting, but their failures are informative, and the accompanying analyses of the CondOT learning process (E2) and of the inference-trajectory geometry (E3) contribute insights into Flow Matching training that stand independently of that negative result.

6.1 Summary of Contributions

To address these questions we made three methodological contributions, each paired with an empirical analysis. We first ported ATS from discrete ε-prediction to continuous velocity regression, driving the REINFORCE policy that learns the timestep distribution with a reward equal to the reduction of the Conditional Flow Matching loss, and benchmarking it against uniform sampling and a suite of static non-uniform priors under a shared U-Net backbone on CIFAR-10 (E1). Motivated by the high variance characteristic of REINFORCE estimators, we then introduced an Advantage Actor-Critic variant, A2C-ATS, which subtracts a learned value-function baseline from the loss-reduction reward (E1). Finally, we designed LTE-ATS, an ATS variant whose reward is derived from the Euler local truncation error and is intended to steer training toward the high-curvature timesteps at which a first-order solver accumulates the most error (E1). To interpret the learning behavior of Flow Matching models, we characterized the per-timestep loss and gradient-variance landscapes of the CondOT objective and tested whether a fixed sampling bias reshapes them (E2), and we analyzed whether the timestep density in training is imprinted on the curvature and integration cost of the learned flow at generation time (E3).

6.2 Summary of Findings

Adaptive sampling gives no benefit over uniform. This is the central negative result. At equal optimizer steps the validation loss of ATS (0.1694) lies within one percent of uniform (0.1709), and its FID is slightly worse (14.18 against 13.83). The adaptive methods also carry a compute premium, a 5.1% increase in backbone forward-pass equivalents and, on our hardware, a 9.6% increase in wall-clock time, so once this cost is charged the adaptive curves never lead uniform (Section 5.2.1). This outcome does not arise from an immobile policy. The learned sampler explores during the first ten epochs, with the mean sampled timestep swinging between 0.34 and 0.60, and then reverts to the uniform $\mathrm{Beta}(1, 1)$ equilibrium for the remainder of training. The reversion follows directly from the reward, whose magnitude on the straight CondOT interpolant is minuscule and decays from about 4×10^{-3} to 10^{-4} as the backbone converges. Once the reward is effectively zero, the entropy term, which is maximal at $\mathrm{Beta}(1, 1)$, dominates the update and returns the policy to uniform.

Among fixed priors, only the noise-bias penalty is robust. Where a static prior places its mass matters, but the effect is one-sided. Biasing toward the noise boundary was consistently and unambiguously harmful, raising the FID from 13.83 for uniform to 14.62 for Beta (noise) and to as much as 17.34 for Bernoulli (noise, 0.2), a penalty E3 confirms at every step budget. The data-proximal direction, by contrast, yields no reliable gain. The best such prior, Bernoulli (data, 0.8) at 13.60, together with uniform (13.83) and the logit-normal density of Esser et al. (2024) (13.80), falls within a 0.23-point FID band that lies inside run-to-run variation, and E3 finds these trained flows geometrically indistinguishable, so the residual gap may reflect the FID estimator rather than a genuine quality difference. The exploitable structure along t is therefore not that data bias helps but that noise bias must be avoided, a constraint uniform sampling already decently satisfies at no cost, while the online policy pays a compute premium to capture nothing beyond it.

The A2C critic raises policy-gradient variance. The two adaptive variants did not recover this ground. The A2C critic raised the policy-gradient variance rather than lowering it, exceeding the baseline-free variance on 67% of updates by a median factor of about 2.9. The variance decomposition explains why. The critic-reward covariance is effectively zero ($\rho \approx 0$), so subtracting the baseline adds uncorrelated noise ($\mathrm{Var}(R - V) \approx 4.5 \times 10^{-5}$ against $\mathrm{Var}(R) \approx 2.8 \times 10^{-5}$) instead of canceling reward variance, and the variance-reduction condition of Equation (5.8) fails throughout training. Two effects could account for this, though only one is decisive in practice. In principle the reward is non-stationary, a moving target that shifts as the backbone learns and that a lagging critic would struggle to predict. In practice this matters little, because after the first epochs the loss-reduction reward is close to zero and carries almost no structure across images, so the critic has essentially nothing to predict and its output ends up uncorrelated with the target. It is therefore the weak, vanishing reward, rather than its non-stationarity, that turns the learned baseline from a variance-reducing device into a source of additional noise.

The curvature reward collapses the policy. The LTE reward failed for a different reason. Within about 1400 steps the policy collapsed onto $t \approx 0.994$, starving the backbone of every

timestep below $t = 0.9$ and destroying generative quality, with the uniform-t validation loss rising to 1.119 and the FID to 395.2. The cause is a mismatch between the conditional field optimized during training and the marginal field integrated at inference. Minimizing the conditional Flow Matching loss at the highest-curvature timesteps does not straighten the marginal velocity field that governs generation, so the curvature the reward measures never falls, the reward never decays, and the collapse never corrects itself. The LTE proxy is therefore not a suitable reward for an adaptive timestep policy.

The difficulty profile is structured but stationary. Experiment E2 explains these outcomes from the side of the backbone. The timestep axis of the CondOT objective is genuinely non-uniform in the two quantities that motivate adaptive sampling, but both settle within the first few epochs and are stationary thereafter. The loss forms a U-shape with its maximum at the data boundary, and the gradient variance forms a second, opposite U that spans about a factor of twenty with its maximum at the noise boundary. Biasing the sampler leaves both profiles essentially unchanged and alters only the fraction of compute each regime receives during training. A sampler can thus reallocate compute but cannot make a timestep easier to fit or its gradient less noisy. The gradient variance is the axis along which timesteps differ most and forms a strong, static difficulty signal that a fixed prior can act on, yet one that a loss-reduction reward does not account for, and once the reducible part of the loss is exhausted early in training that reward provides little further guidance of any kind.

Training biases leave the geometry of the flow unchanged. Finally, E3 shows that the timestep sampling distribution used during training leaves no meaningful imprint on the geometry of the learned flow. The static priors span an order of magnitude in the training mass they place near the noise boundary from which every trajectory launches, yet trajectory curvature, transport energy, and integration error vary more between the trajectories of a single model than between models. A sweep of the Euler step count from one to fifty confirms the null from the quality side, since the FID-versus-step curves of the converged strategies coincide at every budget. The small FID advantage of the data-biased priors therefore cannot be attributed to trajectory geometry. The adaptive samplers settle at uniform and inherit its geometry, and LTE-ATS departs only because its backbone collapsed in training, not because a sampling bias reshaped a working flow. Only logit-normal, which withdraws almost all training mass from both boundaries, separates from the pack by more than the estimation error, and it does so incoherently—highest in transport energy, lowest in curvature at a full step budget and highest at a coarse one—so it marks the density at which a bias first becomes visible in the flow rather than a direction in which a bias steers it.

The online signal of a loss-reduction reward is weak and short-lived. Taken together, these results indicate that on straight CondOT paths the signal an online policy would need is both short-lived and, once the early transient passes, close to absent. The loss-reduction reward is in principle a moving target, since it depends on the current state of the backbone, but in practice its batch mean is mobile only during the first few epochs and then collapses to near zero, in line with the difficulty profiles of E2, which stabilize on the same timescale. What persists is a static, non-uniform difficulty structure that a fixed prior exploits directly and more cheaply than a policy that must first estimate a vanishing reward. We measured only the batch-mean reward

and not its per-timestep profile, so whether any exploitable structure survives in that profile is a question our data leaves open.

6.3 Limitations

A single seed per configuration. Several limitations bound these conclusions. All results rest on a single seed per configuration on CIFAR-10 at 32×32 resolution with one U-Net backbone, so run-to-run variance is not quantified. The sub-one-point FID differences that separate most of the suite are within plausible seed noise and are read only as a direction, not as significant quality gaps. E3 reports standard deviations over 256 trajectories, but these still derive from a single trained model, so they measure trajectory spread rather than training variability. Establishing the stability of both the negative result and the small static-prior advantage would require a multi-seed study.

Compute overhead is only approximately measured. The compute overhead of the adaptive methods is analyzed directly in E1, so it is not itself a limitation, but its measurement is imperfect. Forward-pass equivalents count backbone evaluations and are hardware independent, yet they are a theoretical proxy rather than a FLOP measurement, while wall-clock time captures the real cost of the reinforcement-learning loops but depends on hardware and system load. The true overhead therefore lies somewhere between what these two complementary measures report.

Many reported quantities are estimates. The per-sample gradient variance and mean gradient norm of the backbone are micro-batch estimators, the policy-gradient variance and the Beta shape parameters are within-batch statistics logged at the update cadence, and each is marked with a hat in its definition (Section 5.1.4). A single gradient probe carries a relative standard error of about fifty percent, which we mitigate by averaging the final probes and reading orders of magnitude rather than exact per-bin values, and the batch-averaged Beta parameters are means over a batch of distinct per-image distributions rather than a single sampled density. These estimators are applied identically across strategies, so the relative comparisons hold, but the absolute values should be read with their uncertainty in mind.

Quality rests on FID alone. The measured sample quality in the experiments rests on FID alone, which makes its approximate nature worth restating. The Gaussian model underlying FID captures only the first two feature moments, its Inception-v3 embedding is trained on ImageNet and applied here to low-resolution CIFAR-10, and the score is positively biased at our $10{,}000$-sample budget (Section 5.1.4). Because the sample count and reference statistics are shared across strategies, the relative ranking remains valid while the absolute scores are not comparable to large-sample literature values. The residual FID advantage of the data-biased priors is a case in point. E3 finds the trained fields geometrically equivalent and the step-count sweep finds their FID-versus-step curves coincident, so the small FID gap is not a trajectory-geometry effect and must reside in distributional properties of the generated samples that our geometric diagnostics do not resolve. Whether that gap reflects a genuine quality difference or an artifact of FID's Gaussian, texture-biased estimator cannot be decided from FID alone. Complementing FID with a Fréchet distance computed in a self-supervised feature space such as DINOv2 (Stein et al.,

2023), with the CLIP-based maximum-mean-discrepancy CMMD (Jayasumana et al., 2024), or with a precision-and-recall separation of fidelity from diversity (Kynkäänniemi et al., 2019), would give a fuller quality picture and might account for that residual advantage.

Hyperparameters carried over from the DDPM setting. Finally, the ATS-specific hyperparameters were taken from the DDPM configuration of Kim et al. (2025a) and were not ablated again for the CondOT setting, whose velocity-regression loss differs from the ε-prediction objective they were selected for. We therefore cannot fully exclude that the failure of online adaptation reflects an unsuitable hyperparameter transfer rather than a property of the CondOT objective itself, although the near-zero reward observed across the run makes a purely hyperparameter-driven explanation unlikely.

6.4 Future Work

Isolating why the transfer fails. The findings point to several directions, each tied to a mechanism identified above. Our suite shows only that the ATS formulation of Kim et al. (2025a) does not transfer to CondOT Flow Matching, without isolating the cause. VP-path diffusion and CondOT Flow Matching differ in several respects at once, including path curvature, the loss formulation and its weighting, the ε-versus-velocity parametrization, and the deterministic-versus-stochastic sampling process. Future work could vary these factors in isolation, for instance by training Flow Matching on a VP path or with a reweighted loss, to determine which of them is responsible for the failed transfer.

Mapping the per-timestep reward landscape. In E1 we analyzed the progression of the batch mean of the reward that sets the direction and magnitude of the policy adaptation. A finer diagnosis of the reward itself, resolving it along the timestep axis to reveal which timesteps return which reward on average and how that map shifts between the early phase and the later near-stationary one, could yield additional insight into which timestep regimes are responsible for the stationary near-zero-reward phase. It would also show whether any exploitable structure survives in the per-timestep profile once the batch-mean reward has collapsed, and, if so, point toward a reward that stays informative in the regime where the loss-reduction signal falls silent.

Optimizers matched to a weak, drifting reward. If an online policy is retained despite these obstacles, the adaptation of the policy optimizer could be worth exploring. Several standard reinforcement-learning tools address the difficulty directly. Trust-region or proximal updates (Schulman et al., 2015; Schulman et al., 2017) bound the size of each policy step and so stabilize learning when the gradient estimate is noisy. Scheduling the entropy bonus over training (Williams and Peng, 1991; Mnih et al., 2016), rather than holding it fixed, preserves exploration early and lets the policy commit later instead of being pinned at the maximum-entropy uniform distribution. A parameter-free running-mean baseline (Williams, 1992; Weaver and Tao, 2001) centers the reward with an average of recent returns and avoids the lag of a learned value network, which is the failure mode we observed for A2C-ATS. None of these creates a signal where the reward has vanished, but each could let a policy extract more from the short early window in which the reward is still informative.

A sampler shaped to the CondOT Flow Matching difficulty profile. A complementary route drops the online reward altogether and reads a fixed or scheduled sampler directly off the difficulty structure that E2 characterizes. The natural targets are the two static signals E2 makes available, a sampling density that follows the gradient-variance curve, along which the timesteps differ most, or one that follows the gradient signal-to-noise ratio, which additionally accounts for the magnitude of the optimization signal and not only its variance. Because this difficulty profile is non-stationary during the first epochs and settles thereafter, neither a single fixed density nor a fully adaptive policy is likely to be optimal throughout training. The schedule could instead be divided into phases, with a middle-biased sampler while the interior is still reducible and a boundary-covering sampler for the later refinement of the data-proximal regime. A multi-phase scheme of this kind, such as the two-phase curriculum proposed by Sun (2026), could be adopted and made more targeted with the findings on CondOT Flow Matching training dynamics obtained in E2. Such a sampler would capture the exploitable structure in t at the near-zero cost of a static or precomputed schedule and would sidestep the reward-estimation overhead entirely.

Larger settings. Beyond CIFAR-10, the stationarity of the difficulty profile that E2 established need not persist. Latent-space Flow Matching, backbones with weaker inductive bias such as DiT (Peebles and Xie, 2023), and higher-resolution or more diverse data such as ImageNet (Deng et al., 2009) and FFHQ (Karras et al., 2019) all present richer per-timestep dynamics, and their larger training budgets make even a small efficiency gain worth pursuing. Such settings could revive a non-stationary component of the ATS loss-reduction reward that an online policy could track, and they are the natural place to test whether adaptivity earns its cost once the reward is redesigned. A scalability study could sharpen this last point. Because the policy network and its reward estimation are largely independent of backbone size, their overhead should shrink as a fraction of total compute as the backbone grows, so that an adaptive sampler that cannot justify its cost on a small CIFAR-10 U-Net might become comparatively cheap, and correspondingly more likely to pay off, in the larger settings where the backbone dominates the training budget.

In summary, the reinforcement-learning-based timestep sampling of ATS does not transfer from VP-path diffusion to CondOT Flow Matching in its original form, and neither the actor-critic baseline nor the curvature reward remedies that failure. The structure along the timestep axis that an adaptive sampler would exploit is real but essentially static, so a simple fixed prior captures it more cheaply than an online policy that must chase a weak and vanishing reward. These negative findings and the analyses around them, however, map the ground on which better adaptive methods can be developed in the future, whether through a redesigned reward or a difficulty-matched sampling density. Beyond adaptive timestep sampling itself, the finding that noise-biased timestep sampling is harmful to final sample quality, and the per-timestep account presented here of how Flow Matching loss and gradient variance evolve throughout training, add to the understanding of Flow Matching mechanics and support a more targeted development of future timestep-sampling methods in general. These gains matter beyond any single model. Making the training of generative models computationally cheaper not only drives the development of architectures with ever-better output fidelity, but also dampens their steadily growing energy demand and cost of building them, limiting the environmental footprint of the field and democratizing access to it.

Bibliography

Abramson, J., Adler, J., Dunger, J., Evans, R., Green, T., Pritzel, A., Ronneberger, O., Willmore, L., Ballard, A. J., Bambrick, J., Bodenstein, S. W., Evans, D. A., Hung, C.-C., O'Neill, M., Reiman, D., Tunyasuvunakool, K., Wu, Z., Žemgulytė, A., Arvaniti, E., Beattie, C., Bertolli, O., Bridgland, A., Cherepanov, A., Congreve, M., Cowen-Rivers, A. I., Cowie, A., Figurnov, M., Fuchs, F. B., Gladman, H., Jain, R., Khan, Y. A., Low, C. M. R., Perlin, K., Potapenko, A., Savy, P., Singh, S., Stecula, A., Thillaisundaram, A., Tong, C., Yakneen, S., Zhong, E. D., Zielinski, M., Žídek, A., Bapst, V., Kohli, P., Jaderberg, M., Hassabis, D., and Jumper, J. M. (June 2024). "Accurate Structure Prediction of Biomolecular Interactions with AlphaFold 3." In: *Nature* 630.8016, pp. 493–500.

Benamou, J.-D. and Brenier, Y. (2000). "A Computational Fluid Mechanics Solution to the Monge-Kantorovich Mass Transfer Problem." In: *Numerische Mathematik* 84.3, pp. 375–393.

Bińkowski, M., Sutherland, D. J., Arbel, M., and Gretton, A. (2018). "Demystifying MMD GANs." In: *International Conference on Learning Representations (ICLR)*.

Brooks, T., Peebles, B., Holmes, C., DePue, W., Guo, Y., Jing, L., Schnurr, D., Taylor, J., Luhman, T., Luhman, E., Ng, C. W. Y., Wang, R., and Ramesh, A. (2024). "Video Generation Models as World Simulators." OpenAI technical report, `https://openai.com/index/video-generation-models-as-world-simulators/`, accessed 28 July 2026.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. (2018). "Neural Ordinary Differential Equations." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 31, pp. 6571–6583.

Choi, J., Lee, J., Shin, C., Kim, S., Kim, H., and Yoon, S. (2022). "Perception Prioritized Training of Diffusion Models." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11472–11481.

Chong, M. J. and Forsyth, D. (2020). "Effectively Unbiased FID and Inception Score and Where to Find Them." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6070–6079.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). "ImageNet: A Large-Scale Hierarchical Image Database." In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255.

Dhariwal, P. and Nichol, A. (2021). “Diffusion Models Beat GANs on Image Synthesis.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 34, pp. 8780–8794.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2021). “An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale.” In: *International Conference on Learning Representations (ICLR)*.

Esser, P., Kulal, S., Blattmann, A., Entezari, R., Müller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., Boesel, F., Podell, D., Dockhorn, T., English, Z., Lacey, K., Goodwin, A., Marek, Y., and Rombach, R. (2024). “Scaling Rectified Flow Transformers for High-Resolution Image Synthesis.” In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. PMLR, pp. 12606–12633.

Garrigos, G. and Gower, R. M. (Mar. 2024). “Handbook of Convergence Theorems for (Stochastic) Gradient Methods.” arXiv: 2301.11235.

Go, H., Kim, J., Lee, Y., Lee, S., Oh, S., Moon, H., and Choi, S. (2023). “Addressing Negative Transfer in Diffusion Models.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36, pp. 27199–27222.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). “Generative Adversarial Nets.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 27, pp. 2672–2680.

Hang, T., Gu, S., Li, C., Bao, J., Chen, D., Hu, H., Geng, X., and Guo, B. (2023). “Efficient Diffusion Training via Min-SNR Weighting Strategy.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 7441–7451.

Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. (2017). “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30, pp. 6626–6637.

Ho, J., Jain, A., and Abbeel, P. (2020). “Denoising Diffusion Probabilistic Models.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33, pp. 6840–6851.

Holderrieth, P. and Erives, E. (June 2025). “An Introduction to Flow Matching and Diffusion Models.” arXiv: 2506.02070.

Iserles, A. (2009). *A First Course in the Numerical Analysis of Differential Equations*. 2nd ed. Cambridge University Press.

Jayasumana, S., Ramalingam, S., Veit, A., Glasner, D., Chakrabarti, A., and Kumar, S. (2024). “Rethinking FID: Towards a Better Evaluation Metric for Image Generation.” In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9307–9315.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). "Scaling Laws for Neural Language Models." arXiv: 2001.08361.

Karras, T., Aittala, M., Aila, T., and Laine, S. (2022). "Elucidating the Design Space of Diffusion-Based Generative Models." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35, pp. 26565–26577.

Karras, T., Laine, S., and Aila, T. (2019). "A Style-Based Generator Architecture for Generative Adversarial Networks." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4401–4410.

Kim, M., Ki, D., Shim, S.-W., and Lee, B.-J. (2025a). "Adaptive Non-Uniform Timestep Sampling for Accelerating Diffusion Model Training." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2513–2522.

Kim, M., Ki, D., Shim, S.-W., and Lee, B.-J. (2025b). "Adaptive-Timestep-Sampler." GitHub repository, commit 4b2eae4 (22 May 2025), https://github.com/ku-dmlab/Adaptive-Timestep-Sampler/tree/4b2eae4, accessed 28 July 2026. Reference implementation accompanying the ATS paper.

Kingma, D. P. and Ba, J. (2015). "Adam: A Method for Stochastic Optimization." In: *International Conference on Learning Representations (ICLR)*.

Kingma, D. P. and Welling, M. (2014). "Auto-Encoding Variational Bayes." In: *International Conference on Learning Representations (ICLR)*.

Konda, V. and Tsitsiklis, J. (1999). "Actor-Critic Algorithms." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 12, pp. 1008–1014.

Krizhevsky, A. (2009). "Learning Multiple Layers of Features from Tiny Images." Technical Report. University of Toronto. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

Kynkäänniemi, T., Karras, T., Laine, S., Lehtinen, J., and Aila, T. (2019). "Improved Precision and Recall Metric for Assessing Generative Models." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32, pp. 3927–3936.

LeCun, Y., Boser, B., Denker, J., Henderson, D., Howard, R., Hubbard, W., and Jackel, L. (1989). "Handwritten Digit Recognition with a Back-Propagation Network." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 2, pp. 396–404.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). "Gradient-Based Learning Applied to Document Recognition." In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Li, S., Zhao, Y., Varma, R., Salpekar, O., Noordhuis, P., Li, T., Paszke, A., Smith, J., Vaughan, B., Damania, P., and Chintala, S. (2020). "PyTorch Distributed: Experiences on Accelerating Data Parallel Training." In: *Proceedings of the VLDB Endowment* 13.12, pp. 3005–3018.

Li, Y., Jiang, H., Kodaira, A., Liang, F., Kondratyuk, D., Tomizuka, M., Keutzer, K., and Xu, C. (2024). "Immiscible Diffusion: Accelerating Diffusion Training with Noise Assignment." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 37, pp. 90198–90225.

Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. (2023). "Flow Matching for Generative Modeling." In: *International Conference on Learning Representations (ICLR)*.

Liu, X., Gong, C., and Liu, Q. (2023). "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow." In: *International Conference on Learning Representations (ICLR)*.

McCandlish, S., Kaplan, J., Amodei, D., and OpenAI Dota Team (2018). "An Empirical Model of Large-Batch Training." arXiv: 1812.06162.

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., Silver, D., and Kavukcuoglu, K. (2016). "Asynchronous Methods for Deep Reinforcement Learning." In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. Vol. 48. PMLR, pp. 1928–1937.

Peebles, W. and Xie, S. (2023). "Scalable Diffusion Models with Transformers." In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4195–4205.

Polyak, A., Zohar, A., Brown, A., Chuang, C.-Y., Yan, D., Choudhary, D., Misra, I., Wang, J., Jagadeesh, K., Li, K., et al. (2024). "Movie Gen: A Cast of Media Foundation Models." arXiv: 2410.13720.

Pooladian, A.-A., Ben-Hamu, H., Domingo-Enrich, C., Amos, B., Lipman, Y., and Chen, R. T. Q. (2023). "Multisample Flow Matching: Straightening Flows with Minibatch Couplings." In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Vol. 202. PMLR, pp. 28100–28127.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. (2022). "High-Resolution Image Synthesis with Latent Diffusion Models." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695.

Ronneberger, O., Fischer, P., and Brox, T. (2015). "U-Net: Convolutional Networks for Biomedical Image Segmentation." In: *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Springer, pp. 234–241.

Schulman, J., Levine, S., Moritz, P., Jordan, M. I., and Abbeel, P. (2015). "Trust Region Policy Optimization." In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Vol. 37. PMLR, pp. 1889–1897.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (Aug. 2017). "Proximal Policy Optimization Algorithms." arXiv: 1707.06347.

Sohl-Dickstein, J., Weiss, E. A., Maheswaranathan, N., and Ganguli, S. (2015). "Deep Unsupervised Learning Using Nonequilibrium Thermodynamics." In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Vol. 37. PMLR, pp. 2256–2265.

Song, J., Meng, C., and Ermon, S. (2021a). "Denoising Diffusion Implicit Models." In: *International Conference on Learning Representations (ICLR)*.

Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2021b). "Score-Based Generative Modeling through Stochastic Differential Equations." In: *International Conference on Learning Representations (ICLR)*.

Stein, G., Cresswell, J. C., Hosseinzadeh, R., Sui, Y., Ross, B. L., Villecroze, V., Liu, Z., Caterini, A. L., Taylor, J. E. T., and Loaiza-Ganem, G. (2023). "Exposing Flaws of Generative Model Evaluation Metrics and Their Unfair Treatment of Diffusion Models." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36, pp. 3732–3784.

Sun, P. (Mar. 2026). "Curriculum Sampling: A Two-Phase Curriculum for Efficient Training of Flow Matching." 2nd DeLTa Workshop, ICLR 2026. arXiv: 2603.12517.

Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Adaptive Computation and Machine Learning. Cambridge, MA: The MIT Press.

Tong, A., Fatras, K., Malkin, N., Huguet, G., Zhang, Y., Rector-Brooks, J., Wolf, G., and Bengio, Y. (2024). "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport." In: *Transactions on Machine Learning Research (TMLR)*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). "Attention Is All You Need." In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30, pp. 5998–6008.

Weaver, L. and Tao, N. (2001). "The Optimal Reward Baseline for Gradient-Based Reinforcement Learning." In: *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence (UAI)*. Morgan Kaufmann Publishers Inc., pp. 538–545.

Williams, R. J. (May 1992). "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." In: *Machine Learning* 8.3–4, pp. 229–256.

Williams, R. J. and Peng, J. (Jan. 1991). "Function Optimization Using Connectionist Reinforcement Learning Algorithms." In: *Connection Science* 3.3, pp. 241–268.

Xu, Y., Tong, S., and Jaakkola, T. (2023). "Stable Target Field for Reduced Variance Score Estimation in Diffusion Models." In: *International Conference on Learning Representations (ICLR)*.

Yoon, J. and Lee, J. (Oct. 2024). "Augmented Flow Matching via Variance Reduction with Auxiliary Variables." OpenReview preprint, https://openreview.net/forum?id=z9CCkjVY0h, accessed 28 July 2026.

Yu, H., Shen, L., Huang, J., Li, H., and Zhao, F. (2024). "Unmasking Bias in Diffusion Model Training." In: *Proceedings of the European Conference on Computer Vision (ECCV)*. Vol. 15124. Lecture Notes in Computer Science. Springer, pp. 374–390.

Zheng, T., Geng, C., Jiang, P.-T., Wan, B., Zhang, H., Chen, J., Wang, J., and Li, B. (2024a). "Non-Uniform Timestep Sampling: Towards Faster Diffusion Model Training." In: *Proceedings*

of the 32nd ACM International Conference on Multimedia. MM '24. Melbourne VIC, Australia: Association for Computing Machinery, pp. 7036–7045.

Zheng, T., Jiang, P.-T., Wan, B., Zhang, H., Chen, J., Wang, J., and Li, B. (2024b). "Beta-Tuned Timestep Diffusion Model." In: *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, pp. 114–130.

A Appendix

A.1 Discrepancy in ATS Entropy Regularization

During the implementation and theoretical review of the ATS framework by Kim et al. (2025a), a fundamental discrepancy was identified regarding the implementation of entropy regularization. This appendix details how an analytically sound continuous objective translates into a flawed discrete surrogate loss in PyTorch, analyzing its detrimental injection of gradient variance, and contrasting it with the mathematically rigorous approach utilized in this thesis.

A.1.1 The Analytical Policy Gradient

To identify the discrepancy, we first establish the exact mathematical objective. Kim et al. (2025a) treat the analytical differential entropy of the predicted Beta distribution, $\mathcal{H}[\pi_\phi(\cdot|x_1)]$, as an additive scalar bonus to the environmental reward. For a given data sample x_1, this forms an effective reward formulated as

$$\mathcal{R}_{\text{eff}}(t, x_1) = \tilde{\Delta}_k^t(x_1, x_0) + \eta \cdot \mathcal{H}[\pi_\phi(\cdot|x_1)]. \tag{A.1}$$

The loss reduction proxy $\tilde{\Delta}_k^t(x_1, x_0)$ depends on the input image x_1, the sampled prior noise $x_0 \sim \mathcal{N}(0, I)$, and the sampled timestep t. It acts as an environmental signal and is detached from the policy parameters ϕ.

The continuous mathematical objective is to maximize the expected value of this effective reward over the independent data distribution $x_1 \sim q(x_1)$, the prior noise $x_0 \sim p(x_0)$, and the stochastic policy $t \sim \pi_\phi(\cdot|x_1)$, yielding the objective function

$$J_{\text{Kim}}(\phi) = \mathbb{E}_{x_1, x_0, t}\Big[\tilde{\Delta}_k^t(x_1, x_0) + \eta\mathcal{H}[\pi_\phi(\cdot|x_1)]\Big]. \tag{A.2}$$

When evaluating the analytical gradient of this expected reward, we apply the Leibniz integral rule to pull the gradient operator inside the expectation over x_1 and x_0. Because the analytical entropy $\mathcal{H}[\pi_\phi]$ is inherently parameterized by ϕ, we must apply the product rule when differentiating the policy-weighted integral:

$$\begin{aligned}\nabla_\phi J_{\text{Kim}}(\phi) &= \mathbb{E}_{x_1, x_0}\left[\int \nabla_\phi\Big(\pi_\phi(t|x_1)\big(\tilde{\Delta}_k^t + \eta\mathcal{H}[\pi_\phi]\big)\Big)dt\right] \\ &= \mathbb{E}_{x_1, x_0}\left[\int \Big(\nabla_\phi\pi_\phi(t|x_1)\big(\tilde{\Delta}_k^t + \eta\mathcal{H}[\pi_\phi]\big) + \pi_\phi(t|x_1)\big(0 + \eta\nabla_\phi\mathcal{H}[\pi_\phi]\big)\Big)dt\right].\end{aligned} \tag{A.3}$$

Applying the log-derivative trick ($\nabla\pi = \pi\nabla\log\pi$) to the first term and distributing the integral yields

$$\nabla_\phi J_{\text{Kim}}(\phi) = \mathbb{E}_{x_1,x_0,t}\Big[\big(\tilde{\Delta}_k^t + \eta\mathcal{H}[\pi_\phi]\big)\nabla_\phi \log \pi_\phi(t|x_1)\Big] + \mathbb{E}_{x_1,x_0,t}\Big[\eta\nabla_\phi\mathcal{H}[\pi_\phi]\Big]. \tag{A.4}$$

Because the analytical entropy does not depend on the specific sampled t, we can factor it out of the inner expectation. Knowing that the expected score function evaluates strictly to zero ($\mathbb{E}_t[\nabla_\phi \log \pi_\phi] = 0$), the entropy component of the first term vanishes perfectly:

$$\begin{aligned}\nabla_\phi J_{\text{Kim}}(\phi) &= \mathbb{E}_{x_1,x_0,t}\Big[\tilde{\Delta}_k^t \nabla_\phi \log \pi_\phi(t|x_1)\Big] \\ &\quad + \eta\mathbb{E}_{x_1}\Bigg[\mathcal{H}[\pi_\phi]\underbrace{\mathbb{E}_t\big[\nabla_\phi \log \pi_\phi(t|x_1)\big]}_{=0}\Bigg] + \eta\mathbb{E}_{x_1}\Big[\nabla_\phi\mathcal{H}[\pi_\phi]\Big] \\ &= \mathbb{E}_{x_1,x_0,t}\Big[\tilde{\Delta}_k^t \nabla_\phi \log \pi_\phi(t|x_1)\Big] + \eta\mathbb{E}_{x_1}\Big[\nabla_\phi\mathcal{H}[\pi_\phi(\cdot|x_1)]\Big].\end{aligned} \tag{A.5}$$

This mathematical derivation reveals a critical insight: analytically, treating entropy as a reward component yields a gradient that is strictly identical to the analytical gradient of the standard, decoupled additive formulation. Therefore, in continuous mathematical theory, the intended objective of Kim et al. (2025a) is perfectly sound and would successfully regularize the policy.

A.1.2 Automatic Differentiation and the Surrogate Loss Discrepancy

However, in practice, deep learning frameworks do not evaluate continuous integrals. They perform SGD via automatic differentiation (Autograd) on a discrete surrogate loss function. The gradient is estimated over a sampled batch of size B, forming independent tuples $(x_1^{(i)}, x_0^{(i)}, t^{(i)})$.

The REINFORCE log-derivative trick relies on the mathematical property

$$\nabla_\phi\mathbb{E}[R] = \mathbb{E}[R\,\nabla_\phi \log \pi_\phi]\,. \tag{A.6}$$

To compute this stochastically in an auto-differentiation graph, the standard practice is to define a surrogate loss $L = -R\log\pi_\phi$. Crucially, this trick only produces the correct analytical gradient if the reward R is treated as a detached, constant scalar ($\nabla_\phi R = 0$). If R depends on ϕ, the expected gradient of the surrogate loss will fundamentally fail to match the continuous analytical target.

As shown in Listing 1, the official ATS implementation (Kim et al., 2025b) directly couples the analytical entropy into the empirical REINFORCE reward *before* defining the surrogate loss.

By defining their reward as $R_{\text{eff}} = \tilde{\Delta}_k^t + \eta\mathcal{H}[\pi_\phi]$ and substituting it directly into the REINFORCE surrogate template, the authors violate this core mathematical assumption. For the batch of B samples, PyTorch constructs the following surrogate objective to minimize:

$$\hat{L}_{\text{Kim}}(\phi) = -\frac{1}{B}\sum_{i=1}^{B}\log\pi_\phi(t^{(i)}|x_1^{(i)}) \cdot \Big(\tilde{\Delta}_k^{t^{(i)}} + \eta\mathcal{H}[\pi_\phi(\cdot|x_1^{(i)})]\Big). \tag{A.7}$$

```python
def sample_timesteps(self, x):
    alpha, beta = self.policy(x)
    dist = Beta(alpha.squeeze(), beta.squeeze())
    dist_sampled = dist.sample()

    # 1. Calculate the analytical differential entropy H[\pi]
    log_probs = dist.log_prob(dist_sampled)
    entropy = dist.entropy()

    return timesteps.detach(), log_probs, ..., entropy, dist_sampled


...
# Inside the Policy Update step
if i % self.update_policy == 0:
    reward = kl_diff_sum.detach()

    # 2. DISCREPANCY: Entropy is added directly to the environmental reward
    reward += self.ent_coef * entropy

    # 3. REINFORCE calculation multiplies log_probs by the inflated reward
    actor_loss = -(log_probs * reward).mean()
    actor_loss.backward()
```

Listing 1: **Original ATS Policy Update Implementation.** Code extracted from the official ATS repository (Kim et al., 2025b), demonstrating the structural coupling of the analytical entropy into the empirical reward signal.

When `actor_loss.backward()` is executed, PyTorch mechanically applies the product rule to each individual sample i in the computational graph. Because $\tilde{\Delta}_k^{t^{(i)}}$ is detached, it carries no gradient. However, because the analytical entropy $\mathcal{H}$ depends on the parameters ϕ, it maintains an active path, generating the single-sample Autograd derivative

$$\hat{g}_{\text{Kim}}^{(i)}(\phi) = \Big(\tilde{\Delta}_k^{t^{(i)}} + \eta\mathcal{H}[\pi_\phi]\Big)\nabla_\phi \log \pi_\phi(t^{(i)}) + \eta \log \pi_\phi(t^{(i)})\nabla_\phi \mathcal{H}[\pi_\phi]. \tag{A.8}$$

In contrast, the standard, decoupled additive formulation constructs the surrogate loss as

$$\hat{L}_{\text{Std}}(\phi) = -\frac{1}{B}\sum_{i=1}^{B}\Big(\tilde{\Delta}_k^{t^{(i)}} \cdot \log \pi_\phi(t^{(i)}|x_1^{(i)}) + \eta\mathcal{H}[\pi_\phi(\cdot|x_1^{(i)})]\Big), \tag{A.9}$$

which natively separates the differentiation paths, resulting in the correct single-sample gradient estimate

$$\hat{g}_{\text{Std}}^{(i)}(\phi) = \tilde{\Delta}_k^{t^{(i)}}\nabla_\phi \log \pi_\phi(t^{(i)}) + \eta\nabla_\phi \mathcal{H}[\pi_\phi]. \tag{A.10}$$

Figure A.1 contrasts the two computational graphs and visualizes how the coupled formulation routes the entropy gradient through the product rule, whereas the decoupled formulation provides it a clean, independent path.

A.1.3 Expected SGD Policy Gradient and Structural Corruption

By the Law of Large Numbers, the Monte Carlo batch average computed by PyTorch converges to the expected value of the single-sample gradient ($\frac{1}{B}\sum \hat{g}^{(i)} \approx \mathbb{E}[\hat{g}^{(i)}]$). Evaluating the true expectation of the Kim et al. (2025a) computational artifact yields

$$\begin{aligned}\mathbb{E}_{x_1,x_0,t}[\hat{g}_{\text{Kim}}^{(i)}] = \mathbb{E}_{x_1,x_0,t}\Big[\tilde{\Delta}_k^t\nabla_\phi \log \pi_\phi\Big] + \eta\mathbb{E}_{x_1}\Bigg[\mathcal{H}\underbrace{\mathbb{E}_t[\nabla_\phi \log \pi_\phi]}_{=0}\Bigg] \\ + \eta\mathbb{E}_{x_1}\Bigg[\nabla_\phi\mathcal{H}\underbrace{\mathbb{E}_t[\log \pi_\phi]}_{=-\mathcal{H}}\Bigg].\end{aligned} \tag{A.11}$$

By fundamental information theory, the expectation of the log-probability is the negative differential entropy ($\mathbb{E}_t[\log \pi_\phi] = -\mathcal{H}[\pi_\phi]$). Consequently, the expected PyTorch gradient collapses to

$$\mathbb{E}_{x_1,x_0,t}[\hat{g}_{\text{Kim}}^{(i)}] = \mathbb{E}_{x_1,x_0,t}\Big[\tilde{\Delta}_k^t\nabla_\phi \log \pi_\phi\Big] - \eta\mathbb{E}_{x_1}\Big[\mathcal{H}[\pi_\phi]\nabla_\phi\mathcal{H}[\pi_\phi]\Big]. \tag{A.12}$$

Because the differential entropy of a continuous Beta distribution bounded on $[0, 1]$ is strictly non-positive ($\mathcal{H} \leq 0$, peaking at 0 for the uniform distribution), we substitute $\mathcal{H} = -|\mathcal{H}|$, revealing the effective update direction:

$$\mathbb{E}_{x_1,x_0,t}[\hat{g}_{\text{Kim}}^{(i)}] = \mathbb{E}_{x_1,x_0,t}\Big[\tilde{\Delta}_k^t\nabla_\phi \log \pi_\phi\Big] + \eta\mathbb{E}_{x_1}\Big[|\mathcal{H}[\pi_\phi]|\nabla_\phi\mathcal{H}[\pi_\phi]\Big]. \tag{A.13}$$

While this PyTorch implementation points in the mathematically correct average direction (pushing toward higher entropy), its magnitude is dynamically scaled by the absolute entropy

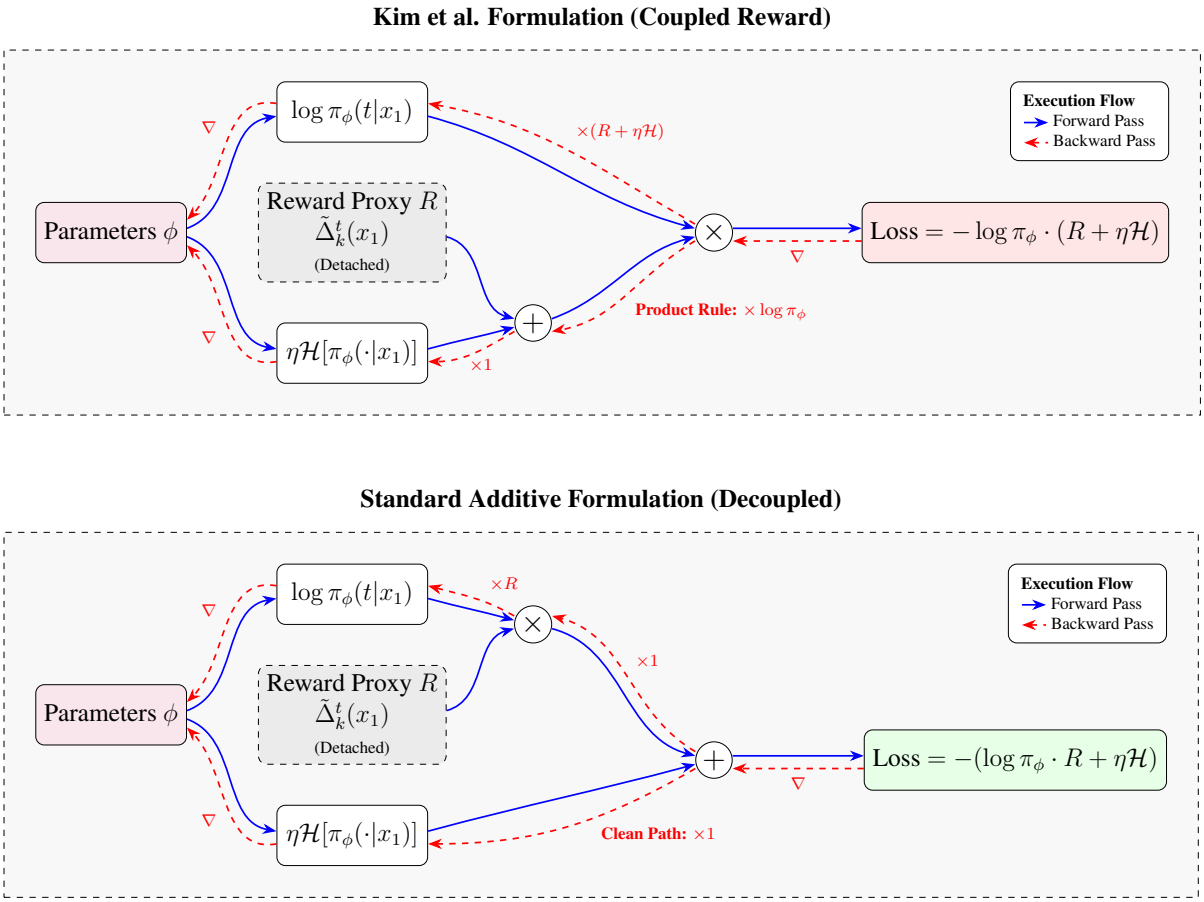


Figure A.1: **Computational Graph and Autograd Flow Comparison.** Blue solid arrows denote the forward pass, while dashed red arrows denote the flow of gradients during backpropagation. The isolated reward proxy acts as a detached environmental signal. In the Kim et al. (2025b) formulation (top), coupling entropy directly into the reward forces Autograd to apply the product rule, creating an unintentional gradient scaling where $\nabla\mathcal{H}$ is multiplied by $\log \pi$. Separating them additively in the standard formulation (bottom) provides a clean path for the entropy gradient completely independent of the stochastic log-probability.

$|\mathcal{H}[\pi_\phi]|$. This introduces theoretical instabilities into the policy adaptation process. As the policy successfully approaches the uniform target, $|\mathcal{H}| \to 0$, which prematurely extinguishes the regularization signal entirely before the ideal distribution is reached. Conversely, if the policy collapses into a near-deterministic spike, $|\mathcal{H}| \to \infty$ and the expected gradients numerically explode.

A.1.4 Empirical Analysis of Implications of Coupled Entropy on Policy Adaptation

To empirically verify how the coupling of the entropy $\mathcal{H}[\pi_\phi]$ with the log probabilities $\log \pi_\phi$ impacts the Flow Matching training with ATS in practice, we train two identical CondOT Flow Matching models with the standard ATS method under the same setup as described in Section 5.1, but only for 100 epochs. For the first model we employ the original coupled entropy formulation (Kim et al., 2025b) and for the second model we use the decoupled formulation (Mnih et al., 2016); for both runs we additionally log entropy-related metrics.

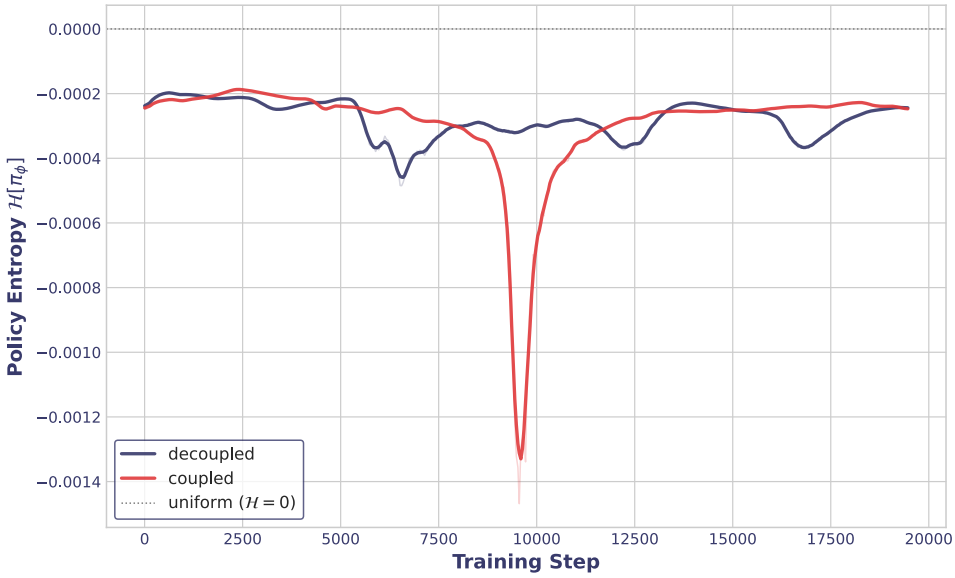


Figure A.2: **Policy entropy $\mathcal{H}[\pi_\phi]$ during training for the decoupled and coupled entropy formulations (CondOT-FM with ATS policy, Experimental setup of Section 5.1).** Under the coupled entropy formulation, the policy rapidly drifts away from a near-uniform distribution around step 9500. Because the $|\mathcal{H}|$-gated regularization remains weak, it fails to directly counter this policy drift, resulting in a severe negative spike in entropy. In contrast, the decoupled formulation maintains a distribution much closer to uniform, indicating a more robust and consistent entropy regularization.

Writing $\mathcal{H} = \mathcal{H}[\pi_\phi]$ for the policy entropy and η for the entropy coefficient, we isolate the entropy penalty term $P_\mathcal{H}$ in the actor loss and its gradient norm $g_\mathcal{H} = \|\nabla_\phi P_\mathcal{H}\|_2$. For the decoupled loss the entropy is an additive bonus, $P_\mathcal{H} = -\eta\,\overline{\mathcal{H}}$, whereas for the coupled loss it enters through the reward, $P_\mathcal{H} = -\eta\,\overline{\log \pi_\phi\,\mathcal{H}}$, where $\overline{(\cdot)}$ denotes the batch mean.

Both runs reach essentially the same image quality (Table A.1), with a final FID of 22.08 versus

22.43 and a validation loss of 0.173 versus 0.174. The difference instead lies in the regularization itself. Since the adaptive policies stay close to uniform in both cases, $|\mathcal{H}|$ remains far below one throughout training, so the coupled entropy gradient, being scaled by $|\mathcal{H}|$, is strongly attenuated: the entropy penalty term is about three orders of magnitude smaller and its impact on the final policy gradient, measured by the gradient norm, is about 190 times smaller than under the decoupled loss (Table A.1).

Table A.1: **Decoupled versus coupled (ATS) entropy regularization with standard ATS (Single run each, setup of Section 5.1).** The entropy penalty term and its gradient norm are means over the full training run, with the standard deviation in parentheses.

Quantity	Decoupled (Mnih et al., 2016)	Coupled (Kim et al., 2025b)
Final FID (↓)	22.08	22.43
Final validation loss	0.173	0.174
Entropy penalty term $P_{\mathcal{H}}$	2.8×10^{-6} (5.6×10^{-7})	1.6×10^{-9} (2.2×10^{-8})
Entropy-gradient norm $g_{\mathcal{H}}$	1.6×10^{-4} (5.1×10^{-5})	8.4×10^{-7} (2.0×10^{-6})

This instability is further evidenced by the temporal evolution of the policy entropy (Figure A.2). Under the decoupled formulation, the entropy stably remains near its uniform maximum. Conversely, the coupled objective permits the policy to diverge from uniformity near step 9500, as the attenuated, $|\mathcal{H}|$-gated regularization signal proves insufficient to counteract the policy drift.

These dynamics are in line with the behavior derived in the preceding subsection and give an idea of how the additional factor affects the entropy term. A single run can only hint at this, yet it suggests that the weakening of the coupled entropy regularization is also observable empirically. We therefore retain the standard decoupled entropy implementation.

A.2 Supplementary Experimental Material

A.2.1 Hyperparameter Configuration

Table A.2 lists the complete set of hyperparameters used across all experiments of the comparison suite (Section 5.1). These settings are shared by every timestep-sampling strategy so that any observed difference is attributable to the sampler alone. The most relevant values are also summarized in the main text (Sections 5.1.2 and 5.1.3).

Table A.2: **Complete hyperparameter configuration for the CIFAR-10 comparison suite.** The actor/critic and ATS reward/policy settings apply only to the adaptive (A2C-)ATS and LTE-ATS strategies; the ATS block is inactive for the static samplers.

Component	Hyperparameter	Value
Dataset		
	Dataset / resolution	CIFAR-10, 32×32
	Normalization	per-channel, mean/std 0.5 ($\rightarrow [-1, 1]$)
	Train / test split	$50{,}000$ / $10{,}000$
	Random seed	1234
Backbone (U-Net)		
	In / out channels	3 / 3
	Base width (channels)	128
	Channel multipliers	$(1, 2, 2, 2)$
	Residual blocks per stage	2
	Self-attention resolution	16×16 stages and 4×4 bottleneck
	Dropout	0.1
	Trainable parameters	≈ 35.7M
Actor / Critic Networks		
	Conv encoder channels	$3 \rightarrow 32 \rightarrow 64 \rightarrow 128$
	Pooled feature dim	$128 \times 4 \times 4 = 2048$
	MLP hidden dim / depth	256 / 2
	Policy initialization	$\mathrm{Beta}(1, 1) = \mathcal{U}[0, 1]$
	Trainable parameters (each)	≈ 0.68M
Optimization		
	Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
	Learning rate (backbone)	2×10^{-4}
	Learning rate (actor)	1×10^{-2} (as Kim et al. (2025a))
	Learning rate (critic)	1×10^{-3}
	Gradient clip (global ℓ_2)	1.0
	LR warmup (backbone)	5000 steps
	EMA decay	0.9999
	Precision	fp32 (no AMP)
Training Schedule / Hardware		
	Total batch size	256 (64 / GPU $\times 4$)
	Epochs	300
	GPUs	$4\times$ NVIDIA Titan X (Pascal), DDP
Flow Matching / Inference		
	Probability path	CondOT
	ODE solver / steps	uniform Euler, 50 steps
	FID samples	$10{,}000$

continued on next page

Table A.2 – continued from previous page

Component	Hyperparameter	Value
	Temporal bins for metrics tracking	10
ATS Policy Updates (Adaptive Only)		
	Policy update interval	every 40 optimizer steps
	Entropy coefficient η	1×10^{-2}
	Entropy formulation	decoupled after Mnih et al. (2016)
	Probe grid size (τ)	1000
	Features selected per update	3 (F-regression)
	Replay buffer length	5
	Stats logging frequency	every 40 steps

A.2.2 Qualitative Samples across Timestep-Sampling Strategies

Figure A.3 extends the three-strategy comparison of Figure 5.6 in Section 5.2.1 to the complete E1 suite, placing the static strategies and the adaptive methods side by side. It confirms that the FID ranking corresponds to no clearly visible difference, with the exception of the mode-collapsed LTE-ATS. This holds even at the edges of the ranking. Bernoulli (noise, 0.2) is the one strategy that FID sets apart from the field (17.34 against 13.60 to 14.62 for the remaining eight), yet we cannot reliably tell its samples apart from the others by eye, beyond an impression that they are perhaps slightly less clear.



Figure A.3: **Qualitative samples across all timestep-sampling strategies.** Ten unconditional CIFAR-10 samples (Euler, $N{=}50$) from the final-epoch ($e{=}299$) EMA model of each E1 strategy. The nine non-collapsed strategies, the seven static samplers together with ATS and A2C-ATS, are perceptually interchangeable. Seven of them, namely the data-biased and uniform priors together with logit-normal, ATS, and A2C-ATS, fall within a sub-1 FID band; the two noise-biased priors score worse (Table 5.2), Bernoulli (noise, 0.2) markedly so at 17.34, yet their samples are at most marginally less clear. LTE-ATS (bottom row) has collapsed to noise, the sample-space signature of the policy collapse diagnosed for H1.3 (Section 5.2.3).